

**UNIVERSITY OF SANTO TOMAS
ELECTRICAL ENGINEERING DEPARTMENT**

**MACHINE LEARNING–BASED SHORT-TERM FORECASTING OF
ELECTRIC VEHICLE CHARGING STATION LOAD DEMAND**

A Research Project
Submitted to the
Electrical Engineering Department
Faculty of Engineering
University of Santo Tomas

In Partial Fulfillment
of the Requirements for the Degree
Bachelor of Science in Electrical Engineering

by

JAVIER, Leister James M.
MARANTAN, Ivan Nickol D.
MATIAS, Justine B.
PINEDA, Carlos Louis P.

May 2026

UNIVERSITY OF SANTO TOMAS
ELECTRICAL ENGINEERING DEPARTMENT



APPROVAL SHEET

The Research Project entitled:

**MACHINE LEARNING-BASED SHORT-TERM FORECASTING OF
ELECTRIC VEHICLE CHARGING STATION LOAD DEMAND**

has been adequately prepared and submitted by the following proponents:

JAVIER, Leister James M.
MARANTAN, Ivan Nickol D.
MATIAS, Justine B.
PINEDA, Carlos Louis P.

has been approved and accepted as partial fulfillment of the requirements for the degree of Bachelor of Science major in Electrical Engineering.



GARA, Ariane Joyce V.
Research Project Adviser

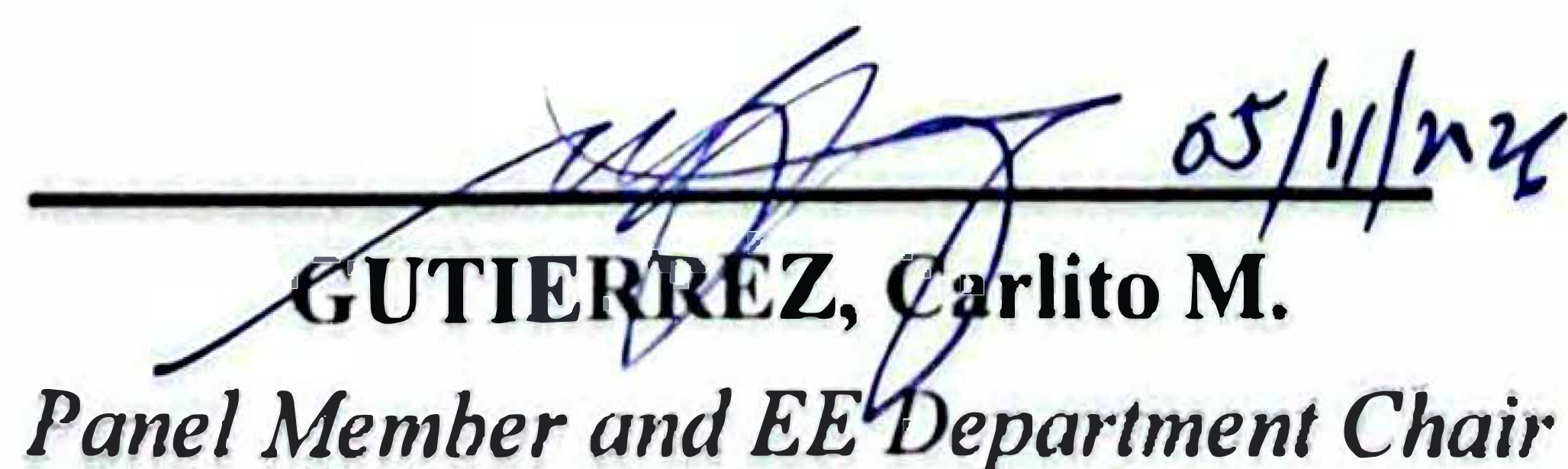
PANEL OF EXAMINERS



JORDAN, Maria Abigail A.
Panel Member



LOBO, Kevin Lester B.
Panel Member



05/11/24

GUTIERREZ, Carlito M.
Panel Member and EE Department Chair

**UNIVERSITY OF SANTO TOMAS
ELECTRICAL ENGINEERING DEPARTMENT**



ENDORSEMENT SHEET



This is an endorsement certifying that the Research Project entitled:

**MACHINE LEARNING–BASED SHORT-TERM FORECASTING OF
ELECTRIC VEHICLE CHARGING STATION LOAD DEMAND**

has been adequately prepared and submitted by the following proponents:

**JAVIER, Leister James M.
MARANTAN, Ivan Nickol D.
MATIAS, Justine B.
PINEDA, Carlos Louis P.**

as a partial fulfillment of the requirements for the degree of Bachelor of Science in
Electrical Engineering in the University of Santo Tomas Faculty of Engineering.

Endorsed by:

A handwritten signature in black ink, appearing to read "A. Gara", written over a horizontal line.

GARA, Ariane Joyce V.

Research Adviser

Date: 5/8/2026

**UNIVERSITY OF SANTO TOMAS
ELECTRICAL ENGINEERING DEPARTMENT**



CERTIFICATE OF ORIGINALITY

We hereby declare that this submission is our work and that, to the best of our knowledge and beliefs, it contains no material previously published or written by another person nor material to which a substantial extent has been accepted for award of any degree or diploma of a university or other institute of higher learning, except where due acknowledgment is made in the text.

We also declare that the intellectual content of this thesis/dissertation is the product of our work, even though we may have assistance from others on style, presentation, and language expression.

A handwritten signature in black ink, appearing to read "Javier".

JAVIER, Leister James M.

Proponent

Date: 5/8/2026

A handwritten signature in black ink, appearing to read "Marantan".

MARANTAN, Ivan Nickol D.

Proponent

Date: 5/8/2026

A handwritten signature in black ink, appearing to read "Matias".

MATIAS, Justine B.

Proponent

Date: 5/8/2026

A handwritten signature in black ink, appearing to read "Pineda".

PINEDA, Carlos Louis P.

Proponent

Date: 5/8/2026

ABSTRACT

The study developed and evaluated hybrid machine learning models, specifically SARIMA–GA–TCN and SARIMA–PSO–TCN, for short-term forecasting of electric vehicle (EV) charging station load demand. It addressed the need for accurate station-level forecasting as EV adoption increases and charging behavior becomes more variable across time, users, and locations. Existing forecasting approaches often capture either seasonal structure or nonlinear behavior, but many lack explicit residual learning, automated hyperparameter optimization, or an integrated hybrid framework. In response, the study combined the linear and seasonal modeling capability of SARIMA with the nonlinear learning capability of Temporal Convolutional Networks (TCN), while using Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) to optimize the TCN component.

The study used historical EV charging data from the City of Boulder covering January 2018 to November 2023. The raw charging-session records were cleaned, standardized, aggregated into hourly demand values, normalized using min–max scaling, and divided chronologically into training, validation, and testing sets. Using a quantitative experimental-comparative design, the study compared SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN. Model performance was evaluated using MAE, RMSE, SMAPE, and R^2 , while the Harvey–Leybourne–Newbold corrected Diebold–Mariano test assessed whether performance differences were statistically significant. The results showed that the EV charging demand series had a clear daily seasonal pattern, making it suitable for seasonal time-series modeling. SARIMA captured the linear and recurring seasonal behavior of the data, while the optimized TCN models learned the remaining nonlinear residual patterns. Both proposed hybrid models performed satisfactorily, but SARIMA–PSO–TCN showed stronger performance than SARIMA–GA–TCN on the aligned City of Boulder test dataset. However, the findings are limited to the dataset, preprocessing procedure, and model configurations used in the study. The final prediction analysis also indicated overestimation in aggregated views, suggesting bias correction, validation using other datasets, additional explanatory variables, and alternative forecasting configurations for future work.

Keywords: *Electric Vehicle, Load Demand, Machine Learning, SARIMA, GA, PSO, TCN*

TABLE OF CONTENTS

APPROVAL SHEET.....	ii
ENDORSEMENT SHEET.....	iii
CERTIFICATE OF ORIGINALITY.....	iv
ABSTRACT.....	v
TABLE OF CONTENTS.....	vi
LIST OF FIGURES.....	ix
LIST OF TABLES.....	x
NOMENCLATURE.....	xi
CHAPTER 1.0. INTRODUCTION.....	1
1.1. Background of the study.....	1
1.2. Statement of the Problem.....	3
1.3. Objectives of the Study.....	4
1.4. Significance of the study.....	5
1.5. Scope and Limitation.....	5
1.6. Framework of the study.....	7
1.7. Definition of Terms.....	8
CHAPTER 2.0. REVIEW OF RELATED LITERATURE.....	10
2.1. Overview of the chapter.....	10
2.2. Overview of EV Adoption Trends and Charging Technologies.....	10
2.3. Grid Impacts of EV Charging Integration.....	11
2.4. How Machine Learning Works.....	13
2.4.1 Normalization and Scaling Techniques.....	14
2.5. Statistical Forecasting Models.....	16
2.5.1. Auto Regressive Integrated Moving Average (ARIMA).....	16
2.5.2. Seasonal Auto Regressive Integrated Moving Averaging (SARIMA).....	17
2.5.3. Application of SARIMA.....	18
2.5.4. Background of the SARIMA Basic Coding Model.....	19
2.6. Deep Learning Forecasting Models.....	23
2.6.1. Temporal Convolutional Network.....	23
2.6.2. Convolutional Neural Network.....	24
2.6.3. Difference between CNN and TCN.....	25
2.6.4. Background of Basic TCN coding model.....	25
2.7. Optimization Model.....	29
2.7.1 Genetic Algorithm.....	29
2.7.2 Particle Swarm Optimization.....	33
2.8 Statistical Significance Testing for Forecast Model Comparison.....	39
2.9. Gap Analysis.....	42

CHAPTER 3.0. METHODOLOGY.....	47
3.1. Overview of the chapter.....	47
3.2. Research Design.....	47
3.3. Data Collection/Gathering Procedure.....	48
3.3.1. Data Gathering.....	48
3.3.2. Data Preprocessing.....	48
3.3.3. Data Cleaning.....	49
3.3.4. Timestamp Standardization.....	49
3.3.5. Temporal Aggregation.....	49
3.3.6. Feature Engineering.....	50
3.3.7. Normalization.....	50
3.3.8. Stationarity Transformation.....	52
3.3.9. Data Splitting.....	52
3.4. Instrumentation.....	52
3.4.1. Hardware Instruments.....	53
3.4.2. Software Instruments.....	53
3.4.3. Evaluation Metrics.....	56
3.5. Developed EV Charging Demand Prediction Model.....	58
3.5.1. Comparison of Genetic Algorithms and Particle Swarm Optimization.....	58
3.5.2. SARIMA-GA-TCN.....	59
3.5.3. SARIMA-PSO-TCN.....	60
3.5.4. SARIMA Model.....	61
3.5.5. TCN Model.....	62
3.5.6. Residual Generation.....	64
3.5.7. GA-TCN Model.....	68
3.5.8. PSO-TCN Model.....	69
3.5.9. Integration.....	70
3.6. Benchmarking and Comparative Analysis.....	72
3.6.3. Test Data and Preparation.....	75
3.6.4. Evaluation Criteria for Energy Load Prediction Models.....	76
3.6.5. Statistical Significance Testing of Forecast Differences.....	77
3.7. Analysis Procedure.....	82
3.7.1. Final Forecasting and Evaluation.....	82
CHAPTER 4.0. RESULTS AND DISCUSSION.....	84
4.1. Overview of the Chapter.....	84
4.2. Dataset Characteristics, Stationarity, and Seasonality Analysis.....	84
4.2.1. Dataset Characteristics Affecting Forecasting Performance.....	92
4.3. Baseline and Hybrid Forecasting Results.....	94

4.3.1 Statistical Significance of Forecasting Performance Differences.....	123
4.4. Comparative Discussion of the Proposed Hybrid Models.....	126
CHAPTER 5.0. SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS....	128
5.1. Overview of the Chapter.....	128
5.2. Summary of the Study.....	128
5.3. Summary of Findings.....	130
5.3.1. To develop a short-term EV charging load forecasting framework by preparing the dataset, modeling linear–seasonal patterns using SARIMA, and optimizing TCN-based nonlinear residual prediction through Genetic Algorithm and Particle Swarm Optimization.....	130
5.3.2. To combine the outputs of both models and evaluate performance using MAE, RMSE, SMAPE, and R ²	131
5.3.3. To analyze the advantages of combining different modeling approaches in improving forecasting performance and reliability.....	132
5.4. Conclusions.....	133
5.5. Recommendations.....	134
REFERENCES.....	137
APPENDIX 1: TIMETABLE OF RESEARCH.....	143
APPENDIX 2: OTHER NEEDED DOCUMENTS.....	144
Table B.1. Budgetary Requirements.....	144
APPENDIX 3: GRAMMARLY REPORT.....	145
Table C.1 Grammarly Report.....	145
APPENDIX 4: CODING SNIPPETS FOR FORECASTING.....	146
Appendix 4.1. : SARIMA Model Fitting.....	146
Appendix 4.2. : TCN Model Architecture.....	146
Appendix 4.3. : Genetic Algorithm Hyperparameter Optimization.....	146
Appendix 4.4. : Particle Swarm Optimization Hyperparameter Optimization.....	147
Appendix 4.5. : Hybrid Forecast Combination.....	148
APPENDIX 5: HLN-DM STATISTICAL ANALYSIS WORKBOOK.....	149
Appendix 5.1. : SARIMA vs TCN.....	149
Appendix 5.2. : TCN vs GA-TCN.....	151
Appendix 5.3. : TCN vs PSO-TCN.....	153
Appendix 5.4. : GA-TCN vs SARIMA-GA-TCN.....	155
Appendix 5.5. : PSO-TCN vs SARIMA-PSO-TCN.....	157
Appendix 5.6. : SARIMA-GA-TCN vs SARIMA-PSO-TCN.....	159
CURRICULUM VITAE.....	161

LIST OF FIGURES

Figure 1.1. Input-Process-Output Diagram.....	7
Figure 3.1. SARIMA-GA-TCN Model.....	55
Figure 3.2. SARIMA-PSO-TCN Model.....	55
Figure 3.3. TCN Model.....	63
Figure 3.4. Residual Generation Network.....	64
Figure 4.1. Charging Demand (kWh).....	84
Figure 4.2. Augmented Dickey-Fuller Test Output.....	85
Figure 4.3. Seasonal Decomposition of the EV Charging Demand Series.....	87
Figure 4.4. Autocorrelation Plot of the EV Charging Demand Series.....	89
Figure 4.5. Partial Autocorrelation Plot of the EV Charging Demand Series.....	91
Figure 4.6. Baseline Model Comparison Metrics.....	96
Figure 4.7. Forecast versus Actual Output for SARIMA, TCN, PSO-TCN, and GA-TCN..	98
Figure 4.8. Residual Comparison of SARIMA, TCN, PSO-TCN, and GA-TCN.....	99
Figure 4.9. Actual EV Load, SARIMA Forecast, PSO Residual Output, and Final Hybrid Forecast.....	103
Figure 4.10. 1 Week Forecast Panel and Error Distribution.....	105
Figure 4.11. Rolling Mean Comparison of Actual and Hybrid Forecasted EV Charging Demand Using a 24-Hour Window.....	108
Figure 4.12. Zoomed 24-Hour Rolling Mean Comparison of Actual and Hybrid Forecasted EV Charging Demand for the First 168 Hours.....	110
Figure 4.13. Contribution of the TCN Residual to the SARIMA Forecast.....	111
Figure 4.14. Zoomed Comparison of the Hybrid Forecasts and the Actual EV Load.....	113
Figure 4.15. Relative Error Comparison between the PSO-TCN and GA-TCN.....	115
Figure 4.16. Histogram of the Forecast Residuals.....	117
Figure 4.17. Correlation between Actual and Predicted EV Load.....	118
Figure 4.18. Residual Volatility and Error Consistency.....	120
Figure 4.19. Final Prediction Time Stamp using the SARIMA-PSO-TCN Hybrid.....	122

LIST OF TABLES

Table 2.1. Gap Analysis.....	42
Table 3.1. Genetic Algorithm Vs Particle Swarm Optimization.....	58
Table 3.2. Benchmark Model Configurations.....	73
Table 3.3. Standard Evaluation Metrics.....	76
Table 4.1. Baseline Model Performance Comparison.....	94
Table 4.2. Best Hyperparameters Identified for TCN.....	100
Table 4.3. Final Hybrid Performance Metrics.....	100
Table 4.4. HLN-Corrected Diebold–Mariano Test Results for Model Comparison.....	123
Table 4.5. Supporting HLN-DM Computation Values.....	125

NOMENCLATURE

Symbol	Name/Description	Units
α	Parameter controlling the rate of inertia weight decrease in AIW PSO	Unitless
α_{DM}	Significance threshold used for the HLN-DM decision	Unitless
β	Random coefficient used in continuous GA crossover	Unitless
γ_k^{DM}	Autocovariance of the loss differential at lag (k)	kWh ⁴
η_i	Learning rate for individual i (GA/PSO hyperparameter)	Unitless
μ	Mean of the actual values (R ² calculation) or constant term (ARIMA)	Varies/kWh
σ	Activation function (specifically ReLU in this study)	Unitless
θ	Set of learnable weights of the network (TCN)	Unitless
$\theta(B)$	Polynomial representing the MA component (ARMA/ARIMA)	Unitless
θ_i	Moving Average coefficient	Unitless
$\Theta^{\varphi}(B^S)$	Polynomial describing the seasonal Moving Average (MA) part	Unitless
$\varphi(B)$	Polynomial representing the AR part (ARMA/ARIMA)	Unitless
φ_i	Autoregressive coefficient	Unitless
$\Phi^P(B^S)$	Polynomial describing the seasonal Autoregressive (AR) part	Unitless
ω_t	White noise error term	Unitless

$\odot$	Element-wise multiplication operation (Dropout)	Unitless
AIC	Akaike Information Criterion used to compare model fit and complexity	Unitless
$AR(p)$	Autoregressive term in SARIMA using past data points	Unitless
B	Backshift operator	Unitless
b	Bias term (CNN/TCN)	Unitless
$b^{(l)}$	Bias term for layer l (TCN convolution)	Unitless
B_i	Batch size for individual i (GA/PSO hyperparameter)	Unitless
BIC	Bayesian Information Criterion measuring model fit with stronger penalty for complexity	Unitless
C_{HLN}^{DM}	Harvey–Leybourne–Newbold correction factor	Unitless
c_1	Cognitive coefficient (PSO)	Unitless
c_2	Social coefficient (PSO)	Unitless
<i>Chromosome</i>	GA variable representing candidate TCN parameters	Unitless
D	Seasonal differencing order in SARIMA	Unitless
d	Non-seasonal differencing order in SARIMA	Unitless
d_t^{DM}	Loss differential between Model 1 and Model 2 at aligned time index (t)	kWh^2
$\overline{d}^{DM}$	Mean loss differential across all timestamp-aligned observations	kWh^2
$d_{week}(t)$	Day type or day-of-week feature at time t	Unitless

DM	Diebold–Mariano forecast comparison test	Unitless
DM_{stat}	Diebold–Mariano statistic before HLN correction	Unitless
e	Dilation factor (TCN)	Unitless
$d_{m,t}^{DM}$	Forecast error of model (m) used in the DM/HLN-DM test	kWh
$f(\cdot)$	Nonlinear activation function (CNN)	Unitless
$f_{TCN}(\cdot)$	TCN function	Unitless
F_i	Number of filters for individual i (GA/PSO hyperparameter)	Unitless
g	Scalar parameter (scaling factor in Weight Normalization)	Unitless
GA	Genetic Algorithm used for TCN hyperparameter optimization	Unitless
g_{best}	Best position found by the entire swarm (Global Best in PSO)	Unitless
$H_{0,DM}$	Null hypothesis: the two compared models have equal predictive accuracy	Unitless
$H_{1,DM}$	Alternative hypothesis: the two compared models have significantly different accuracy	Unitless
h	Vector of neuron activations in a layer (Dropout)	Unitless
$\tilde{h}$	Resulting vector containing the activations after dropout has been applied	Unitless
$h(t)$	Hour-of-day feature at time t	Unitless
h_{DM}	Forecast comparison horizon used in the DM/HLN-DM test	Unitless

HLN	Harvey–Leybourne–Newbold correction applied to the DM statistic	Unitless
$HLN - DM$	Harvey–Leybourne–Newbold corrected Diebold–Mariano test	Unitless
$HLN - DM_{stat}$	HLN-corrected Diebold–Mariano statistic	Unitless
I	Integrated or differencing component in ARIMA and SARIMA	Unitless
k	Kernel size in TCN filters	Unitless
k'	Number of parameters of ARIMA/SARIMA method	Unitless
K	Kernel size (number of time steps the filter looks at)	Unitless
K_i	Kernel size for individual i (GA/PSO hyperparameter)	Unitless
L	Likelihood function (of the model)	Unitless
$L_{m,t}^{DM}$	Squared error loss of model (m) used in the DM/HLN-DM test	kWh^2
$\ln(n)$	Natural logarithm of the series size n	Unitless
m	Model index (where $m=1$ is Model 1 and $m=2$ is Model 2)	Unitless
$MA(q)$	Moving Average term in SARIMA relating observations to residual errors	Unitless
MAE	Mean Absolute Error measuring average absolute prediction error	kWh
$MAPE$	Mean Absolute Percentage Error showing average percent difference between predictions and actuals	%
n	Total number of observations (used in metric calculations)	Unitless
n_{DM}	Number of timestamp-aligned observations used in the DM/HLN-DM test	Unitless

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

P	Seasonal autoregressive order in SARIMA	Unitless
p	Predefined dropout rate	Unitless
p_{DM}	p-value of the HLN-DM test	Unitless
$pbest_i$	Best position found by particle i (Personal Best in PSO)	Unitless
Q	Seasonal moving average order in SARIMA	Unitless
$r(t)$	Residual sequence generated by SARIMA at time t	kWh
$\hat{r}_{TCN}(t)$	Predicted nonlinear correction or forecast output from TCN	kWh
$\hat{r}_{GA-TCN}(t)$	Nonlinear forecast output from GA-TCN	kWh
r_1	Random value between 0 and 1 (PSO)	Unitless
r_2	Random value between 0 and 1 (PSO)	Unitless
R^2	Coefficient of Determination showing model fit quality	Unitless
$RMSE$	Root Mean Square Error showing average magnitude of prediction error	kWh
s	Length of the seasonal period	Unitless
S	Seasonal period length in SARIMA	Unitless
$SARIMA$	Seasonal ARIMA model for linear and seasonal forecasting	Unitless
SE_{DM}	Standard error of the mean loss differential used in the DM statistic	kWh^2
$SMAPE$	Symmetric Mean Absolute Percentage Error showing balanced percent error	%
t	Current time step or iteration	Unitless

t_{max}	Maximum number of iterations or epochs (AIW PSO)	Unitless
TCN	Temporal Convolutional Network for nonlinear time-series forecasting	Unitless
v	Vector representing the direction of the weights (learned)	Unitless
v_i	Velocity of particle i (PSO)	Unitless
w	Inertia weight (PSO)	Unitless
W	Kernel weights (CNN)	Unitless
w'	Actual weight vector used in the model (Weight Normalization)	Unitless
$w(i)$	Weight of the convolution filter at position i	Unitless
w_i	Weight of the TCN filter	Unitless
$w_k^{(l)}$	Convolutional weights for kernel of size K in layer l	Unitless
w_{max}	Maximum inertia weight (AIW PSO)	Unitless
w_{min}	Minimum inertia weight (AIW PSO)	Unitless
X_t	Observed time series	kWh
X_i	Hyperparameter set (chromosome) for individual i	Unitless
x_i	Original value of feature i	Varies
x_i^*	Normalized value of x_i	Unitless
x_{max}	Maximum value of the feature	Varies
x_{min}	Minimum value of the feature	Varies

x_i	Position of particle i (PSO)	Unitless
$x^{(l)}(t)$	Input to layer l at time t (TCN convolution)	Varies
$x^{(l+1)}(t)$	Output of layer l+1 at time t (TCN convolution)	Varies
y	Actual EV charging load value	kWh
y_t^{DM}	Actual EV charging demand used in the DM/HLN-DM test at index (t)	kWh
$\hat{y}_{m,t}^{DM}$	Forecasted EV charging demand of model (m) at aligned time index (t)	kWh
$\hat{y}_i$	Predicted value (used in metric calculations)	kWh
$\hat{y}_{final(t)}$	Final predicted EV charging load	kWh
$\hat{y}_{SARIMA(t)}$	Linear forecast output from SARIMA	kWh
y_t	Observed time series value or output at time t	kWh
y_{t-i}	Past data point or lagged observed time series	kWh
$\ v\ $	Euclidean norm (magnitude) of v	Unitless

CHAPTER 1.0. INTRODUCTION

1.1. Background of the study

The adoption of electric vehicles (EVs) is increasing rapidly worldwide, driving significant changes in energy distribution networks. These advancements have caused a sudden increase in charging sessions and an increase in variability of charging loads across different infrastructures. New challenges are rising in modern power systems due to these advancements. Accurate short-term and station-level forecasting of EV charging demand is essential for capacity planning, efficient grid management, and maintaining energy reliability [1].

Forecasting EV charging load is more complex than predicting conventional household electricity consumption because it depends on individual charging sessions influenced by multiple contextual factors.

Space and time affect the behavior when charging. In order to operate at different time sessions that can vary from minutes, hours, or even days, forecasting models are required [1]. Detailed charging patterns are captured from high-resolution data but may introduce noise and sparsity, while aggregated data reduces variability but risks obscuring session-level dynamics. Distinct EV user charging behavior adds a level of uncertainty, influenced by various factors such as arrival time, remaining battery capacity, and charging duration. Other external factors, such as time of day and a specific day of the week, also influence charging activity and are considered in forecasting different prediction models [2] - [6].

Machine learning (ML) has emerged as a well-known approach for predicting EV charging demand, as it can analyze large datasets and identify complex nonlinear patterns [1], [7], [8]. ML-based forecasting typically involves data collection, preprocessing, feature selection, model training, validation, testing, and prediction. Preprocessing the dataset involves the conversion of irregular session-based data into continuous time series data. This helps the model effectively identify variations and changes in EV charging behavior; such methods enhance model efficiency, which promotes data-driven management of EV charging infrastructure [7].

EV charging load data exhibit linear seasonal trends—such as daily and weekly peaks—and nonlinear fluctuations. Traditional statistical methods, like the Seasonal Autoregressive Integrated Moving Average (SARIMA) model, effectively capture linear and recurring seasonal patterns in time series [9], [10]. SARIMA has been applied to model daily and weekly charging cycles in EV demand forecasting [11]. However, these methods cannot fully account for nonlinear structures present in the data. Deep learning architectures, such as the Temporal Convolutional Network (TCN), have been proposed to address these limitations. TCNs utilize causal and dilated convolutions to efficiently model short and long-term temporal dependencies [12], [10].

Despite advances in modeling, challenges remain in integrating decomposition techniques, deep learning architectures, and automated optimization for EV load forecasting [27]–[30]. This study developed a hybrid Seasonal Autoregressive Integrated Moving Average–Genetic Algorithm–Temporal Convolutional Network (SARIMA–GA–TCN) and Seasonal Autoregressive Integrated Moving Average–Particle Swarm Optimization–Temporal Convolutional Network (SARIMA–PSO–TCN)

framework to address these gaps. In this approach, SARIMA captures the linear and seasonal components of the time series and generates residuals representing nonlinear dynamics. A TCN, optimized by Genetic Algorithm (GA) or Particle Swarm Optimization, models these residuals to capture complex temporal dependencies. The GA optimization model, inspired by evolutionary principles, optimizes parameters such as the number of filters, kernel size, learning rate, and batch size for the TCN, supporting better prediction of nonlinear residuals [10], [13], [14]. The final forecast combines the linear SARIMA output with the nonlinear correction from the TCN, providing a more adaptive and comprehensive prediction framework [10], [27]–[30].

The models: SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN were evaluated using standard forecasting metrics [12], [10], [5]. By integrating statistical modeling, optimization, and deep learning, this study aimed to develop a tool capable of predicting accurate short-term forecasts of EV charging load demand, supporting EV infrastructure management and operation.

1.2. Statement of the Problem

The growing adoption of electric vehicles (EVs) presents opportunities and challenges for the distribution network. While EVs support sustainable transportation, the charging demand can introduce new stresses on the system. Issues such as voltage fluctuations, harmonic distortion, and load imbalances may arise if charging patterns are not correctly forecasted and managed. To address this, predictive modeling provides a tool to anticipate demand behavior and assess its potential impact on the distribution network. It allows stakeholders to prepare effective strategies for reliable and stable charging integration.

The effective integration of growing EV charging demand into the power grid is hindered by the lack of highly accurate short-term, station-level load forecasts that can reliably capture the inherent complexity and variability of charging sessions.

The study addressed the following questions:

1. How can machine learning-based forecasting models be used to forecast short-term EV charging load demand?
2. How do different machine learning model configurations, SARIMA-GA-TCN and SARIMA-PSO-TCN, perform in forecasting short-term EV charging station load demand?
3. What advantages can be observed from combining different models in terms of improving forecasting reliability?

1.3.Objectives of the Study

The general objective for this study is to develop and evaluate SARIMA-GA-TCN and SARIMA-PSO-TCN models for forecasting short-term electric vehicle charging station load demand. To achieve this, the researchers followed the following specific research objectives:

Specific Objectives:

1. To develop a short-term EV charging load forecasting framework by preparing the dataset, modeling linear-seasonal patterns using SARIMA, and optimizing TCN-based nonlinear residual prediction through Genetic Algorithm and Particle Swarm Optimization.

2. To combine the outputs of both models and evaluate performance using MAE, RMSE, SMAPE, and R^2 .
3. To analyze the advantages of combining different modeling approaches in improving forecasting performance and reliability.

1.4. Significance of the study

The researchers aim to develop a data-driven machine learning model to predict charging demand for electric vehicles (EVs). Good predictions of EV charging demand would provide valuable insights across the energy sector. This information could be helpful for several stakeholders:

To Electric Utilities, predicting charging demand helps them plan better. It allows utilities to manage load more efficiently, reduce equipment stress, and prevent overloading during peak hours. It can also guide them on where to expand charging stations or upgrade grid parts.

To EV Charging Providers, demand forecasting helps service providers run their operations more effectively. It aids in making key decisions about where to install new charging stations, how to allocate resources during peak times, and how to maintain high service quality. This results in better efficiency and improved customer satisfaction.

To Researchers, a hybrid forecasting model that combines traditional statistical analysis with machine learning can be a valuable reference for future academic work in energy forecasting, artificial intelligence applications, and innovative grid development.

1.5. Scope and Limitation

The study focused on forecasting electric vehicle (EV) charging load demand using two hybrid models: SARIMA–GA–TCN and SARIMA–PSO–TCN. These models

combine the Seasonal Autoregressive Integrated Moving Average (SARIMA) model for linear and seasonal pattern extraction with the Temporal Convolutional Network (TCN) for nonlinear residual prediction. The Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) are used to optimize the TCN's hyperparameters, enhancing learning performance and improving forecasting accuracy.

The dataset was obtained from the City of Boulder Electric Vehicle Charging Station Data, covering the period from January 2018 to November 2023. The data was chronologically divided into 80% for training, 10% for validation, and 10% for testing to ensure consistent and accurate model evaluation. The forecasting goal is to predict one week (seven steps ahead) of future charging demand using hourly data.

The SARIMA component handled the linear and seasonal dependencies, while the residuals were processed by the GA-optimized TCN and PSO-optimized TCN to capture nonlinear variations. Model performance was assessed using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Symmetric Mean Absolute Percentage Error (SMAPE), and the Coefficient of Determination (R^2), following established methods in recent forecasting studies.

This study did not cover power quality, grid operation, or energy distribution aspects. Its scope is limited to data-driven short-term forecasting and model evaluation. The main objective is to develop and compare hybrid SARIMA–GA–TCN and SARIMA–PSO–TCN models that improve the accuracy, reliability, and efficiency of EV charging demand predictions.

1.6. Framework of the study

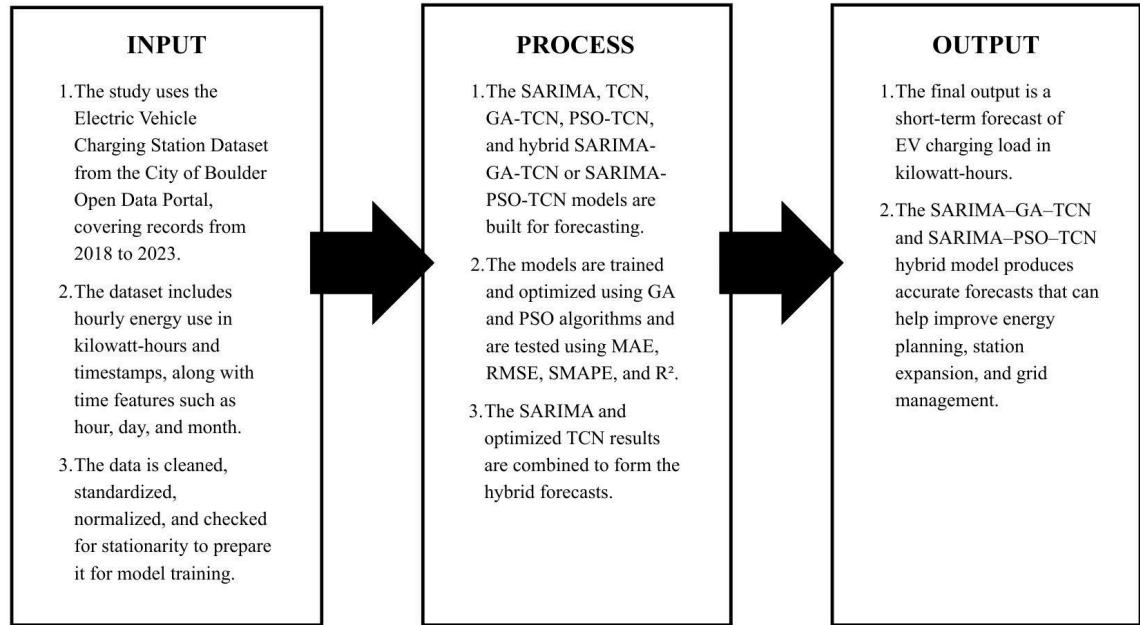


Figure 1.1. Input-Process-Output Diagram

The Input-Process-Output (IPO) model is used to summarize the forecasting process that was used in the study. The input stage includes electric vehicle (EV) charging data with timestamps, energy use, session details, and processed features such as hour of day, day of week, and other selected parameters for model training and optimization. The process stage has two main parts. The SARIMA, TCN, GA-TCN, PSO-TCN, and hybrid SARIMA-GA-TCN or SARIMA-PSO-TCN models are built for forecasting. The SARIMA model captures the linear and seasonal aspects of the data, while the GA-TCN and PSO-TCN models learn the nonlinear patterns that SARIMA cannot explain. The Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) methods fine-tune the hyperparameters of the Temporal Convolutional Network (TCN) to make learning more efficient and improve forecasting accuracy. The SARIMA and optimized TCN results are combined to form the hybrid forecasts. The final output stage

provides the predicted EV charging demand for the forecast period, measured in kilowatt-hours (kWh). The results will also show which model gives the most accurate and efficient forecasts. This combined approach will capture both linear trends and nonlinear changes for more accurate and reliable short-term prediction of EV charging behavior.

1.7. Definition of Terms

1. **Activation Function (ReLU)** – Introduces non-linearity, enabling the model to learn complex relationships.
2. **Batch Size** – Number of data samples processed before updating the model's weights.
3. **Convolution Layer (Conv1D)** – A neural network layer that detects patterns in sequential data.
4. **Dilation Rate** – Determines the spacing between elements in the kernel, allowing the model to capture long-term patterns.
5. **Dropout** – A regularization technique that temporarily ignores some neurons during training to prevent overfitting.
6. **Epochs** – Complete passes of the training dataset through the model.
7. **EV Charging Load Demand** – The total amount of electricity required by electric vehicle chargers within a certain period.
8. **Genetic Algorithm (GA)** – An optimization technique inspired by natural selection that is used to improve model parameters.
9. **Kernel / Filter** – A small matrix that slides over data to extract meaningful patterns.

10. **Loss Function (MSE)** – A metric that measures the difference between predicted and actual values.
11. **MAE, RMSE, SMAPE, R^2** – Statistical indicators used to measure forecasting accuracy.
12. **Optimizer (Adam)** – An algorithm that adjusts model weights to minimize prediction errors.
13. **Other Features Input** – Additional variables that may influence predictions, such as day-of-week.
14. **Padding** – Adds extra values to the data to control the output size of convolutional layers.
15. **Particle Swarm Optimization (PSO)** – A population-based metaheuristic method inspired by the social behavior observed in fish schools and bird flocks.
16. **Prediction / Output Layer** – The layer that produces the forecasted values from the model.
17. **Residual / Skip Connection** – A shortcut that helps preserve information across layers in deep networks.
18. **SARIMA (Seasonal Autoregressive Integrated Moving Average)** – A statistical model that captures linear and seasonal patterns in time series data.
19. **TCN (Temporal Convolutional Network)** – A deep learning model that processes sequential data and captures nonlinear patterns.
20. **Time Series Input** – Sequential data representing observations over time, used as input for forecasting models.
21. **Weight Normalization** – A method to stabilize and accelerate model training.

CHAPTER 2.0. REVIEW OF RELATED LITERATURE

2.1. Overview of the chapter

This chapter presents a comprehensive review of literature drawn from various sources. These references offer valuable insights pertinent to the study. It encompasses a range of perspectives, including both supporting evidence and counterarguments.

2.2. Overview of EV Adoption Trends and Charging Technologies

Electric vehicle (EV) adoption has increased worldwide, leading to a rapid rise in charging sessions and greater variability in charging loads across distribution networks. This growth shows the need for reliable short-term and station-level forecasts to support grid management, capacity planning, and to prevent local overloads. Recent research points out that forecasting charging demand differs from predicting household load because each session drives EV charging and depends highly on context [2].

Beyond charging technologies, considering spatial and temporal detail is essential when modeling EV demand. Data can be represented at different levels: per minute, session, hourly, or daily intervals. Finer granularity allows for more detailed modeling but brings more noise and sparse data. Aggregating data lessens noise but might obscure session-level variability. Differences in location, such as urban versus suburban areas and varying travel patterns, add to the complexity of forecasting. Recent studies have used spatio-temporal graph-based models, showing that including inter-station correlations boosts prediction accuracy [3], [5].

External factors also significantly influence charging demand. Variables such as time of day and day of the week have been added to forecasting models to capture

variability and enhance performance. Probabilistic forecasting models have mainly used these features to measure uncertainty in short-term predictions [4], [6].

The randomness and quality of charging session data make things even harder. Datasets from the real world are often incomplete and do not match up between stations. Many entries are either bad or not there at all. It is hard to predict because user behavior is unpredictable, including when they arrive, how much battery they have left, and how long they stay. Some studies have addressed these issues by employing hybrid modeling techniques or enhancing data. Others have developed probabilistic models to address uncertainties. These results show how difficult it is to predict EV charging demand and how important it is to use efficient machine learning techniques to control data-driven variability.

2.3. Grid Impacts of EV Charging Integration

As more people move toward electric vehicles (EVs), charging demand adds extra stress to the power grid, which can cause low voltage, higher power loss, and congestion in the network when left without proper management. Rana et al. [32] and Sarda et al. [33] explained that grid stability disruptions are influenced by fast charging EV chargers, causing voltage fluctuations and rising reactive power demand. The growth of charging stations was observed to have an influence on how energy flows through a system [39], calling for the need for better real-time control and overloading prevention in order to maintain stable operations.

Studies show that the most significant effects of EV charging occur in local networks, specifically in residential areas, where the number of vehicles connected

influences the overall demand for the distribution network [34] and [35]. Li and Jenna [34] also predicted that by 2045, about two-thirds of California's distribution feeders will need upgrades to meet home charging demand. These studies suggest that load shifting and charging management could help in reducing congestion.

To tackle these challenges, several researchers have looked into new charging strategies. Studies looking into a coordinated charging system backed by improved grid systems and Vehicle-to-Grid (V2G) technology have shown promising results in addressing the challenges [32] and [39]. V2G allows electric vehicles to supply power back to the grid during peak demand, helping keep voltage stable, easing the stress on the electrical network. The integration of distributed generation with EV charging can aid grid performance, even more so when combined with strategically placed renewable sources [36].

Using renewable energy sources in charging systems is a great way to support sustainable practices. Linking renewable sources such as solar and wind-based energy has been proven to help in balancing the supply and demand of the charging network [33]. The use of predictive control and communication technologies can improve efficiency by scheduling charging based on when renewable energy is available [39].

Large-scale EV charging creates both technical and operational challenges for modern power systems. Renewable integration and bidirectional charging, integrated with a smart control system, help the grid better meet the growing demand for EVs. These findings provide the basis for creating data-driven forecasting models that help balance load demand and keep the grid operating steadily and sustainably.

2.4. How Machine Learning Works

Machine learning (ML) is a data-driven method used to recognize patterns in past information and apply them to predict future outcomes. In electric vehicle (EV) charging load forecasting, ML models study historical charging behavior along with other influencing factors to estimate future demand [1], [7], [8]. The process usually involves several key stages, including data collection, preprocessing, feature selection, model training, validation, testing, and prediction [37], [38].

The first stage, data collection, involves gathering important information such as the start and end times of charging sessions, total energy consumed, and time-related details like the hour and day. For this study, the data come from EV charging stations in Colorado, which provide the main input for analysis. Once the data is collected, the preprocessing stage begins. Here, errors or missing values are corrected, and the dataset is standardized to make it consistent. Since EV charging sessions occur at irregular intervals, the data are also organized into a continuous time series so that the model can better capture changes in charging behavior [7].

The next step is feature extraction and selection, which identifies the variables that most affect charging demand. Common features include past demand, time indicators like the hour or month, which can influence travel and charging patterns [1]. Methods like the maximal information coefficient (MIC) are used to remove less useful features and keep only those that provide meaningful information [7].

After selecting features, the model training stage begins. The chosen machine learning algorithm learns the relationship between the input variables and actual demand.

Different models can be applied, including k-nearest neighbors, support vector regression, random forest, and neural networks such as ANN, RNN, LSTM, Bi-LSTM, GRU, and DNN [1], [7], [8]. The general regression neural network (GRNN) is noted for its accuracy and simplicity because it requires adjusting only one main parameter [7].

After the model has been trained, it goes through validation and testing to check how well it performs on new data. This step ensures that the model can generalize and make reliable predictions. Common evaluation measures include the root mean square error (RMSE), mean absolute error (MAE), and the coefficient of determination (R^2). Some models are designed to automatically retrain when their accuracy drops below a certain level, such as an R^2 value of 0.81, to keep performance stable [7].

Finally, the prediction stage produces the actual forecast of charging demand. These predictions are valuable for planning and managing EV charging stations, preventing grid congestion, and ensuring efficient energy use [8], [40]. Through these steps, machine learning provides a structured and effective way to analyze EV charging behavior and support smarter energy management [1], [7], [8], [37], [38].

2.4.1 Normalization and Scaling Techniques

In many machine-learning tasks, data must be preprocessed to ensure that all features are on a similar scale. When feature ranges differ greatly, models may favor variables with larger values or struggle to train efficiently.

The following is a summary of widely used scaling and normalization methods.

1. Standardization (Z-score scaling)

This method standardizes each feature by subtracting its mean and dividing by its standard deviation, resulting in a distribution with a mean of zero and a standard deviation of one. It is particularly effective when the data roughly follow a normal distribution or when models perform better with zero-centered inputs, and it is generally less affected by outliers than simple min-max scaling.

2. Robust Scaling

Robust scaling uses the median and interquartile range instead of the mean and standard deviation. By concentrating on the median and the middle 50% of the data, this method minimizes the effect of outliers and skewed values, making it suitable for datasets where extreme values could distort other scaling approaches.

3. Max-Absolute Scaling

In this approach, each value is divided by the feature's maximum absolute value, preserving the sign of the data. The transformed values fall within the range $[-1, 1]$ or $[0, 1]$ if all data are non-negative. Since it does not shift the data, this method is useful when the data are already centered and when maintaining sparsity is important, such as with sparse input matrices.

4. Logarithmic (Log) Transformation

Log transformation applies a logarithm, usually base e or 10 , to data values, compressing larger numbers more than smaller ones. This reduces skewness and helps make skewed distributions more symmetric. It is especially useful for data that span multiple orders of magnitude or are heavily skewed, such as power usage, income, or

event counts. After applying a log transform, additional scaling or normalization may still be performed depending on the model.

5. Unit-Vector Scaling (Vector Normalization)

Unit-vector scaling normalizes each sample (row) instead of each feature, adjusting the vector so its length is one by dividing the feature values by the sample's Euclidean norm. This method is useful when the relative proportions of features are more important than their absolute values, such as in text processing, cosine-similarity-based clustering, or certain distance-based algorithms.

2.5. Statistical Forecasting Models

2.5.1. Auto Regressive Integrated Moving Average (ARIMA)

The Auto Regressive Integrated Moving Average (ARIMA) model is known for its common utilization across various fields, using fundamental techniques when dealing with time series data forecasting. The ARIMA model was developed through the Box-Jenkins Methodology, providing a systematic approach for modeling observed data [9],[10]. The model focuses on capturing linear dependencies in time series datasets by combining three components:

Auto Regression (AR):

$$AR(p) : y_t = \mu + \sum_{i=1}^p (\phi_i y_{t-i}) + \omega_t \quad (1)$$

Moving Averages (MA):

$$MA(q) : y_t = \mu + \sum_{i=1}^q (\theta_i \omega_{t-i}) + \omega_t \quad (2)$$

These two (1),(2) compose the ARIMA (p,q) model, which is mathematically represented as shown in Equations (3) and (4)

$$y_t = \mu + \sum_{i=1}^p (\phi_i y_{t-i}) + \sum_{i=1}^q (\theta_i \omega_{t-i}) + \omega_t$$

(3)

or

$$\phi(B)y_t = \mu + \theta(B)\omega_t \quad (4)$$

The ARMA model is used, along with an additional component, Integration (I) to form the general formula of the ARIMA (p,d,q) model seen in Eq. (5)

$$\phi(B)(1 - B)^d X_t = \mu + \theta(B)\omega_t \quad (5)$$

where X_t represents the observed time series, B as the backshift operator, which is equivalent to $(BX_t = X_{t-1})$, and ω_t which denotes the white noise error. The parameters p , d , and q represent the orders of autoregressive, integration, and moving average components, respectively. The polynomial $\phi(B)$ represents the AR part, $\theta(B)$ the MA component, and μ a constant term.

The ARIMA model works by transforming a non-stationary time series into a stationary one through differencing, represented by the operator. The model then linearly relates the present value of the series to its past forecast error of the outputted stationary data.

2.5.2. Seasonal Auto Regressive Integrated Moving Averaging (SARIMA)

Seasonal ARIMA (SARIMA) is a variation of the ARIMA model that takes into consideration the seasonality of a time series data set. This covers the weakness of ARIMA when it comes to real-world data that exhibits both trends and seasonal patterns. Seasonality addresses the repeating patterns or cycles that occur at a regular interval in a dataset for a specific period within a year. These patterns appear due to different factors,

such as energy usage per day or peak hours, such as during specific hours. The SARIMA model is defined mathematically as

$$\phi_p(B)\Phi_p(B^s)(1-B)^d(1-B^s)^D X_t = \theta_q(B)\Theta_q(B^s)\omega_t \quad (6)$$

where s is the length of the seasonal period, and P , D , and Q represent the orders of the seasonal autoregressive, differencing, and moving average components. The polynomials $\Phi_p(B^s)$ and $\Theta_q(B^s)$ describe the seasonal AR and MA parts, respectively. The model is generally written in compact notation as $SARIMA(p, d, q)(P, D, Q)s$. [9]

In this formulation, the non-seasonal components (p, d, q) capture short-term relationships, while the seasonal components (P, D, Q) represent periodic variations occurring every s time step. For example, $s = 12$ represents monthly seasonality in yearly data. This dual structure allows SARIMA to effectively model both trend and seasonal behavior in time series data, such as monthly temperature, energy consumption, or air quality measurements.

2.5.3. Application of SARIMA

SARIMA models are often used for environmental, climate, and energy study predictions. Farsi et al. [9] applied SARIMA to forecast monthly temperatures from the National Climate Data Center (NCDC). They found that choosing the correct model parameters is crucial for making accurate predictions. To improve this process, they introduced a variation of SARIMA called GA-SARIMA, which uses a Parallel Genetic Algorithm. This approach helps find the best parameters.

Ren et al. [11] used SARIMA to estimate electric vehicle charging demand in the energy sector. Their model successfully identified daily and weekly usage patterns typical

of user behavior. They then combined SARIMA with an LSTM network to capture more nonlinear trends. The SARIMA-LSTM model improved prediction performance, as SARIMA identified seasonal patterns while LSTM detected complex irregular fluctuations.

Sun and Tian [10] also created a SARIMA-GA-TCN model to forecast the Air Quality Index (AQI). SARIMA addressed linear and seasonal trends, while the Temporal Convolutional Network (TCN) handled short-term nonlinear variability. They optimized SARIMA parameters using Farsi et al.'s Genetic Algorithm (GA) approach [9]. This hybrid model outperformed standard SARIMA and individual neural networks, achieving lower Root Mean Square Error (RMSE) and Mean Absolute Error (MAE).

2.5.4. Background of the SARIMA Basic Coding Model

Public repositories were utilized to better understand the basic structure of the Seasonal Autoregressive Integrated Moving Average (SARIMA) model that was used in the study. The codes used in the preceding sections were adapted from a publicly available GitHub repository under reference [43]. The SARIMA model builds on the Autoregressive Integrated Moving Average (ARIMA) method to capture both trend and seasonal patterns in time series data. This study used the Seasonal Autoregressive Integrated Moving Average with Exogenous Regressors (SARIMAX) version for more flexible modeling. The Augmented Dickey-Fuller (ADF) test checked if the data were stationary. When it was not, transformations such as logarithmic scaling, rolling mean, and differencing were applied to stabilize the dataset.

The Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots guided parameter selection, which was optimized using GridSearch Cross-Validation (GridSearchCV) based on the Akaike Information Criterion (AIC). Walk-Forward Validation (WV) was applied to make the model more adaptive. The version with the lowest Mean Absolute Error (MAE) and Mean Absolute Percentage Error (MAPE) was used for forecasting. The dataset for training and testing was obtained from the Federal Reserve Economic Data (FRED), ensuring consistent and reliable input for model evaluation [43].

Stationarity Check (Augmented Dicky-Fuller)

The Augmented Dicky-Fuller (ADF) test is a statistical method primarily used to determine whether a time-series data set is stationary or non-stationary. Stationarity means that the dataset's statistical properties, such as mean and variance, are constant, meaning it is constant over an identified timeframe, which is an important parameter to consider in using ARIMA and SARIMA. If the dataset is non-stationary, in contrast, it means that the dataset changes over time, which can cause inaccuracies in forecasting quality. The basic model code for the ADF test can be seen in the following code:

```
Python
from statsmodels.tsa.stattools import adfuller

def dickey_test(data):
    X_dickey=data.values
    result = adfuller(X_dickey)
    print('ADF Statistic: %f % result[0])
```

```
print('p-value: %f % result[1])
print('Critical Values:')
for key, value in result[4].items():
    print('\t%s: %.3f % (key, value))
```

The function “dicky_test” takes the dataset, extracts its numerical values, and runs the ADF test. The test produces outputs such as the ADF statistic, the p-value, and the critical values. The ADF statistic measures how strongly the data support being stationary, and the p-value indicates the level of statistical significance. If the p-value is below 0.05, the data is considered stationary. The critical values at one, five, and ten percent confidence levels serve as reference points for interpreting the result [10]. This function helps evaluate the stability of the dataset before applying forecasting models like ARIMA or SARIMA.

Model Parameters Identification

The following code is used to analyze time series data by showing how current values relate to past observations.

```
Python
import statsmodels.api as sm

fig, ax = plt.subplots(2,1)

fig = sm.graphics.tsa.plot_acf(data, ax=ax[0])

fig = sm.graphics.tsa.plot_pacf(data, ax=ax[1], method='ywm' )

plt.show()
```

It helps identify patterns or dependencies that are important when building forecasting models. The code imports the Statsmodels library to create two plots: the Autocorrelation Function (ACF) and the Partial Autocorrelation Function (PACF). The line `plt.subplots(2,1)` divides the figure into two sections, where the first plot displays the ACF and the second shows the PACF. The ACF plot, created with `sm.graphics.tsa.plot_acf(data, ax=ax[0])`, measures how data points are correlated with their previous values over time. The PACF plot, drawn using `sm.graphics.tsa.plot_pacf(data, ax=ax[1], method='ywm')`, shows the direct relationships between observations after removing indirect effects. Finally, `plt.show()` displays both plots. These graphs help determine how many past data points should be considered when developing forecasting models such as ARIMA or SARIMA.

SARIMA Model Fitting

This code is used to build a forecasting model that can capture both short-term patterns and repeating seasonal trends in time series data. It uses the SARIMA model from the Statsmodels library.

```
Python
model=sm.tsa.statespace.SARIMAX(data,
                                order=(1,1,1),
                                seasonal_order=(0,1,1,36))
```

The function `sm.tsa.statespace.SARIMAX()` sets up the model and analyzes the data stored in `data`. The parameter `order=(1,1,1)` defines the non-seasonal components,

represented as p, d, and q. Here, p equals 1, meaning the model uses one past observation. The value d equals 1, which applies one differencing step to make the data stationary. The value q equals 1, meaning the model includes one past error to adjust predictions. The parameter “*seasonal_order=(0,1,1,36)*” defines the seasonal components, represented as P, D, Q, and S. In this case, P equals 0, meaning no seasonal autoregressive term is used. D equals 1, applying one seasonal differencing step to account for repeating patterns. Q equals 1, adding one seasonal moving average term. S equals 36, meaning the seasonal pattern repeats every 36 time intervals.

2.6. Deep Learning Forecasting Models

2.6.1. Temporal Convolutional Network

The Temporal Convolutional Network (TCN) is a deep learning model made for sequence modeling tasks. It is especially good at capturing both short-term and long-term dependencies in time-series data [12]. Unlike recurrent neural networks (RNNs), TCNs use only convolutional layers. This design allows for parallel computation and helps avoid problems like vanishing or exploding gradients. A TCN includes causal convolutions and dilated convolutions. Causal convolutions make sure that the prediction at time step t depends only on inputs from time steps up to t , which keeps the temporal order intact. Dilated convolutions, defined as

$$y_t = \sum_{i=0}^{k-1} w_i x_{t-d \cdot i} \quad (7)$$

Expand the receptive field by adding gaps controlled by the dilation factor d' . This method allows the model to capture long-range temporal dependencies without increasing

computational complexity [44]. The TCN also includes residual connections and normalization layers to improve convergence and stability during training.

In forecasting tasks like predicting electric vehicle (EV) charging demand, TCNs can effectively extract temporal features from changing data patterns while keeping computational efficiency high. In the hybrid SARIMA-GA-TCN framework, the TCN component models nonlinear temporal dependencies [10].

2.6.2. Convolutional Neural Network

The Convolutional Neural Network (CNN) is a feedforward neural network structure mainly built for feature extraction using convolutional and pooling operations [34]. CNNs use localized filters on input data, creating feature maps based on

$$y_t = f(W * x_t + b) \quad (8)$$

where $*$ denotes the convolution operation, W represents the kernel weights, b is the bias, and $f(\cdot)$ is a nonlinear activation function. In time-series forecasting, CNNs are used to capture local time-based dependencies and identify short-term fluctuations or sudden changes in data patterns [47]. However, because standard CNNs do not specifically enforce time causality, they struggle to model long-term dependencies when compared to architectures built for sequence learning, like TCNs.

Even with this limitation, CNNs are still useful for short-term forecasting and feature extraction in hybrid frameworks. They can act as preprocessing models to extract local time-based features from EV charging data before applying more complex time models like TCN.

2.6.3. Difference between CNN and TCN

Despite using convolutional operations to extract features, CNNs and TCNs handle temporal information differently. CNNs use standard convolutions to merge data from past and future time steps. This can upset the natural order of the data [48]. Conversely, TCNs use causal and dilated convolutions. These maintain the sequential structure while assisting the model in accurately capturing dependencies over extended periods. This design difference allows TCNs to perform better in sequence prediction and time-series forecasting tasks [12].

In practical applications like EV charging demand forecasting, CNNs are good at identifying short-term patterns, while TCNs are better at learning multi-scale and long-range temporal relationships.

2.6.4. Background of Basic TCN coding model

A publicly available implementation of the Temporal Convolutional Network (TCN) will help us understand the basic structure and function of TCN-based forecasting models. The source code, developed by MFHChade [49], provides a model design for medium-term load forecasting. It combines one-dimensional convolutional layers with dilation, pooling, and fully connected layers. This design effectively captures time-related dependencies in sequential data. This implementation will serve as a structural guide for developing the TCN model in the study to predict electric vehicle (EV) charging load demand.

Input Variables

The variable “*time_series*” is the model's input. It contains historical charging load data with a defined shape of (n_inputs, 1), where “*n_inputs*” represents the number of time steps or past data points used for prediction.

```
Python
time_series = Input(shape=(n_inputs, 1))

other_feat = Input(shape=(n_inputs, n_channels-1))
```

Convolutional Layers

The first function used is “*Conv1D()*”, which means one-dimensional convolution. This function acts as a pattern detector that moves along the time-series data to find significant trends. It includes several parameters: “*filters*” specifies how many patterns the model will find; “*kernel_size*” determines how many past time points are considered together; “*padding*” controls whether the data size remains the same (“*same*”) or slightly decreases (“*valid*”); and “*dilation_rate*” defines the spacing between data points being analyzed, allowing the model to capture wider time dependencies.

Three convolutional layers are used in sequence: “*conv1*”, “*conv2*”, and “*conv3*”. This setup lets the model learn patterns at different scales. The first layer captures short-term changes in charging demand, while the second and third layers identify more complex and long-term patterns.

Python

```
conv1 = Conv1D(filters = 3, kernel_size = 2, padding = 'causal', dilation_rate = 5)
(time_series)

conv2 = Conv1D(filters = 3, kernel_size = 2, padding = 'same', dilation_rate = 5)
(conv1)

conv3 = Conv1D(filters = 3, kernel_size = 2, padding = 'same', dilation_rate = 5)
(conv2)

pool = MaxPooling1D(pool_size = 5, strides = 4, padding = 'same') (conv3)
```

Pooling and Flattening Layers

After these, the “*MaxPooling1D()*” function reduces the data size by selecting the most important information from small groups of time points. Its parameters include “*pool_size*”, which determines how many points are grouped, and “*strides*”, which specifies how far the pooling window shifts. This step simplifies the data and lowers computational costs. The “*Flatten()*” function reshapes the processed data into a one-dimensional array. This transformation prepares the extracted information for the dense layers, which will be used to make predictions.

Python

```
pool = MaxPooling1D(pool_size = 5, strides = 4, padding = 'same') (conv3)

flat_ts = Flatten() (pool)

flat_ft = Flatten() (other_feat)
```

Dense Layers and Output Layer

The “*Dense()*” layers are fully connected layers responsible for decision-making. Three Dense layers are defined with 100, 50, and 25 neurons each, and they all use a “*linear*” activation to keep the numerical relationships between inputs and outputs. These layers combine and refine the extracted features to create meaningful forecasts. The final “*Dense()*” layer, labeled “*output_ts*”, has 7 neurons that represent the predicted EV charging load for the next seven time intervals.

```
Python
dense_network = Dense(100, activation = 'linear') (merged)

dense_network = Dense(50, activation = 'linear') (dense_network)

dense_network = Dense(25, activation = 'linear') (dense_network)

output_ts = Dense(7, activation = 'linear') (dense_network)
```

Model Compilation and Training

The model is compiled using the “*compile()*” function with two key parameters: “*optimizer='adam'*”, which adjusts the learning rate to improve accuracy, and “*loss='mse'*”, which stands for Mean Squared Error and measures how close the model’s predictions are to the actual values. The training process is done through the “*fit()*” function, where “*X_ts_train*” and “*y_train*” are the input and output training data, while “*X_ts_val*” and “*y_val*” are used for validation. The “*epochs=300*” parameter means that the model will train on the entire dataset 300 times, continuously enhancing its predictions. The “*batch_size=10*” parameter indicates that data will be processed in groups of 10 samples at a time to make learning more efficient.

To prevent overtraining, the “*EarlyStopping()*” function is applied. This function watches the model’s validation loss with the parameter “*monitor='val_loss'*” and automatically stops training when no further improvement occurs after 100 iterations, defined by “*patience=100*”. The “*min_delta=1e-3*” parameter sets the minimum required improvement for training to continue, ensuring efficiency and avoiding unnecessary computation.

```
Python
epochs = 300;

TCN_func.compile(optimizer = 'adam', loss = "mse")

monitor = EarlyStopping(monitor='val_loss', min_delta=1e-3, patience= 100,
verbose=2, mode='min')

TCN_history = TCN_func.fit(x = [X_ts_train, X_ft_train], y = y_train,
validation_data=[[X_ts_val,X_ft_val], y_val], epochs = epochs, batch_size =
10,callbacks=[monitor])
```

2.7. Optimization Model

2.7.1 Genetic Algorithm

The Genetic Algorithm, or GA, is an optimization method inspired by natural selection, where the fittest solutions survive and evolve through a process of selection, crossover, and mutation. It is known for its adaptability and efficiency in handling complex optimization problems [14].

In the SARIMA–GA–TCN forecasting model, the GA is used to adjust the hyperparameters of the Temporal Convolutional Network, or TCN, to achieve better

prediction accuracy for Electric Vehicle load residuals [10]. Manual tuning of parameters can be slow and unreliable, while the GA automatically searches for the best combinations that reduce forecasting errors [13].

Each individual in the population represents one possible set of model parameters. For instance, an individual can contain the following structure:

```
Python  
[p, d, q, learning rate, batch size]
```

The following outlines show the mathematical equivalents of the GA are implemented in the SARIMA–GA–TCN framework:

Fitness Function

The Temporal Convolutional Network, or TCN, handles the nonlinear residuals produced by the SARIMA model. To improve this prediction, the Genetic Algorithm adjusts key TCN hyperparameters, including the number of filters, kernel size, learning rate, and batch size, to reduce forecasting errors [10]. The fitness function guiding this optimization is based on the Mean Absolute Error, or MAE, which serves as the minimization goal of the Genetic Algorithm. The equation for MAE can be seen in equation (14).

Each individual is evaluated based on how accurately it forecasts EV charging demand. The forecasting error (MAE, RMSE, SMAPE, or R^2) serves as the fitness score. In the pseudocode given, the lower the scores, the better:

Python

for individual in population:

 fitness = evaluate_model(individual)

Chromosomes

In the Genetic Algorithm, the chromosome represents a set of numerical values that form one possible solution to the problem being optimized.

In the SARIMA–GA–TCN model, the GA optimizes the hyperparameters of the Temporal Convolutional Network (TCN), including the number of filters, kernel size, learning rate, and batch size [10]. These parameters make up the chromosome array, expressed as:

$$\text{chromosome} = [\text{num_Filters}, \text{kernel_size}, \text{learn_rate}, \text{batch_size}] \quad (9)$$

Since some parameters, such as the learning rate, are continuous, the algorithm is likely to use a continuous genetic approach. In this case, the chromosome is encoded as an array of real or floating-point numbers, rather than binary values. This setup enables more precise optimization as the GA searches within defined numerical ranges for the optimal combination of hyperparameters.

Selection Probability and Operators

The GA evolves the population through selection, crossover, and mutation, simulating natural evolution to identify fitter solutions progressively.

Selection:

Chromosomes with lower Mean Absolute Error, or MAE, values are more likely to be chosen for reproduction. If any fitness values are negative, a transformation such as rank weighting is applied to ensure that all probabilities remain non-negative. The chance that a chromosome (C_i) is selected, represented as $(P(C_i))$, increases with its fitness level [10].

A simple selection process might look like the following:

```
Python
def select_parents(population, fitness):

    sorted_pop = [x for _, x in sorted(zip(fitness, population))]

    return sorted_pop[0], sorted_pop[1]
```

It selects two parents with the best fitness. This ensures that individuals with low errors are more likely to reproduce.

Crossover:

Two high-performing parent chromosomes are combined to create new offspring. In a continuous GA, crossover blends real-number values using a random coefficient (β) instead of swapping bits [10].

Mutation:

With a small probability (around 0.001), mutation introduces random variations by altering real-number parameters within their allowed ranges. This preserves

population diversity and prevents premature convergence [10]. This process blends traits from both parents, allowing exploration of new parameter combinations.

2.7.2 Particle Swarm Optimization

Particle Swarm Optimization (PSO) is an optimization algorithm inspired by the collective movement of birds flocking or fish schooling. It is widely used to optimize forecasting models by efficiently searching for the best set of parameters. In electric vehicle (EV) charging load forecasting, PSO helps improve prediction accuracy by automatically adjusting model parameters to minimize forecasting errors, such as Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE) [15], [16].

In PSO, a group of particles represents possible solutions within the search space. Each particle has a position x_i and velocity v_i , which are continuously updated based on individual experience and group knowledge [15]. The mathematical model of PSO is defined as:

$$v_i^{t+1} = wv_i^t + c_1r_1(pbest_i - x_i^t) + c_2r_2(gbest - x_i^t) \quad (10)$$

$$x_i^{t+1} = x_i^t + v_i^{t+1} \quad (11)$$

where v_i^t is the current velocity of the particle i , x_i^t is its current position, $pbest_i$ is the best position found by the particle, $gbest$ is the best position found by the entire swarm, w is the inertia weight that controls exploration and exploitation, c_1 and c_2 are acceleration coefficients, and r_1 and r_2 are random values between 0 and 1.

During the optimization process, particles move within the search space and adjust their positions to find the global best solution for the global market. This iterative process continues until the algorithm reaches the optimal result or meets a stopping criterion. In EV charging demand forecasting, PSO is often integrated with models such as Temporal Convolutional Networks (TCN), Long Short-Term Memory (LSTM), or other hybrid deep learning models. It enhances convergence speed and prevents the model from getting trapped in local minima, leading to more reliable and precise multi-step forecasting results [15], [17].

Basic Coding Background for PSO

The Adaptive Inertia Weight Particle Swarm Optimization, or AIW PSO, is a version of the standard Particle Swarm Optimization algorithm. It improves how the algorithm balances exploration and exploitation during the search process. The method is implemented in Python using the Meaply library, which offers built-in metaheuristic algorithms for optimization. The class `meaply.swarm_based.PSO.AIW_PSO` defines this algorithm and inherits its main functions from the base optimizer class.

In AIW PSO, a group of candidate solutions called particles moves within the search space to find the best solution that minimizes or maximizes an objective function. Each particle has a position and a velocity. These are updated every iteration based on the particle's own best result and the best result found by the entire swarm. The key feature of AIW PSO is its adaptive inertia weight, which changes as the search goes on. At first, a higher inertia weight lets particles explore more widely to avoid early convergence. As iterations continue, the weight decreases to focus on the most promising regions. This

adaptive change improves both the speed of convergence and the quality of the final solution.

In the Mealpy version, several parameters control how the algorithm runs:

- **epoch**

Controls how long the optimization process runs. More epochs allow the algorithm to explore and refine model parameters more thoroughly, which can reduce forecasting errors such as MAE and RMSE. However, too many iterations may cause overfitting or unnecessary computation once the model has already converged.

- **pop_size**

Determines how many particles (candidate solutions) search the parameter space. A larger population increases diversity and helps the optimizer avoid local minima, improving model generalization. Too small a population may lead to premature convergence and higher forecast error, while an excessively large one increases computation time.

- **c_1 (Cognitive Coefficient)**

Controls how much each particle learns from its own best result. In model optimization, this helps fine-tune local parameter combinations specific to each particle. If c_1 is too large, the swarm loses cooperation and may explore inefficiently; if too small, particles depend too heavily on global feedback and may stagnate.

- **c_2 (Social Coefficient)**

Defines how strongly particles follow the global best solution. A higher c_2

accelerates convergence toward the best-found model configuration but risks premature convergence if diversity decreases. Balancing c_1 and c_2 ensures both individual learning and collective guidance, leading to more stable and accurate forecasting performance.

- **α (Alpha – Inertia Adaptation Constant)**

Controls the adaptive rate of the inertia weight that shifts the search from global exploration to local exploitation. In forecasting model tuning, a higher α encourages early exploration of wide parameter ranges, while a lower α focuses on precise adjustments near the optimum. Proper tuning of α stabilizes convergence, prevents oscillations, and improves the model's predictive reliability [5], [8].

```
Python
problem_dict = {

    "bounds": FloatVar(lb=(-10.) * 30, ub=(10.) * 30, name="delta"),

    "obj_func": objective_function,

    "minmax": "min",

}

model = PSO.AIW_PSO(epoch=1000, pop_size=50, c1=2.05, c2=2.05, alpha=0.4)

g_best = model.solve(problem_dict)
```

The problem dictionary defines the variable bounds from -10 to 10 for each of 30 dimensions, specifies the objective function, and declares that the problem is a minimization task. In this setup, the algorithm runs for 1000 iterations with 50 particles.

Each particle keeps updating its position and velocity using the PSO equations until it reaches the best possible solution. The results at the end show the best solution value stored in `g_best.solution` and its fitness score in `g_best.target.fitness`, which indicates how close the result is to the optimal target.

The `AIW_PSO` class in Mealpy gives a practical version of the adaptive PSO algorithm. The inertia weight changes gradually during the run. This change helps the model explore a wide range of solutions at the start and improve accuracy later on. The result is faster and more stable convergence toward the global best solution.

The overall phases share similarities to the genetic algorithm because both are population-based evolutionary algorithms, but they differ in how each phase operates and how solutions are generated.

1. Initialization Phase

The algorithm starts by creating a group of particles, where each one represents a possible solution within the set search limits. Each particle gets a random starting position and velocity. In the Mealpy version, this process happens automatically when the variable limits are defined using the `FloatVar` function in the problem dictionary.

2. Fitness Evaluation Phase

```
Python
def objective_function(solution):
    return np.sum(solution**2)
```

Each particle is then evaluated using the objective function, which measures how well it performs based on the optimization goal. The objective function computes the sum of squared values, and its result is called the fitness value. For a minimization problem, a lower fitness value means a better solution.

3. Global Best Update

After evaluation, each particle compares its current fitness value with its own best position, called pbest or personal best. The best position found among all particles is stored as gbest or global best. These two values guide how the particles adjust their movement in the next iteration.

4. Velocity Update

The velocity of each particle is updated based on its inertia, its distance from the personal best, and its distance from the global best. This is calculated using the PSO velocity equation (10)

5. Position Update

Once the velocity is updated, the new position of each particle is calculated. This allows particles to move through the search space toward areas with higher potential fitness.

6. Weight Adjustments

Unlike standard PSO, AIW PSO changes the inertia weight w during iterations. The weight decreases gradually to shift the search from exploration to exploitation using the formula:

$$w = w_{max} - \alpha \times (w_{max} - w_{min}) \times \frac{t}{t_{max}} \quad (12)$$

A larger w at the start encourages broad exploration, while a smaller w toward the end focuses on refining the best solution. The parameter α controls how quickly this change happens.

7. Termination

The algorithm stops when it reaches the maximum number of iterations or when changes between iterations become very small. The final result shows the best solution found ($g_best.solution$) and its fitness value ($g_best.target.fitness$), representing the most optimal outcome found by the swarm.

These steps allow AIW PSO to move efficiently from a wide global search to precise local optimization. The adaptive inertia weight helps it converge faster and more reliably than standard PSO.

2.8 Statistical Significance Testing for Forecast Model Comparison

Forecasting studies commonly use accuracy metrics such as MAE, RMSE, SMAPE, and R^2 to describe the magnitude of forecasting errors. However, these metrics only show the numerical difference between models and do not determine whether the

difference in predictive accuracy is statistically significant. In electricity forecasting studies, formal forecast-comparison tests such as the Diebold–Mariano test have been used to determine whether the predictive performance of one model is significantly different from another. Maciejowska and Weron [18] used the Diebold–Mariano test in electricity price forecasting to determine whether some forecasting models were significantly more accurate than others. Similarly, Uniejewski et al. [19] applied variants of the Diebold–Mariano test to evaluate whether forecast-performance differences across electricity spot price datasets were statistically significant. Saez-Gallego and Morales [20] also used Diebold–Mariano testing in short-term price-responsive load forecasting to compare forecasting methods statistically.

The use of formal forecast-comparison testing has also been applied in load-demand and hybrid forecasting studies. Jagait et al. [21] emphasized that error metrics alone may not be sufficient for comparing forecasting models and used the Diebold–Mariano test to examine whether forecast differences were significant in hybrid load forecasting under concept drift. Ijaz et al. [22] applied the Diebold–Mariano statistical inference test at a 95% confidence level when comparing an LSTM-based short-term electrical load forecasting model with benchmark models. These studies show that statistical testing is useful when comparing competing forecasting models beyond descriptive error values.

In hybrid and deep-learning energy forecasting, the Diebold–Mariano test has also been used to support model-comparison claims. Liu et al. [23] applied the Diebold–Mariano test to validate the performance differences of a hybrid TCN-LSTM wind-power forecasting model. Çağlayan-Akay and Topal [24] used two forms of the

Diebold–Mariano test in comparing single and hybrid electricity-consumption forecasting models. Iftikhar et al. [25] used the Diebold–Mariano test to assess the superiority of an electricity-demand ensemble forecasting technique across several competing models. Baek and Seo [26] also applied the Diebold–Mariano test together with validation and diagnostic procedures in hybrid university electricity-demand forecasting. These studies support the use of a Diebold–Mariano-based statistical test when comparing statistical, deep-learning, and hybrid forecasting models on the same test observations.

Based on these related studies, the present research adopted the Harvey–Leybourne–Newbold corrected Diebold–Mariano test as an additional statistical validation method. While MAE, RMSE, SMAPE, and R^2 were retained to describe the magnitude of forecasting errors, the HLN-corrected Diebold–Mariano test was used to determine whether the observed performance differences among the benchmark and hybrid models were statistically significant. This strengthened the model-comparison procedure by verifying whether the differences between TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN were not only numerically different but also statistically meaningful under the selected loss criterion.

2.9. Gap Analysis

Table 2.1. Gap Analysis

Study / Model	Model / Technique	Residual Handling	Optimization	Nonlinear Modeling	Hybrid / Ensemble Structure
SARIMA	Seasonal ARIMA (statistical time series)	✓ (Models residual errors)	✗	✗	✗
TCN	Temporal Convolutional Network	✓ (Implicit in convolutions)	✗	✓	✗
PSO-TCN	TCN optimized via Particle Swarm Optimization	✓	✓ (PSO)	✓	✓
GA-TCN	TCN optimized via Genetic Algorithm	✓	✓ (GA)	✓	✓
An Ensemble Methodology for Hierarchical Probabilistic EV Load Forecasting [27]	GBRT + QRF + QRNN Ensemble	✓ (Hierarchical residual correction)	✗	✓	✓
Integrated Approaches in Resilient Hierarchical Load Forecasting via TCN [28]	Temporal Convolutional Network	✓ (Resilient to missing data)	✗	✓	✗
Real-Time Hosting Capacity Assessment for EVs [29]	Adaptive Spatio-Temporal Graph CNN (ASTGCN)	✓ (Probabilistic error modeling)	✓ (Embedded optimization)	✓	✓
Short-Term EV Charging Load Forecasting Based on TCN-LSTM [2]	Hybrid TCN + LSTM	✓ (Temporal smoothing)	✗	✓	✓

Spatio-Temporal Forecasting of EV Charging Demand [30]	Temporal Graph CNN + Traffic Models	✗	✗	✓	✓
EV Charging Load Forecasting Using PSO-LSTM [31]	LSTM optimized via PSO	✗	✓ (PSO)	✓	✓
SARIMA-GA-TCN	SARIMA + GA-optimized TCN	✓ (SARIMA residuals fed to TCN)	✓ (GA)	✓	✓
SARIMA-PSO-TCN	SARIMA + PSO-optimized TCN	✓ (SARIMA residuals input)	✓ (PSO)	✓	✓
Study / Model	Forecasting Scope	Feature Inputs	Evaluation Metrics	Gap / Limitations	
SARIMA	Short-term, univariate	Load data only	MAE, RMSE, MAPE	Cannot capture nonlinear relationships; lacks adaptive learning	
TCN	Short-term, multivariate	Load, time features	MAE, RMSE	No explicit trend/seasonality modeling; requires hyperparameter tuning	
PSO-TCN	Short-term, multivariate	Weather, load, time	MAE, RMSE, R^2	Strong nonlinear capture but lacks decomposition of linear trend components	
GA-TCN	Short-term, multivariate	Weather, time, load	MAE, RMSE, SMAPE	Robust but computationally heavier; lacks explicit seasonal pre-modeling	
An Ensemble Methodology for Hierarchical Probabilistic EV Load Forecasting [27]	Probabilistic, multi-level	Mainly load data	CRPS, probabilistic score	Not short-term; lacks deep learning temporal modeling	

Integrated Approaches in Resilient Hierarchical Load Forecasting via TCN [28]	Short-term, residential	Weather, load	MAE, RMSE	No optimization; lacks seasonal-trend extraction
Real-Time Hosting Capacity Assessment for EVs [29]	Real-time, probabilistic	Spatial-temporal features	RMSE, MAE, risk	High complexity; not purely forecasting model
Short-Term EV Charging Load Forecasting Based on TCN-LSTM [2]	Short-term, station-level	Weather, similar-day	MAE, RMSE, MAPE	No automatic optimization or explicit seasonality handling
Spatio-Temporal Forecasting of EV Charging Demand [30]	City-level, spatio-temporal	Traffic, user data	MAPE, R^2	Focus on spatial, not temporal decomposition; lacks optimization
EV Charging Load Forecasting Using PSO-LSTM [31]	Short-term, seasonal	Weather, seasonality	MAE, RMSE, R^2	Focused on sequential modeling only; no residual or trend decomposition
SARIMA-GA-TCN	7-day-ahead, station-level	Load, weather, time, seasonal	MAE, RMSE, SMAPE, R^2	Fills prior gaps: combines decomposition + global optimization for nonlinear learning
SARIMA-PSO-TCN	7-day-ahead, station-level	Load, seasonality, user behavior, weather	MAE, RMSE, SMAPE, R^2	Enhances model efficiency, reduces training cost, improves temporal precision

Recent studies on electric vehicle (EV) charging demand forecasting have made progress, but improvements are still needed. Some research uses ensemble-based probabilistic forecasting in hierarchical setups [27], yet these models do not combine deep learning methods or automated hyperparameter tuning. Temporal Convolutional Networks (TCNs) have been used for hierarchical load forecasting [28], but most do not include decomposition or ensemble structures that could help lower forecasting errors.

Other works focus on hosting capacity and grid simulations instead of prediction accuracy [29], leaving short-term, data-driven forecasting less explored.

Combined models, such as TCN-LSTM frameworks, can learn nonlinear temporal patterns and handle multiple input features [2]. However, they often lack hyperparameter optimization and residual decomposition. Spatio-temporal models linked to traffic data improve spatial forecasting [31], but they still do not use residual learning or optimization techniques such as Genetic Algorithms (GA) or Particle Swarm Optimization (PSO).

The gap analysis in Table 2.1 summarizes these differences by comparing traditional, deep learning, and hybrid models across several performance areas. Classical models like SARIMA can model seasonality well but cannot capture nonlinear patterns. Deep learning models like TCN manage nonlinearities but need manual parameter tuning and lack trend decomposition. Optimized models such as PSO-TCN and GA-TCN improve parameter selection but still miss linear trend extraction.

The SARIMA-GA-TCN and SARIMA-PSO-TCN frameworks address these gaps. They combine SARIMA's strength in modeling linear and seasonal trends with GA and PSO's optimization power and TCN's nonlinear learning ability. This hybrid setup allows automatic hyperparameter tuning, efficient residual learning, and better temporal precision. It provides a more adaptive and accurate method for short-term EV charging demand forecasting.

Model performance is evaluated using Root Mean Square Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), and the Coefficient of Determination (R^2). RMSE highlights large errors, which helps detect peak load

deviations. MAE measures average errors in actual units. MAPE shows percentage errors at different load levels, and R^2 indicates how well the model explains the observed variations.

CHAPTER 3.0. METHODOLOGY

3.1. Overview of the chapter

This chapter contains the step-by-step procedure that the researchers performed to test and measure the results of this research. It included the research design, research approach, and the various methods and analytical techniques that were applied throughout the course of the study.

3.2. Research Design

This study used a quantitative experimental-comparative research design. It focused on creating and assessing a data-driven machine learning model to predict electric vehicle (EV) charging station load demand. The study included building, training, validating, and testing predictive models that used secondary historical EV charging data. It also compared the performance of six forecasting models: SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN. The aim was to find out which model provided the most accurate and reliable short-term forecasts by effectively capturing both linear and nonlinear patterns in EV charging demand. Model performance was assessed using accuracy metrics like Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Symmetrical Mean Absolute Percentage Error (SMAPE), and the Coefficient of Determination (R^2). This experimental-comparative design supports objective testing and fair evaluation under controlled conditions. It ensured that the results were valid, measurable, and based on consistent model performance.

3.3.Data Collection/Gathering Procedure

3.3.1. Data Gathering

The dataset used for this study was the Electric Vehicle Charging Station Dataset from the City of Boulder Open Data Portal. It contained about 14,000 records of public charging sessions, including timestamps, station IDs, session durations, and energy delivered (kWh).

This dataset was selected for its reliability, detailed timeframe, and wide coverage. These features suited daily, weekly, and seasonal charging habits. Zhao et al. [50] note that datasets with prolonged periods are essential for accurately capturing trends in EV charging demand.

All data were downloaded directly from the open-source repository and saved in a secure research directory. The raw dataset was maintained as the source copy. Preprocessed versions were used to train and evaluate the model. This method will ensure transparency and reproducibility.

3.3.2. Data Preprocessing

Sun and Tian's research was considered when modifying the preprocessing procedure [10]. They introduce a hybrid forecasting model that combines SARIMA, GA, and TCN for time series prediction. Although their study focused on air quality index (AQI) data, the same framework was applied for preprocessing data from EV charging sessions. Seasonal, nonlinear, and time-related characteristics are similar in both regions.

3.3.3. Data Cleaning

Records of specific charging sessions are included in the raw data from the Boulder Electric Vehicle Charging Station Dataset. During the data cleaning procedure, duplicate entries, incomplete records, and odd readings, like negative or zero energy values, were eliminated. By doing this, the series is guaranteed to show only legitimate charging activity. According to Sun and Tian [10] and Wang et al. [51], cleaning time-series data to eliminate noise and fill in gaps increases model accuracy.

3.3.4. Timestamp Standardization

All timestamps were converted to a uniform time zone, Mountain Time, UTC-7, and standardized to a date-time format. Temporal irregularities, such as overlapping or misaligned start and end times, were corrected. Consistent time alignment is essential for SARIMA-based models, which need equally spaced intervals for accurate autocorrelation and seasonality detection [10].

3.3.5. Temporal Aggregation

Individual charging sessions were grouped into hourly intervals. The total energy delivered (kWh) for each station per hour was calculated. This hourly grouping reduced noise and revealed demand changes at the level of charging networks. Sun and Tian [10] used hourly AQI measurements to capture regular changes. Similarly, the hourly detail highlighted peak and off-peak EV charging behaviors identified in previous studies [52], [53].

3.3.6. Feature Engineering

Daily and seasonal charging demand cycles were modeled using time-based features such as the hour of the day, day of the week, and month. Using common techniques from temporal learning models, sine and cosine functions were applied to modify these cyclical variables in order to maintain their periodic nature [54].

3.3.7. Normalization

Normalization means adjusting numerical data so that all features stay within a similar range. Time series data often has wide differences in scale. EV charging load values can reach several kilowatt-hours, while features like hour of day or day of week are much smaller. These gaps in size can cause problems for machine learning models. Models such as TCN, GA-TCN, and PSO-TCN learn through gradients, and large values can overshadow smaller ones. This slows training and can make the learning process unstable. Normalization fixes this by placing the data on a shared scale so each feature has a fair effect on the model.

In this study, normalization placed the EV charging load, the features taken from the timestamps, and the processed residuals within the same range before they entered the SARIMA GA-TCN and SARIMA PSO-TCN models. Two common ways to do this were min-max scaling, which adjusted the values into the zero-to-one range, and standardization, which used the mean and the standard deviation. For the deep learning part of the model, min-max scaling was used because it helped the gradients stay steady and supported faster training. The scaling values came only from the training set, so the model does not gain information from the validation or test sets.

With normalization in place, the hybrid models learned the patterns in the data more clearly, avoided numerical issues, and produced better forecasts. This step was important for preparing the time series data for both the SARIMA model and the TCN-based models.

All numerical features were normalized to a [0, 1] range using min–max scaling:

$$x_i^* = \frac{x_i - x_{min}}{x_{max} - x_{min}} \quad (13)$$

Min-max scaling was used because it kept all numerical values in the same range, usually from zero-to-one. This helped the model handle each feature fairly. When one feature had much larger values than the others, it could take over the learning process and make the model unstable. Scaling removed this imbalance and kept the training steady.

Models like TCN, GA-TCN, and PSO-TCN depend on gradients. These gradients moved through the network more smoothly when the inputs stayed within a fixed range. This led to faster training and lowered the chance of gradients blowing up or fading out. Min-max scaling also kept the original shape of the data, which was important for time series work. The model could still see the changes and patterns in the EV charging data.

Overall, min max scaling made training more stable. It helped the model learn faster and improved the accuracy of the forecasts in the hybrid SARIMA GA-TCN and SARIMA PSO-TCN models.

3.3.8. Stationarity Transformation

The Augmented Dickey-Fuller (ADF) test was used to test the combined demand series, as the SARIMA model requires a stationary input. If the null hypothesis of non-stationarity ($p > 0.05$) could not be rejected, first-order differencing would be applied until stationarity was achieved. This approach would follow the method of Sun and Tian [10], which used the SARIMA submodel to extract linear components.

3.3.9. Data Splitting

For the study, the dataset that covers the electric vehicle charging sessions, which spans from January 2018 to November 2023, was divided into training (80%), validation (10%), and testing (10%) sets, respectively, following chronological order. This 80-10-10 split was selected due to the dataset spanning almost six years, providing sufficient data for robust model training, leaving enough data for validation and training, respectively. The chosen partition ensured that the models effectively learned long-term seasonal and trend components while maintaining adequate data for validation and testing. This setup also supported comparing different forecasting approaches by maintaining a consistent data structure across all model configurations.

3.4. Instrumentation

This section presents the instruments, tools, and resources used to develop, train, and evaluate the proposed Seasonal Autoregressive Moving Average-Genetic Algorithm-Temporal Convolutional Network (SARIMA-GA-TCN) and Seasonal Autoregressive Moving Average-Particle Swarm Optimization-Temporal Convolutional

Network (SARIMA-PSO-TCN) model for forecasting Electric Vehicle (EV) charging demand.

3.4.1. Hardware Instruments

Description of Hardware Environment

The system that was used for model development and training is equipped with high-performance components designed to handle deep learning workloads efficiently. It provided sufficient computational capacity for developing, training, and testing complex models, such as SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN models, supporting GPU-accelerated computations, which enabled efficient processing and effective management of large EV charging demand datasets.

System Specification

Processor: AMD Ryzen 7 7435HS (3.10 GHz)

Installed RAM: 16.0 GB (15.8 GB usable, 4800 MT/s)

Graphics Processing Unit: NVIDIA GeForce RTX 4050 Laptop GPU (6 GB)

Storage: 512 GB Solid-State Drive (SSD)

Operating System: Windows 11 Home Single Language, Version 24H2 (64-bit, x64-based processor)

3.4.2. Software Instruments

The software tools that were used in this study supported the development, training, and testing of the SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN models. These included programming languages, libraries, visualization tools, and system platforms required for executing and testing the EV charging demand forecasting framework.

Programming Language and Core Libraries

Python is the primary programming language that was used to model the SARIMA, TCN, GA-TCN, PSO-TCN, SARIMA-GA-TCN, and SARIMA-PSO-TCN models that were tested in the study. The pandas and numpy libraries handled data preparation and numerical computation, including loading datasets, cleaning invalid values, and creating input features. The datetime and collections libraries helped manage time-based data and organize sequences for time series forecasting. The pmdarima Library was applied to construct and tune the SARIMA model parameters.

Machine Learning and Deep Learning Libraries

Scikit-learn supported preprocessing, feature scaling, and model evaluation. The deep learning modules were developed using TensorFlow, with Keras acting as a high-level interface for building, training, and refining the TCN. Hyperparameter optimization was performed using both Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). PSO is a population-based metaheuristic inspired by the social behavior of birds or fish, where candidate solutions explore the search space and iteratively adjust their positions based on their own experience and that of neighboring particles to find the optimal solution.

Visualization and Analysis Tools

Matplotlib was used for visualization, helping display training performance, accuracy trends, and comparisons of forecasting results among models.

Deep Learning Framework

The project utilized a deep learning setup that combines SARIMA (Seasonal Autoregressive Integrated Moving Average) and TCN (Temporal Convolutional

Network). The study utilized two optimizer models, GA (Genetic Algorithm) and PSO (Particle Swarm Optimization). These models identified the best hyperparameter values for the TCN model, such as filter size, kernel size, batch size, and learning rate. This optimization improved the model's accuracy and performance in forecasting Electric Vehicle charging load demand.

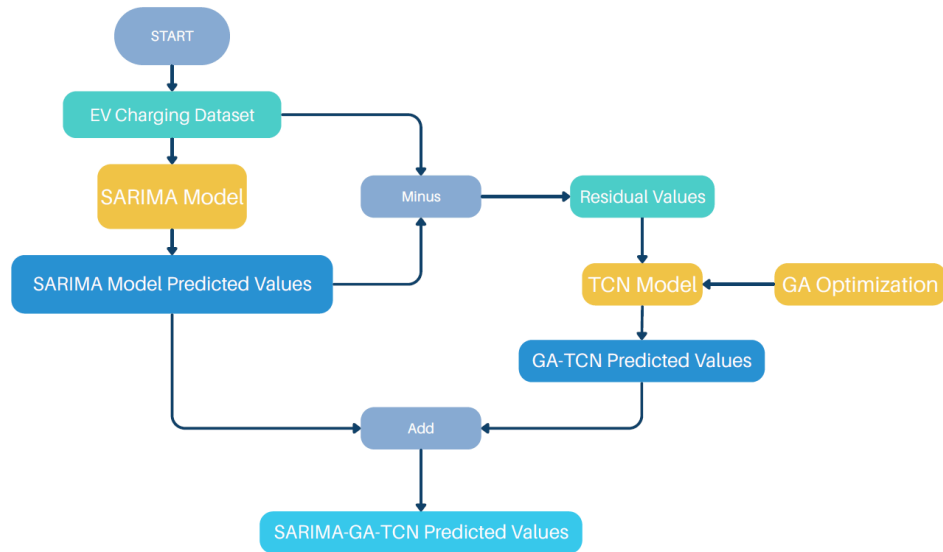


Figure 3.1. SARIMA-GA-TCN Model

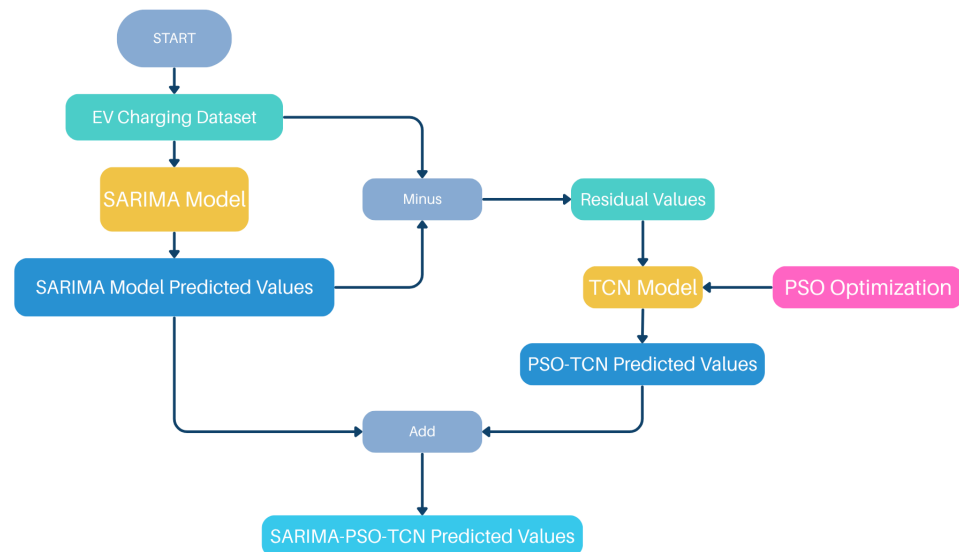


Figure 3.2. SARIMA-PSO-TCN Model

3.4.3. Evaluation Metrics

The performance of the models was measured using standard statistical metrics to assess accuracy and reliability. These metrics helped determine how well the predicted values match the actual Electric Vehicle (EV) charging demand data.

1. Mean Absolute Error (MAE) measures the average difference between the predicted and actual values.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \bar{y}_i| \quad (14)$$

n = total number of observations

y_i = actual value

$\bar{y}_i$ = predicted value

2. Root Mean Square Error (RMSE) evaluates the square root of the average squared differences between predictions and actual values.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \bar{y}_i)^2} \quad (15)$$

n = total number of observations

y_i = actual value

$\bar{y}_i$ = predicted value

3. The Symmetrical Mean Absolute Percentage Error (SMAPE) expresses forecast accuracy as a percentage, facilitating easier comparison across different data scales.

$$SMAPE = \frac{100\%}{n} \sum_{i=1}^n \frac{|y_i - \bar{y}_i|}{(|y_i| + |\bar{y}_i|)/2} \quad (16)$$

n = total number of observations

y_i = actual value

$\bar{y}_i$ = predicted value

4. The coefficient of Determination (R^2) shows how well the model explains the variation in actual data.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \bar{y}_i)^2}{\sum_{i=1}^n (y_i - \mu)^2} \quad (17)$$

n = total number of observations

y_i = actual value

$\bar{y}_i$ = predicted value

μ = mean of the actual values

Together, these metrics offered a complete evaluation of forecasting performance, ensuring that the model maintained high accuracy, stability, and reliability. They are widely recognized in forecasting research for capturing complementary aspects of accuracy and model fit. The SARIMA–VMD–GA–TCN model achieved low MAE and RMSE with a high R^2 [12]. Likewise, ANN-based methods for medium-term EV demand forecasting yielded lower SMAPE and MAE than traditional statistical methods [1]. On the other hand, the GRNN model maintained a high R^2 and low MSE in self-supervised EV charging prediction tasks [7]. Overall, MAE, RMSE, SMAPE, and R^2 continue to

serve as robust and complementary indicators for evaluating performance in time series forecasting.

3.5. Developed EV Charging Demand Prediction Model

3.5.1. Comparison of Genetic Algorithms and Particle Swarm Optimization

Table 3.1. Genetic Algorithm Vs Particle Swarm Optimization

Feature	PSO's unique strength	GA's unique strength
Problem Domain Focus	PSO is specifically successful for continuous optimization problems and is efficient for continuous optimization problems.	GA is highly effective for problems with large search spaces and complex constraints.
	PSO is proven effective in the context of parameter optimization and control engineering.	GA demonstrates exceptional performance in addressing resource allocation and engineering design challenges.
Computational Characteristics	PSO is characterized as Fast.	GA shows strength in optimization and design, with interest in complexity.
	PSO has a Low memory usage, whereas GA uses High memory.	GA exhibits a Strong compatibility with the real environment, while PSO's compatibility is rated as Good.
	PSO's computing cost is rated as Low to medium, whereas GA's computing cost is Medium.	GA's analysis highlights its Flexibility and strength in solving complex problems.
Scale and Adaptation	PSO has versatile adaptation and is suitable for large-scale problems.	GA's key advantage is being effective for combinatorial problems with large, discrete search spaces.
General Effectiveness	PSO is renowned for solving continuous and discrete optimization problems without difficulty and effectiveness.	GA is suitable for problems where complexity and constraints are high.

Particle Swarm Optimization (PSO) and Genetic Algorithms (GA) each have strengths that fit different types of problems. PSO works well for continuous optimization

and for tuning model parameters. It runs fast, uses minimal memory, and handles large numerical problems with minimal cost. GA works better when the search space is large and includes many constraints. They can explore complex problem setups and can adjust well to different design requirements. [56] The study proposes the use of two hybrid models, SARIMA-GA-TCN and SARIMA-PSO-TCN, leveraging the unique strengths of both optimization methods. Using GA allows the model to explore broader and more complex parameter landscapes, while PSO enables rapid, efficient tuning for continuous residual patterns. Using both approaches enables researchers to determine which one produces more dependable short-term forecasts for electric vehicle charging loads. It also helps show whether the strengths of each method can improve the overall accuracy and stability of the forecasting system.

3.5.2. SARIMA-GA-TCN

This section develops a hybrid SARIMA–GA–TCN forecasting model that integrates Seasonal Autoregressive Integrated Moving Average, Genetic Algorithm, and Temporal Convolutional Network to forecast Electric Vehicle charging demand. SARIMA captures linear and seasonal patterns in EV load data, such as daily and hourly charging peaks.

The Temporal Convolutional Network predicts the nonlinear residuals that SARIMA cannot capture. It applies causal and dilated convolutions to learn complex temporal patterns in the data. The Genetic Algorithm fine-tunes key TCN hyperparameters, including the number of filters, kernel size, and learning rate, to improve accuracy and computational efficiency.

SARIMA models the linear components, the GA-optimized TCN handles nonlinear residuals, and their outputs combine to produce an accurate EV load forecast. Model performance is evaluated using Mean Absolute Error, Root Mean Squared Error, Symmetric Mean Absolute Percentage Error, and R-squared.

The flow chart of the model is shown in Figure. 3.1.

3.5.3. SARIMA-PSO-TCN

The SARIMA–PSO–TCN model combines the Seasonal Autoregressive Integrated Moving Average (SARIMA), Particle Swarm Optimization (PSO), and Temporal Convolutional Network (TCN) to forecast electric vehicle (EV) charging demand. The SARIMA component captures linear and seasonal behaviors in the Colorado EV charging dataset, such as daily and hourly demand variations.

The residuals generated from the SARIMA forecast are then modeled by the TCN, which uses causal and dilated convolutions to identify nonlinear dependencies in the time series. The PSO algorithm is applied to automatically tune key TCN hyperparameters, including the number of filters, kernel size, dilation rate, and learning rate. By simulating swarm behavior, PSO searches for the optimal configuration that minimizes forecasting error and improves model convergence.

The outputs of SARIMA and the PSO-optimized TCN are combined to generate the final forecast of EV charging demand. Model performance was assessed using Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), Symmetric Mean Absolute

Percentage Error (SMAPE), and the Coefficient of Determination (R^2). The overall process of the hybrid model is shown in Figure 3.2.

3.5.4. SARIMA Model

The SARIMA model, which is an improved ARIMA model, is fitted to evaluate and consider seasonality or seasonal patterns present in the data. For this thesis, this was implemented as the first step in the prediction model.

In using a SARIMA or an ARIMA model, there are three main steps in their utilization: step 1, model identification; step 2, parameter estimation; and step 3, diagnostic checking [9]. The identification process involves determining the proper stationary time series. In order to say that a dataset is stationary, the mean and standard deviation should be constant, and there should be no seasonality, since the SARIMA model itself deals with this seasonality; this step is not prioritized. Analysis of parameters can be done using the autocorrelation function (ACF) and partial autocorrelation function (PACF). ACF is used as a measure of the correlation between the previous values of the time series and the latest values, checking their relationships with each other. While PACF deals with the low-order lags, representing the values of the correlation coefficient between the variable and its time lag [55].

Two main methods applied when choosing the appropriate models are Akaike's Information Criterion (AIC) and the Bayesian Information Criterion of Schwartz (BIC) [9]. The mathematical formulas for both AIC and BIC are shown below:

$$(18) \quad AIC = -2\log(L) + 2k' = -2\log(L) + 2(p + q + P + Q)$$

$$(19) \quad BIC = -2\log(L) + k\ln(n) = -2\log(L) + (p + q + P + Q)\ln(n)$$

The variable “ n ” refers to the series size, and “ k ” refers to the number of parameters of the ARIMA/SARIMA method. For both AIC and BIC, a low value would result in a better fit for the model.

3.5.5. TCN Model

The Temporal Convolutional Network (TCN) is an alternative to traditional recurrent neural network (RNN) architectures often used in time series forecasting. Its structure is built on one-dimensional causal convolutions that are usually dilated to ensure that predictions at any time step rely only on past information. Each TCN block consists of several convolutional layers, and the overall model is formed by stacking multiple blocks while keeping the input and output sizes consistent across all layers. Temporal Convolutional Networks (TCNs) tend to perform better than standard recurrent neural networks (RNNs) and long short-term memory networks (LSTMs) for time series prediction. They are well-suited for long sequences because they use causal and dilated convolutions, which allow the model to learn patterns over time while keeping the influence of earlier data intact.

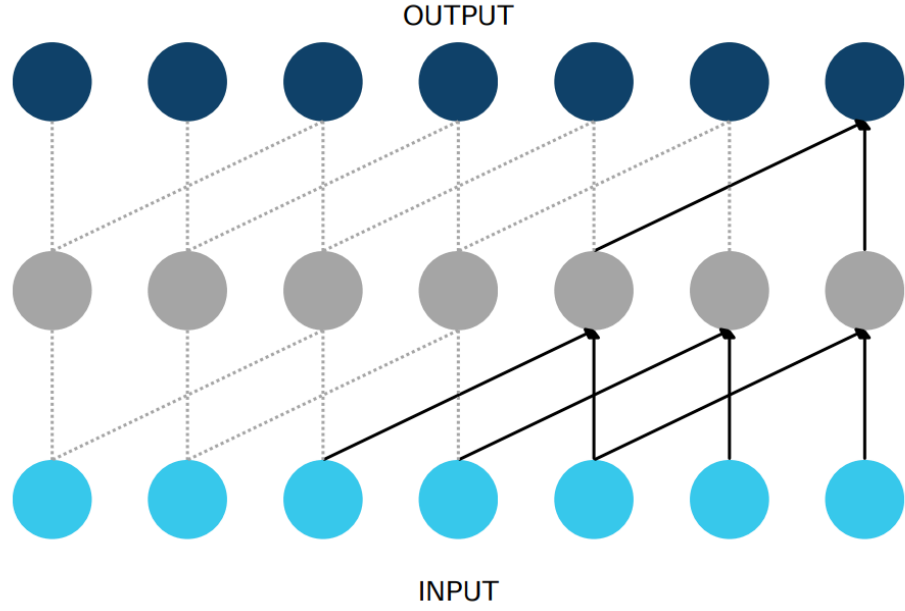


Figure 3.3. TCN Model

The input to the TCN is the residual sequence $r(t) = y(t) - \hat{y}_{SARIMA}(t)$, combined with contextual features such as hour of day $h(t)$ and day type $d_{week}(t)$. The TCN maps this input to a predicted nonlinear correction $\hat{r}_{TCN}(t)$ through multiple stacked convolutional layers:

$$\hat{r}_{TCN}(t) = f_{TCN}(r(t), h(t), d_{week}(t); \theta) \quad (20)$$

where f_{TCN} represents the TCN function and θ denotes the set of learnable weights of the network. Each convolutional layer applies the operation:

$$X^{(l+1)}(t) = \sigma\left(\sum_{k=0}^{K-1} w_k^{(l)} \cdot x^{(l)}(t - d' \cdot k) + b^{(l)}\right) \quad (21)$$

Here:

$x^{(t)}$ is the input to layer l at time t ,

$w_k^{(l)}$ are the convolutional weights for the kernel of size K ,

d' is the dilation factor,

$b^{(l)}$ is the bias term, and

σ is the activation function (ReLU in this study).

3.5.6. Residual Generation

The residuals from the SARIMA model was used directly as input for both GA-TCN and PSO-TCN. Each residual kept its original time stamp, so it stayed aligned with features like the hour of the day and day of the week. This allowed the GA-TCN and PSO-TCN to focus purely on the nonlinear patterns in the charging demand that SARIMA couldn't capture. The process was carried out programmatically, producing a residual dataset that kept the same structure and size as the original data.

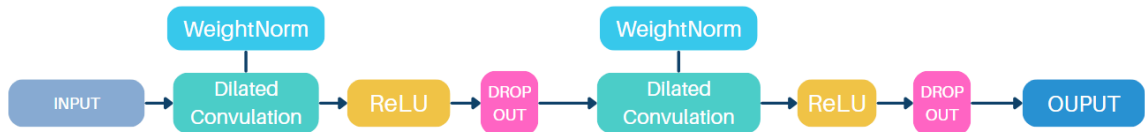


Figure 3.4. Residual Generation Network

In this study, residuals was generated by subtracting the SARIMA model predictions from the actual EV charging demand values for each time step. Formally, this is expressed as:

$$r(t) = y(t) - \hat{y}_{SARIMA}(t) \quad (22)$$

Convolutional layers learn patterns in time-series data by applying filters across the input. These filters, or kernels, move through the sequence to detect features like spikes, drops, or repeated shapes in EV charging load. Each filter computes weighted sums over a set time window to extract a specific feature. Unlike recurrent layers, which process data step by step, convolutional layers handle the full sequence at once. This makes them faster and more efficient. In TCNs, the convolutions are causal, so the model only uses past data. This is required for forecasting.

A 1D convolution operation (used in TCNs for time series) can be expressed as:

$$y(t) = \sum_{i=0}^{k-1} w(i) \cdot x(t - i) \quad (23)$$

Where:

$y(t)$: output at time step t

$x(t - i)$: input value at time $t - i$

$w(i)$: weight of the convolution filter at position i

k : kernel size (number of time steps the filter looks at)

In dilated convolutions (used in TCN to expand the receptive field), the equation becomes:

$$y(t) = \sum_{i=0}^{k-1} w(i) \cdot x(t - d' \cdot i) \quad (24)$$

Where d is the dilation factor — how much the filter is spaced out.

Weight normalization is a technique used to improve training efficiency and stability in neural networks. Instead of directly learning the raw weight values, the network learns a separate direction vector and scaling factor. This allows the optimizer to adjust the size and orientation of weights independently, leading to faster convergence and more stable gradients. In convolutional layers, particularly in deep networks like TCNs, weight normalization helps prevent issues like exploding or vanishing gradients when many layers are stacked. This makes the training process more stable and reliable.

Weight normalization reparameterizes the weights $\mathbf{w}$ as:

$$\mathbf{w}' = \frac{g}{\|\mathbf{v}\|} \cdot \mathbf{v} \quad (25)$$

Where:

$\mathbf{v}$: a vector representing the direction of the weights (learned)

$\|\mathbf{v}\|$: the Euclidean norm (i.e., magnitude) of $\mathbf{v}$

g : a scalar parameter (also learned) representing the scaling factor

$\mathbf{w}'$: the actual weight vector used in the model

This helps decouple the magnitude and direction of the weight vector, making training more stable.

ReLU is an activation function used after convolution to add non-linearity to the model. It's defined as $f(x) = \max(0, x)$, meaning it outputs the input if it's positive, and zero otherwise. This simple function allows the network to learn complex, non-linear relationships in the data. In time series forecasting, ReLU helps the model focus on important signals while ignoring irrelevant or negative noise. It's also computationally efficient and reduces the risk of the vanishing gradient problem during backpropagation.

The ReLU activation function is defined as:

$$\text{ReLU}(x) = \max(0, x) \quad (26)$$

Its derivative (for backpropagation) is:

$$\text{ReLU}'(x) = \begin{cases} 1, & \text{if } x > 0 \\ 0, & \text{if } x \leq 0 \end{cases} \quad (27)$$

This function introduces non-linearity into the model and allows it to learn complex patterns.

Dropout is a regularization technique employed in neural networks to improve generalization and mitigate overfitting in deep architectures. During the training phase, dropout randomly deactivates a subset of neurons within a given layer according to a predefined dropout rate, denoted as p , which typically ranges between 0.1 and 0.5. Formally, if h represents the vector of neuron activations in a layer, the dropout operation can be expressed as:

$$\tilde{h} = h \odot r \quad (28)$$

where r is a binary mask vector sampled from a Bernoulli distribution with probability $1 - p$, and $\odot$ denotes element-wise multiplication. The resulting vector $\tilde{h}$ contains the activations after dropout has been applied. By temporarily omitting certain neurons, the network is prevented from developing excessive reliance on specific connections, thereby encouraging redundancy in feature representations and enhancing robustness. During inference, all neurons are active, with outputs scaled appropriately to compensate for the dropout applied during training. In deep networks such as Temporal

Convolutional Networks (TCNs), which are prone to memorization due to their depth, the application of dropout in both convolutional and fully connected layers facilitates the learning of meaningful temporal features, improving predictive performance in tasks such as short-term electric vehicle charging load forecasting.

3.5.7. GA-TCN Model

The Genetic Algorithm (GA) is first applied to enhance the performance of the Temporal Convolutional Network (TCN). GA is an evolutionary optimization method inspired by natural selection, where a population of candidate solutions is iteratively improved through selection, crossover, and mutation to identify the best-performing configuration. In the GA-TCN framework, the algorithm is used to automatically optimize key TCN hyperparameters such as the number of filters, kernel size, batch size, and learning rate. Let the hyperparameter set for an individual chromosome be defined as

$$X_i = \{F_i, K_i, B_i, \eta_i\} \quad (29)$$

where F_i is the number of filters, K_i is the kernel size, B_i is the batch size, and η_i is the learning rate. Each of these combinations is tested by training a TCN model on the EV charging demand data. Its performance is measured using an error metric, usually Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE), which acts as the fitness value:

$$fitness(x_i) = \frac{1}{N} \sum_{t=1}^N |y_t - \hat{y}(x_i)| \quad (30)$$

Here, y_t is the actual charging load at time t , and $\hat{y}(x_t)$ is the predicted value produced using that parameter set. Solutions with lower error are considered fitter, so they're more likely to be kept and combined to create new candidates. Through repeated selection, crossover, and mutation, GA narrows down the options until it finds the best-performing configuration. That final parameter set χ^* is then used to configure the TCN within the SARIMA–GA–TCN framework, which improves the model's ability to forecast nonlinear residual patterns in EV charging demand.

3.5.8. PSO-TCN Model

The Particle Swarm Optimization (PSO) algorithm was applied to improve the performance of the Temporal Convolutional Network (TCN) in forecasting the nonlinear residuals generated by the SARIMA model. PSO is a population-based optimization method inspired by the collective behavior of birds and fish, where each particle represents a potential solution that moves through the search space toward the optimal configuration. In the PSO–TCN framework, each particle corresponds to a set of TCN hyperparameters, such as the number of filters, kernel size, batch size, and learning rate.

The fitness of each particle is evaluated by training a TCN model on the EV charging demand residuals and calculating its forecasting error, typically measured by Mean Absolute Error (MAE) or Root Mean Squared Error (RMSE). Each particle adjusts its position according to its best-known solution and the best solution found by the swarm, as expressed by:

$$x_i^{t+1} = x_i^t + wv_i^t + c_1r_1(p_{best,i} - x_i^t) + c_2r_2(g_{best} - x_i^t) \quad (31)$$

where x_i^t is the current position of particle i , v_i^t is its velocity, $p_{best,i}$ is its best previous position, and g_{best} is the global best position found by the swarm. The parameters w , c_1 , and c_2 control the balance between exploration and convergence, while r_1 and r_2 are random values between 0 and 1.

Through several iterations, the swarm converges to the optimal hyperparameter configuration that minimizes forecasting error. The best parameter set is then used to configure the TCN, enhancing its ability to learn nonlinear residual patterns and improve forecasting accuracy within the SARIMA–PSO–TCN framework.

3.5.9. Integration

The integration component functions as the fusion layer that combines the output of both sub-models to generate the final prediction. The implementation was carried out programmatically after obtaining the predicted arrays from each component. The SARIMA model provided the baseline linear forecast, while the GA–TCN and PSO–TCN networks produced the nonlinear residual correction. Both outputs were aligned based on their corresponding time indices to ensure temporal consistency.

In the context of the study, the integration served as the output synthesis mechanism, where the SARIMA represented the primary trend and seasonal behavior of EV charging demand, and the TCN output provided fine corrections for non-linear fluctuations. The result of this integration produced the hybrid forecast vector, which is subsequently compared to the actual charging demand values to evaluate performance. The hybrid forecast vector formula is as shown:

$$\hat{y}_{final(t)} = \hat{y}_{SARIMA(t)} + \hat{r}_{GA-TCN(t)} \quad (32)$$

$$\hat{y}_{final(t)} = \hat{y}_{SARIMA(t)} + \hat{r}_{PSO-TCN(t)} \quad (33)$$

Rationale in Combining Linear and Nonlinear Models

The combined model is necessary because EV charging demand exhibits both seasonal patterns and sudden changes in usage. A linear model can follow the regular cycles in the data, but it cannot handle the unexpected shifts in charging behavior. A nonlinear model can learn those shifts, but it becomes less stable when it tries to learn everything at once. Using SARIMA for the linear part first leaves the nonlinear part to the TCN, which makes both models easier to train. It also helps avoid unnecessary errors and reduces the chance that the nonlinear model will overfit.

The hybrid setup offers practical benefits for electric utilities beyond higher accuracy. The nonlinear model only works on the remaining errors, allowing it to be updated more frequently without affecting the entire system. This helps the forecasting process adjust when charging habits change. The split between linear and nonlinear tasks keeps the model easier to manage as station demand grows. It also provides utilities with steadier forecasts during uncertain periods, which supports planning and day-to-day decisions. This makes the system more adaptable as EV use continues to rise.

3.5.10. Novelty of the Proposed Framework

This study develops two innovative models that systematically separate and model linear and nonlinear components through explicit residual learning, unlike previous hybrid approaches that use SARIMA only as a preprocessing step. The research

optimizes the nonlinear residual model using two distinct metaheuristic algorithms. Instead of merely stacking SARIMA and TCN, the study introduces a structured residual pipeline where the isolations from SARIMA feed into a TCN. Here, the hyperparameters are independently tuned using Genetic Algorithm and Particle Swarm Optimization. These two optimization pathways produce functionally distinct nonlinear learners, enabling a comparative analysis of optimization behavior, convergence stability, and forecasting consistency. This dual-hybrid setup introduces a novel approach to EV charging load forecasting, moving beyond the typical single hybrid or single optimization workflows found in existing literature.

3.6. Benchmarking and Comparative Analysis

The benchmarking in this research proposal was conducted using a quantitative experimental-comparative research design. The objective is to thoroughly evaluate the performance, accuracy, and reliability of the proposed Seasonal Autoregressive Integrated Moving Average-Genetic Algorithm-Temporal Convolutional Network (SARIMA-GA-TCN) model and Seasonal Autoregressive Integrated Moving Average-Particle Swarm Optimization-Temporal Convolutional Network (SARIMA-PSO-TCN) model.

This process systematically evaluates the forecasting performance of the models by comparing them under six configurations using the same datasets and evaluation criteria. To determine whether optimization truly improves forecasting performance, it is necessary to compare the optimized hybrid models with their unoptimized counterparts. Without this comparison, it would be unclear whether performance gains come from model design or merely from optimization algorithms. Benchmarking against

unoptimized SARIMA and TCN models allows the study to quantify the contribution of GA and PSO to error reduction, convergence stability, and robustness. The benchmarking framework aims to determine the extent to which the main architectural components—enhanced SARIMA decomposition, TCN’s nonlinear learning, and both GA and PSO optimization—contribute to the overall accuracy of forecasting.

3.6.1. Comparative Research Design and Benchmark Models

The benchmarking involves evaluating six forecasting models to assess their performance.

Table 3.2. Benchmark Model Configurations

Model Category	Benchmark Models Evaluated	Role in Benchmarking
Combine Models	SARIMA–GA–TCN	Model combining linear decomposition (SARIMA) and GA-optimized nonlinear learning (TCN).
	SARIMA–PSO–TCN	Tests the effect of replacing GA with Particle Swarm Optimization (PSO).
Optimized TCN Models	GA–TCN	Tests the impact of removing the SARIMA linear decomposition stage.
	PSO–TCN	Evaluates the performance of a TCN optimized by PSO without SARIMA decomposition.
Single Models	SARIMA	Baseline model that captures only linear and seasonal patterns.
	TCN	Baseline model representing a single deep learning framework

		without decomposition or optimization.
--	--	--

All models were trained using 80% of the historical data, validated on 10%, and tested on the remaining 10%, following a chronological split to ensure reliable evaluation on unseen time periods. The models are developed for short-term forecasting, specifically to forecast one week (seven steps) of future charging demand using hourly data.

3.6.2. Parameter Optimization Strategy

The metaheuristic optimization procedure adjusts the primary hyperparameters to ensure fairness and to achieve the best possible performance from all neural networks.

Objective

The Genetic Algorithm (GA) and Particle Swarm Optimization (PSO) in comparative models automatically optimize key TCN hyperparameters:

- Number of filters (num_filters)
- Kernel size (kernel_size)
- Batch size (batch_size)
- Learning rate (learn_rate)

Fitness Function

The optimization process minimizes the Mean Absolute Error (MAE) on the validation set, which serves as the fitness score.

Process

The last parameter set, which was set by the optimizer's configuration, enables TCN within the SARIMA-GA-TCN and SARIMA-PSO-TCN framework, thereby improving its ability to predict nonlinear residual patterns.

3.6.3. Test Data and Preparation

The benchmarking dataset comprises historical Electric Vehicle charging data from the City of Boulder's Open Data Portal, spanning January 2018 to November 2023.

Data Preprocessing

Raw session data is cleaned by removing duplicates, incomplete entries, and records with zero or negative energy values. The cleaned data is standardized and aggregated into hourly intervals.

Feature Engineering

Time-based features such as hour, day, and month are generated and encoded using sine and cosine transformations to represent daily and seasonal charging cycles.

Normalization

All numerical features are scaled between 0 and 1 using min-max normalization to stabilize the learning of the neural network.

Stationarity

The Augmented Dickey-Fuller test is used to check for stationarity. If the data is non-stationary, first-order differencing is applied to stabilize it for the SARIMA component.

3.6.4. Evaluation Criteria for Energy Load Prediction Models

The evaluation metrics, MAE, SMAPE, and RMSE, are standard loss functions used for assessing regression tasks, such as time series forecasting of energy load or electric vehicle (EV) charging demand. For all error metrics (MAE, RMSE, SMAPE), performance is considered better when the value is closer to zero. Conversely, performance is considered better when the value is closer to one.

Based on recent academic research for EV charging demand prediction models, here are the standard interpretations and benchmark ranges found in the sources:

Table 3.3. Standard Evaluation Metrics

Metric	Measurement and Standard Interpretation	Typical Performance Benchmarks and Practical Relevance (EV/Energy Forecasting)
R^2	A value closer to 1 (or 100%) signifies stronger predictive capability [4].	Hybrid models consistently achieve high fit, with values of 0.977 (Beijing AQI) and 0.962 (Aba AQI) observed [10]. High R^2 confirms the strong explanatory power of combined model approaches [10].
MAE	A value closer to 0 indicates higher accuracy [10].	In EV demand forecasting, the lowest network MAE recorded was 124.670 kWh, representing less than 12% of average daily demand [1]. For individual stations, the lowest average MAE reached 31.315 kWh, confirming practical effectiveness [1].
RMSE	A value closer to 0 indicates better model stability [10].	Reducing RMSE is essential for grid reliability. Optimal EV load prediction models achieved RMSE values around 1.03 [5], while hybrid AQI forecasting models reported RMSE as low as 0.983 and 6.701 [10].

SMAPE	A value closer to 0% is preferred.	For network-level EV demand, ANN models achieved 5.9% SMAPE, with the best hybrid variant reaching 5.925% [1]. However, forecasts for individual charging stations often show higher errors, averaging 21.3% SMAPE [1].
-------	------------------------------------	---

The metrics R^2 , MAE, RMSE, and SMAPE play a key role in measuring forecasting accuracy and reliability. High R^2 values and low error scores show strong model performance. This study uses these metrics to evaluate the proposed SARIMA-GA-TCN model and SARIMA-PSO-TCN model, ensuring consistent and credible measurement of their forecasting ability. The model combines linear analysis of SARIMA, optimization of GA and PSO, and nonlinear learning of TCN in order to generate low forecasting errors and high predictive accuracy for the short-term demand of Electric Vehicle charging.

3.6.5. Statistical Significance Testing of Forecast Differences

In addition to MAE, RMSE, SMAPE, and R^2 , this study applied the Harvey-Leybourne-Newbold corrected Diebold-Mariano test to determine whether the observed differences in forecasting performance were statistically significant. Performance metrics such as MAE, RMSE, SMAPE, and R^2 describe the magnitude of forecasting error, but they do not determine whether the difference between two competing forecasting models is statistically meaningful. Previous forecasting studies have used the Diebold-Mariano test to compare whether model-performance differences are statistically significant in electricity-price forecasting, short-term load forecasting, electricity-demand forecasting, and hybrid energy forecasting applications [18], [19], [20], [21], [22]. Similar hybrid and deep-learning forecasting studies also used the

Diebold–Mariano test to validate whether one forecasting model significantly outperformed another on the same test observations [23], [24], [25], [26].

Before conducting the statistical test, the forecast outputs of the benchmark and hybrid models were aligned by timestamp. This ensured that every pairwise comparison used the same actual EV charging demand observations. The test used squared error loss because it is consistent with RMSE-based evaluation and gives greater penalty to larger forecasting errors. This approach is appropriate because recent load-demand and energy forecasting studies have used formal statistical comparison tests together with conventional forecasting metrics to strengthen model-performance evaluation [21], [22], [25].

For the statistical comparison, let:

$$y_t^{DM}$$

represent the actual EV charging demand at aligned time index t , and let:

$$\hat{y}_{m,t}^{DM}$$

represent the forecasted EV charging demand of model m at aligned time index t , where $m=1$ refers to Model 1 and $m=2$ refers to Model 2.

The forecast error for each model was computed as:

$$e_{m,t}^{DM} = y_t^{DM} - \hat{y}_{m,t}^{DM} \quad (34)$$

The squared error loss was computed as:

$$L_{m,t}^{DM} = (e_{m,t}^{DM})^2 \quad (35)$$

The loss differential between Model 1 and Model 2 was then computed as:

$$d_t^{DM} = L_{1,t}^{DM} - L_{2,t}^{DM} \quad (36)$$

The average loss differential was computed as:

$$\bar{d}^{DM} = \frac{1}{n_{DM}} \sum_{t=1}^{n_{DM}} d_t^{DM} \quad (37)$$

where n_{DM} is the number of timestamp-aligned observations used in the statistical comparison.

The autocovariance of the loss differential at lag k was computed as:

$$\gamma_k^{DM} = \frac{\sum_{t=k+1}^{n_{DM}} (d_t^{DM} - \bar{d}^{DM})(d_{t-k}^{DM} - \bar{d}^{DM})}{n_{DM} - 1} \quad (38)$$

For the general Diebold–Mariano test, the standard error may include lagged autocovariance terms when multi-step forecast errors are compared. However, since the statistical comparison in this study used timestamp-by-timestamp forecast outputs, the forecast comparison horizon was set to:

$$h_{DM} = 1$$

For $h_{DM} = 1$, the summation of lagged gamma terms is empty. Therefore, the final standard error used γ_0^{DM} , which is equivalent to the variance of the loss differential series:

$$SE_{DM} = \sqrt{\frac{\gamma_0^{DM}}{n_{DM}}} \quad (39)$$

The Diebold–Mariano statistic was then computed as:

$$DM_{stat} = \frac{\bar{d}^{DM}}{SE_{DM}} \quad (40)$$

The Harvey–Leybourne–Newbold correction factor was computed as:

$$C_{HLN}^{DM} = \sqrt{\frac{n_{DM} + 1 - 2h_{DM} + \frac{h_{DM}(h_{DM}-1)}{n_{DM}}}{n_{DM}}} \quad (41)$$

The corrected HLN-DM statistic was computed as:

$$HLN - DM_{stat} = DM_{stat} \times C_{HLN}^{DM} \quad (42)$$

The p-value was computed using a two-tailed test:

$$p_{DM} = 2 \times P(T_{n_{DM}-1} > |HLN - DM_{stat}|) \quad (43)$$

The null hypothesis states that the two compared models have equal predictive accuracy:

$$H_{0,DM} : E(d_t^{DM}) = 0 \quad (44)$$

The alternative hypothesis states that the two compared models have significantly different predictive accuracy:

$$H_{1,DM} : E(d_t^{DM}) \neq 0 \quad (45)$$

The statistical decision used a significance threshold of:

$$\alpha_{DM} = 0.05 \quad (46)$$

If:

$$p_{DM} < \alpha_{DM} \quad (47)$$

then the null hypothesis was rejected, indicating that the forecasting performance difference between the two models was statistically significant. If $\bar{d}^{DM} > 0$, Model 2 had the lower average squared loss. If $\bar{d}^{DM} < 0$, Model 1 had the lower average squared loss.

The following model pairs were tested using the HLN-corrected Diebold–Mariano test:

1. TCN vs GA-TCN
2. TCN vs PSO-TCN
3. GA-TCN vs SARIMA-GA-TCN
4. PSO-TCN vs SARIMA-PSO-TCN
5. SARIMA-GA-TCN vs SARIMA-PSO-TCN

These comparisons were selected to determine whether optimization improved the baseline TCN model, whether the hybrid SARIMA-TCN structure significantly improved

the optimized TCN models, and whether the PSO-based hybrid model significantly differed from the GA-based hybrid model. This allowed the study to support its model-comparison findings using both descriptive error metrics and statistical significance testing, consistent with prior forecasting studies that combined conventional performance indicators with Diebold–Mariano-based model comparison [21], [22], [23], [24], [25], [26].

3.7. Analysis Procedure

3.7.1. Final Forecasting and Evaluation

The final prediction of the EV charging load demand was obtained by combining both components of the hybrid model using either Equation (32) or Equation (33). This approach integrates the linear output generated by the SARIMA model, which captures the temporal and seasonal patterns of the historical load data, with the nonlinear output produced by the GA-TCN or PSO-TCN model, which accounts for complex nonlinear dependencies and residual patterns in the dataset. By merging these outputs, the model is able to generate a more comprehensive forecast that leverages the strengths of both linear and nonlinear modeling techniques. After generating the final hybrid forecasts, the predicted values were inverse-transformed back to their original kilowatt-hour scale so that the final MAE and RMSE values could be interpreted directly in terms of EV charging demand.

To evaluate the performance of the hybrid model, several statistical metrics were employed, including Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Symmetric Mean Absolute Percentage Error (SMAPE), and the Coefficient of

Determination (R^2), with MAPE retained where applicable for supplementary reference. These metrics provide insights into the model's accuracy, error magnitude, percentage-based reliability, and predictive consistency. In addition to these descriptive metrics, the Harvey–Leybourne–Newbold corrected Diebold–Mariano test was applied to determine whether the differences between selected model forecasts were statistically significant. Before conducting the test, the benchmark and hybrid forecast outputs were aligned by timestamp so that each comparison used the same actual EV charging demand observations. The test used squared error loss and a 5% significance level to determine whether the observed differences in predictive accuracy were statistically meaningful. This addition is supported by related energy and load forecasting studies that used Diebold–Mariano-based testing to compare forecasting models beyond descriptive error metrics.

Furthermore, the forecast results were visualized using time-series plots to compare predicted versus actual EV charging loads over time, residual and error-distribution plots to examine forecasting deviations, zoomed comparison plots to observe short-term tracking performance, and predicted-versus-actual scatter plots to assess correlation and dispersion. These visualization techniques allow for a clear and intuitive assessment of the reliability, precision, and robustness of the SARIMA-GA-TCN and SARIMA-PSO-TCN models. Together, the descriptive metrics, HLN-corrected Diebold–Mariano statistical test, and visualization results provide a more complete evaluation of the effectiveness of the hybrid approach in capturing the dynamics of short-term EV charging demand.

CHAPTER 4.0. RESULTS AND DISCUSSION

4.1. Overview of the Chapter

This chapter presented the results of the exploratory analysis, the baseline forecasting stage, the residual-learning stage, and the comparative evaluation of the proposed hybrid models. The figures were arranged to show the behavior of the cleaned EV charging demand series, the suitability of the data for seasonal modeling, and the extent to which the optimized TCN layers improved the forecast after they were combined with SARIMA.

4.2. Dataset Characteristics, Stationarity, and Seasonality Analysis

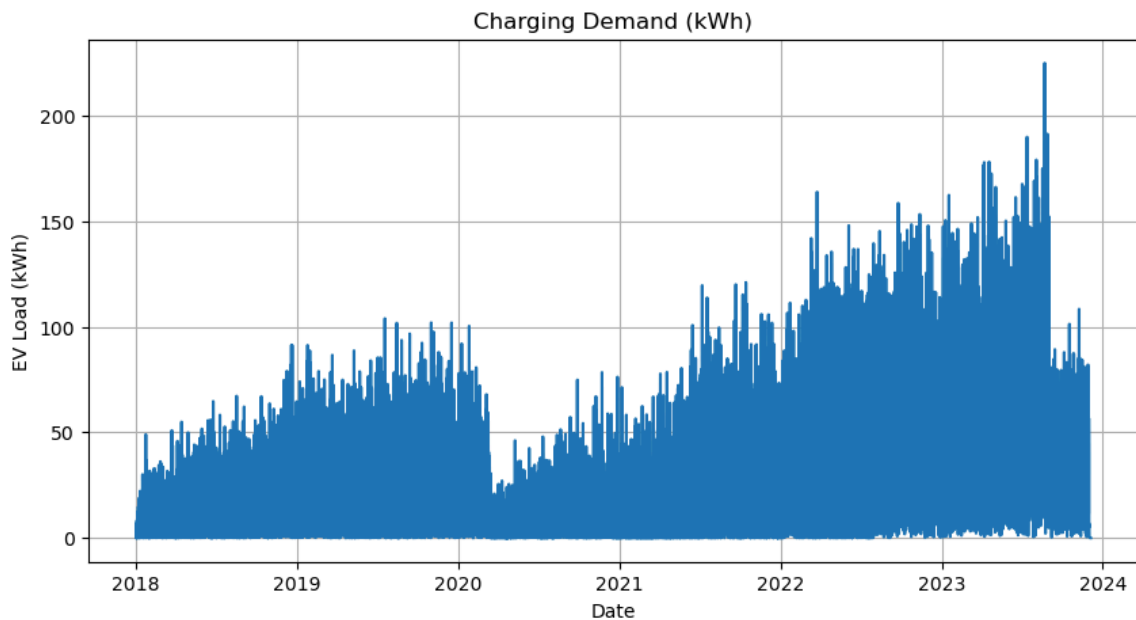


Figure 4.1. Charging Demand (kWh)

Figure 4.1 showed the cleaned and aggregated EV charging demand series used in model development. The series exhibited a clear daily loading rhythm, but it also displayed changes in intensity across the longer observation horizon. Charging activity became more pronounced in the later portion of the dataset, which indicated that the demand profile contained both recurring seasonal movement and irregular short-term

variation. This behavior justified the use of a forecasting framework that separated seasonal structure from nonlinear residual patterns.

```
--- STATIONARITY CHECK (ADF TEST) ---  
ADF Statistic: -7.5266  
p-value: 0.0000
```

Figure 4.2. Augmented Dickey-Fuller Test Output

Figure 4.2 presented the Augmented Dickey-Fuller test result for the cleaned EV charging demand series. The ADF statistic of -7.5266 and the p-value of 0.0000 showed strong evidence against the null hypothesis of non-stationarity. The result indicated that the cleaned series possessed stable statistical properties and was suitable for seasonal time-series modeling. This finding supported the use of SARIMA as the linear forecasting component of the hybrid framework.

P-Value Interpretation

The Augmented Dickey-Fuller (ADF) test yielded a p-value of 0.0000, which is significantly lower than the standard significance level of 0.05. This result leads to the rejection of the null hypothesis, which assumes that the time series is non-stationary. Consequently, it can be inferred that the EV charging load data is stationary, indicating that its statistical properties, such as mean and variance, remain stable over time.

ADF Test Statistic

The computed ADF test statistic is -7.5266, which is substantially more negative than the typical critical values used in stationarity testing. Since more negative values provide stronger evidence against the null hypothesis, this result further confirms the

stationarity of the dataset. The magnitude of this statistic indicates a high level of confidence in rejecting the presence of a unit root.

Implications for Time Series Modeling

The confirmation of stationarity is essential for the application of time series models such as SARIMA. Since the data does not exhibit non-stationary behavior, differencing may not be required, or only minimal transformation may be needed. This simplifies the modeling process and ensures that the assumptions underlying the forecasting models are satisfied, ultimately contributing to more reliable and accurate predictions.

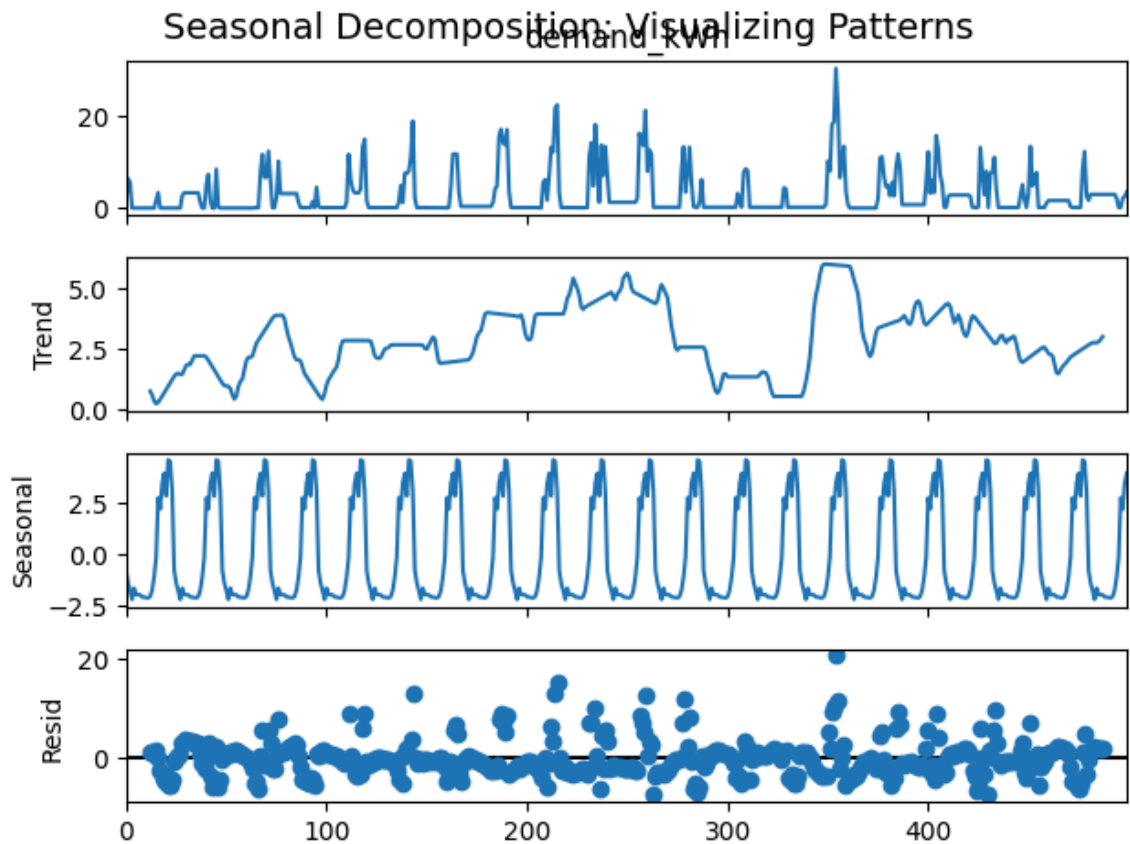


Figure 4.3. Seasonal Decomposition of the EV Charging Demand Series

Figure 4.3 decomposes the charging demand into observed, trend, seasonal, and residual components. The seasonal component showed a regular and strong daily cycle, while the residual component remained smaller and more irregular. Together with the Augmented Dickey-Fuller result, this decomposition confirmed that the cleaned hourly demand series contained a stable seasonal structure that was appropriate for SARIMA-based modeling.

The stationarity of the EV charging load data has direct implications for the configuration of the SARIMA model. Since the dataset is already stationary, there is no need to apply differencing to stabilize the mean of the series. This allows the integration

parameter d in the non-seasonal component of the SARIMA $(p,d,q)(p, d, q)$ model to be set to zero. As a result, the modeling process becomes more straightforward, as unnecessary transformations are avoided.

With differencing no longer required, the focus shifts to determining the appropriate values of the autoregressive and moving average parameters, specifically p and q for the non-seasonal component, as well as P and Q for the seasonal component. These parameters can be effectively identified through the analysis of the Autocorrelation Function (ACF) and Partial Autocorrelation Function (PACF) plots. In addition, the seasonal period s is defined based on the observed daily cycle of the data, which is set to 24 hours.

Overall, the inherent stationarity of the dataset simplifies the SARIMA model configuration and enhances its reliability, as the model can directly capture the underlying temporal dependencies without requiring additional preprocessing steps. This contributes to a more efficient and accurate forecasting framework.

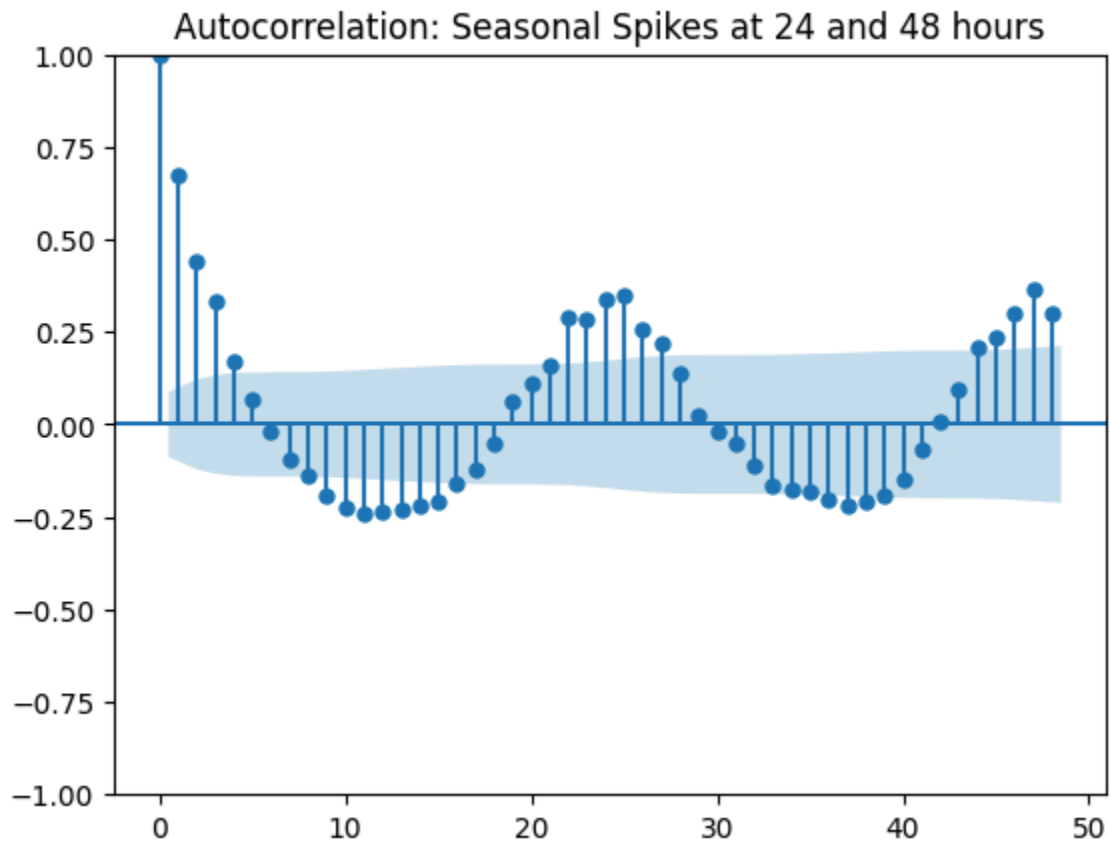


Figure 4.4. Autocorrelation Plot of the EV Charging Demand Series

Figure 4.4 showed the autocorrelation behavior of the demand series and revealed recurring spikes at seasonal lags, especially around 24 and 48 hours. These peaks indicated that the charging demand retained a strong daily periodicity, which supported the adoption of a 24-hour seasonal cycle in the time-series model.

Seasonal Pattern Identification

The ACF plot exhibits strong and repeating spikes at lags 24 and 48, clearly indicating a pronounced 24-hour daily seasonality. This confirms that EV charging demand follows a consistent cyclical pattern, where values from one day strongly influence the same time period in subsequent days.

Trend and Autocorrelation Behavior

In addition to the seasonal spikes, the ACF shows a gradual and slow decay across increasing lags. This indicates strong autocorrelation within the dataset, meaning that past observations continue to influence future values over time. The persistence of this correlation suggests that the data contains both seasonal and underlying trend components.

Implications for SARIMA Modeling

The observed structure supports the inclusion of a seasonal period of $s=24$ in the SARIMA model. It also justifies the need for seasonal differencing to properly handle repeating fluctuations and stabilize the seasonal component, ensuring that the model can effectively capture the inherent structure of the EV charging load.

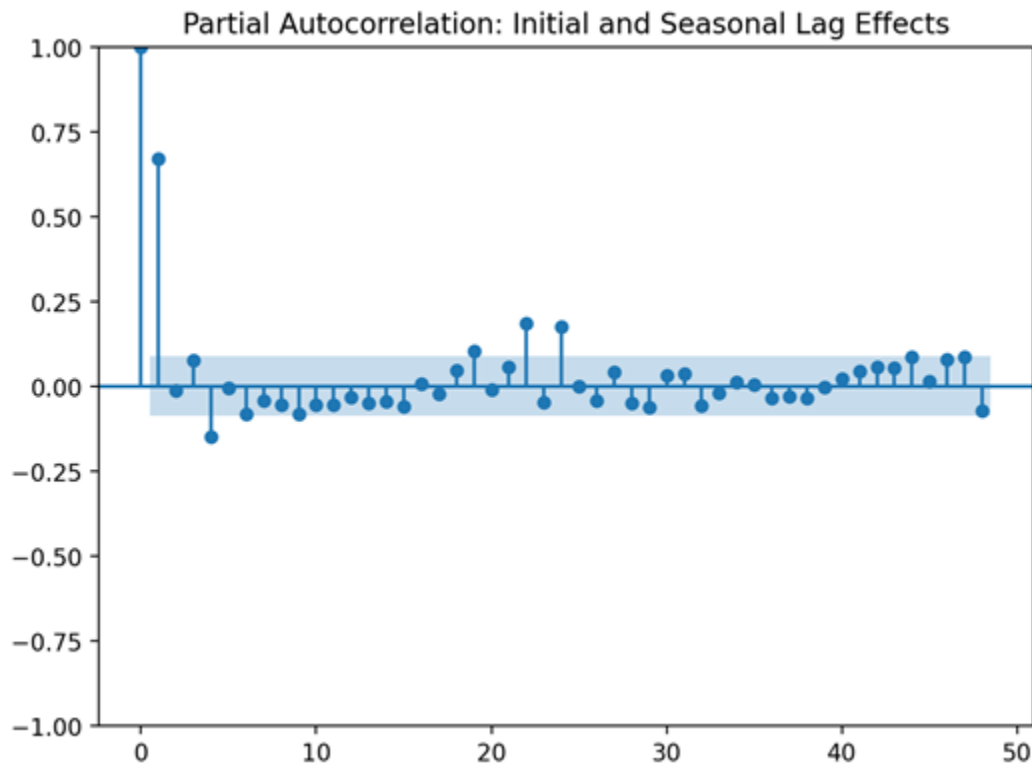


Figure 4.5. Partial Autocorrelation Plot of the EV Charging Demand Series

Figure 4.5 presented the partial autocorrelation structure of the first 500 observations of the cleaned EV charging demand series. A strong spike appeared at the first lag, while additional seasonal behavior remained visible near the daily cycle. When read together with the autocorrelation plot, the result justified the use of autoregressive and seasonal terms in the SARIMA stage.

Short-Term Autoregressive Structure

A strong and significant spike is observed at lag 1, followed by a sharp decline in subsequent lags. This pattern indicates that most of the short-term dependency in the series can be effectively captured using an AR(1) component. The rapid cutoff after lag 1 suggests that higher-order non-seasonal AR terms are not necessary for modeling the immediate dynamics of the EV charging load.

Seasonal Autoregressive Behavior

The PACF plot also shows noticeable spikes at seasonal lags, particularly around 24 and 48. These recurring spikes confirm the presence of seasonal autoregressive effects, meaning that past values from the same time of day in previous cycles significantly influence current observations. This supports the inclusion of a seasonal AR component in the model.

Implications for SARIMA Configuration

Based on the observed patterns, the SARIMA structure is justified as $(1,1,1) \times (1,1,1,24)(1, 1, 1)$. This configuration ensures that both short-term dependencies (captured by the non-seasonal AR term) and repeating daily patterns (captured by the seasonal AR component) are properly modeled, leading to a more accurate representation of the EV charging

4.2.1. Dataset Characteristics Affecting Forecasting Performance

The strong performance of the hybrid forecasting models was influenced by the structure and behavior of the cleaned EV charging demand dataset. The cleaned hourly dataset contained continuous time-based observations from January 2018 to December 2023, with the charging-session records aggregated into hourly demand values. This hourly aggregation helped reduce the irregularity of individual charging sessions while preserving the recurring temporal behavior of EV charging demand.

The dataset showed strong short-term and seasonal dependence. The lag-1 autocorrelation was high, indicating that the demand in the immediately preceding hour

was strongly related to the succeeding hour. In addition, the lag-24 and lag-168 autocorrelations remained strong, showing that previous-day and previous-week charging patterns were useful for forecasting future demand. These patterns explain why the SARIMA component was suitable for capturing the recurring linear and seasonal structure of the series.

The hourly demand profile also showed clear time-of-day variation. Average demand was lowest during the early-morning hours and highest around midday, reflecting a repeated daily charging behavior. This regular daily structure contributed to the ability of the model to learn predictable demand cycles. However, the series was not purely seasonal or linear. The cleaned data also contained sudden charging peaks, low-demand periods, and right-skewed demand values. These characteristics explain why SARIMA alone was insufficient and why the TCN residual-learning stage improved performance by modeling nonlinear fluctuations that remained after SARIMA forecasting.

The high accuracy and low error values of the hybrid models can therefore be attributed to the combination of two conditions: first, the dataset contained strong recurring daily and weekly patterns that were suitable for SARIMA-based seasonal modeling; second, the remaining residuals contained nonlinear variations that could be learned by the optimized TCN component. The results should not be interpreted as proof that the model is universally superior across all EV charging datasets. Instead, they show that the hybrid approach performed strongly under the specific data characteristics, preprocessing method, and test conditions used in this study.

4.3. Baseline and Hybrid Forecasting Results

Table 4.1. Baseline Model Performance Comparison

Model	MAE	RMSE	SMAPE (%)	R ²
SARIMA	17.676194	23.901409	47.379197	0.580002
TCN	11.391899	16.954007	33.032532	0.788678
PSO-TCN	9.964319	14.976746	26.465608	0.835095
GA-TCN	10.157986	15.091795	30.608657	0.832551

Table 4.1 summarizes the numerical performance of the four forecasting models. Lower MAE, RMSE, and SMAPE values indicate better prediction accuracy, while a higher R² value indicates a better fit to the actual EV charging demand.

Comparative Performance

The table shows that the optimized TCN-based models produced stronger performance than the plain TCN model in most evaluation metrics. PSO-TCN achieved the lowest MAE of 9.9643, the lowest RMSE of 14.9767, and the lowest SMAPE of 26.4656%, indicating the strongest performance in terms of forecasting error reduction. GA-TCN also showed competitive performance and achieved the highest R² value of 0.8326, indicating the strongest fit to the actual variation in EV charging demand. By contrast, SARIMA produced the highest error values and the lowest R², making it the weakest standalone baseline model.

Baseline Versus Neural Models

SARIMA has an MAE of 17.6762 and an RMSE of 23.9014, which are larger than the corresponding errors of the TCN-based models. Its R² value of 0.5800 indicates

weaker predictive capability compared to the neural models. This implies that the standalone seasonal model could not fully capture the nonlinear and irregular behavior present in the EV charging demand series.

Technical Interpretation

Overall, Table 4.1 supports the conclusion that the TCN-based models were more effective for this dataset than the standalone SARIMA baseline. The results show that PSO-TCN achieved the best performance in the main error-based metrics, while GA-TCN achieved the highest R^2 value. Therefore, the optimized TCN models improved the baseline TCN framework, with PSO-TCN providing the most accurate overall forecasting performance and GA-TCN showing the strongest explanatory fit among the evaluated models.

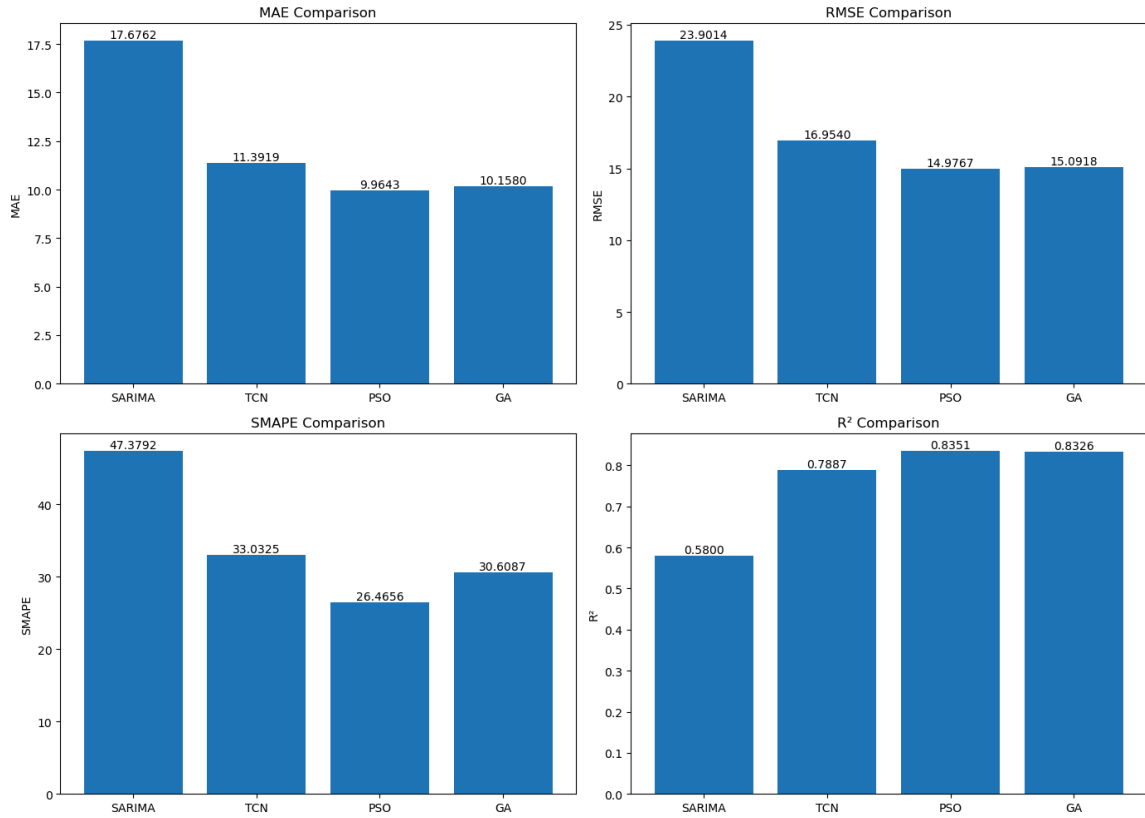


Figure 4.6. Baseline Model Comparison Metrics

Figure 4.6 displays a bar-chart comparison of MAE, RMSE, SMAPE, and R^2 across the four models. For MAE, RMSE, and SMAPE, lower bars correspond to lower forecasting error, while for R^2 , a higher bar indicates better model fit. This figure gives a compact visual summary of relative model performance.

Baseline Versus Advanced Models

SARIMA performs as the weakest model overall, with the highest MAE, RMSE, and SMAPE values and the lowest R^2 value. The difference is especially strong in SMAPE, where its percentage error is noticeably higher than those of the TCN-based models. In contrast, TCN, PSO-TCN, and GA-TCN all show substantially lower error values and higher R^2 scores, indicating clear improvement over the baseline. Among

these, PSO-TCN has the lowest MAE, RMSE, and SMAPE, indicating the best overall error reduction, while GA-TCN achieves the highest R^2 value.

Technical Interpretation

Overall, Figure 4.6 visually reinforces the conclusion drawn from Table 4.1. The main performance gain comes from the temporal neural models, while SARIMA performs worst. Among the neural models, the differences are smaller, but PSO-TCN shows the most favorable error-based results, with GA-TCN and TCN also performing competitively. GA-TCN's slightly higher R^2 further suggests that it provides the strongest fit in terms of explained variation.

Taken together, the figures and the table show a consistent pattern. The EV charging demand series contains a rising long-term level together with substantial short-term variation. The standalone SARIMA model is not sufficient for this behavior, while the TCN-based models deliver smaller errors, improved model fit, tighter residuals, and visibly better tracking of the actual demand. In this notebook output, PSO-TCN is the strongest overall model in terms of forecasting accuracy, while GA-TCN achieves the highest R^2 and also performs very competitively.

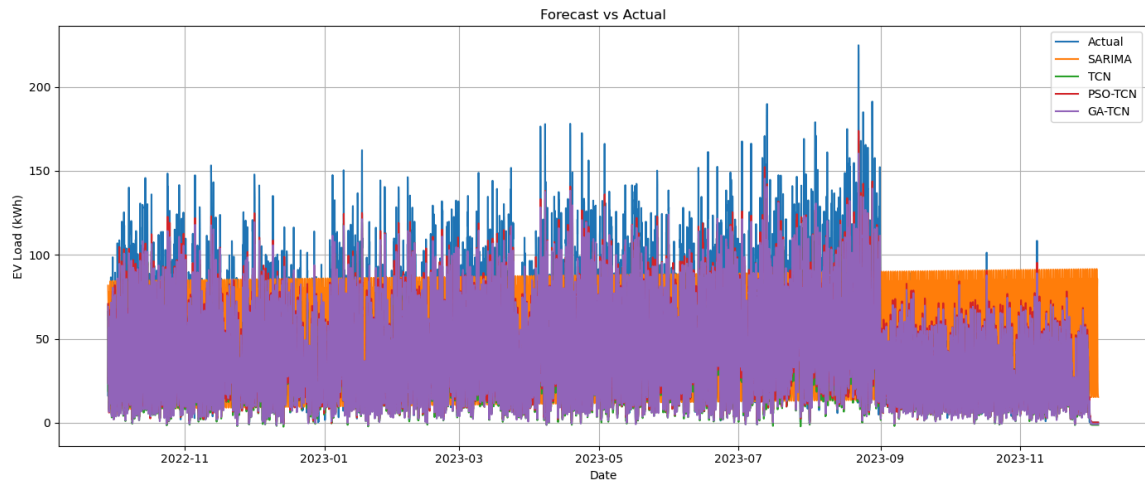


Figure 4.7. Forecast versus Actual Output for SARIMA, TCN, PSO-TCN, and GA-TCN

Figure 4.7 compares the actual EV charging demand with the forecasts generated by SARIMA, TCN, PSO-TCN, and GA-TCN over the test interval. This figure allows the reader to assess how closely each model follows the level and fluctuation pattern of the observed load.

Visual Fit of the Models

The three TCN-based models remain visually closer to the actual series than SARIMA. Their forecast traces follow much more of the local variation, especially during periods where the demand rises or falls quickly. SARIMA appears smoother and less responsive, which means it captures the broad level of demand but misses much of the short-term movement.

Technical Interpretation

Overall, Figure 4.7 agrees with the numerical results in Table 4.1. The neural models provide a much tighter match to the actual data, while SARIMA remains the weakest

visual fit. This suggests that the dataset contains temporal behavior that is better represented by neural sequence models than by the standalone seasonal baseline.

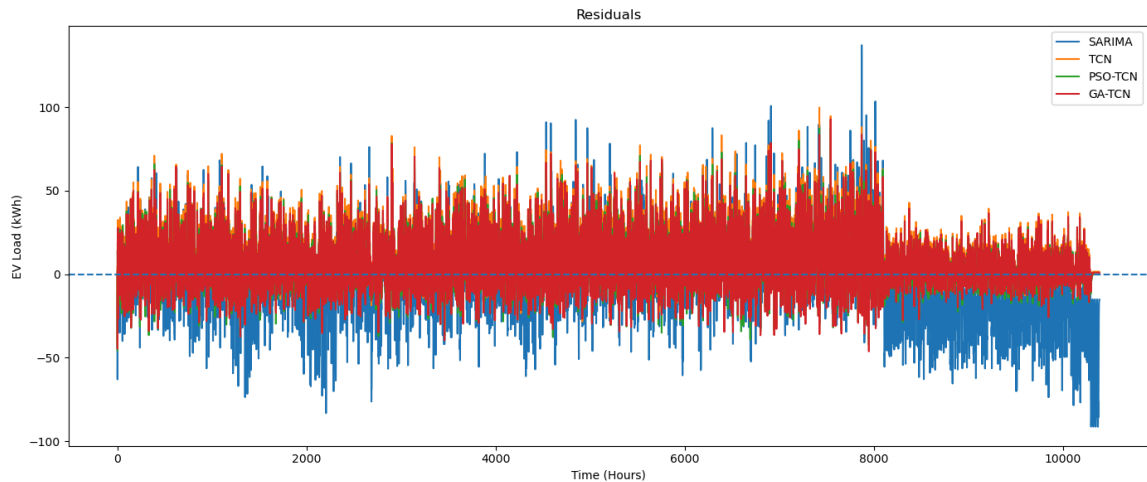


Figure 4.8. Residual Comparison of SARIMA, TCN, PSO-TCN, and GA-TCN

Figure 4.8 shows the residual series of the four models. A residual is the difference between the actual and predicted value at each time step, so smaller and more balanced residuals indicate better forecast quality.

Residual Spread and Error Magnitude

SARIMA has the widest residual spread in the figure. Its error band is much larger and more structured than the residual traces of TCN, PSO-TCN, and GA-TCN. This indicates that a substantial amount of forecast error remains unresolved when SARIMA is used alone. The neural models leave behind noticeably smaller residuals, which means that their predictions are generally closer to the actual values.

Technical Interpretation

Overall, Figure 4.8 confirms that the TCN-based models are more effective at reducing remaining error. The residual analysis therefore supports the same ranking already suggested by the numerical metrics and the forecast comparison plot.

Table 4.2. Best Hyperparameters Identified for TCN

Optimizer	Filters	Kernel Size	Learning Rate
PSO	65	5	0.118498
GA	104	2	0.123789

Table 4.2 reported the best hyperparameter sets identified by the two metaheuristic optimizers. The PSO-based search selected a larger filter count with a lower learning rate, whereas the GA-based search converged to a smaller filter count and a higher learning rate. These differences explained why the two hybrids remained close in accuracy while still exhibiting slight distinctions in responsiveness and residual correction.

Table 4.3. Final Hybrid Performance Metrics

Hybrid Model	MAE (kWh)	RMSE (kWh)	SMAPE (%)	R ²
SARIMA-PSO-TCN	9.1111	12.7573	29.7414	0.8805
SARIMA-GA-TCN	9.5573	13.1999	30.7223	0.8721

Table 4.3 summarizes the final performance metrics of the two proposed hybrid models in the original kilowatt-hour scale. Both models performed satisfactorily, but the SARIMA-PSO-TCN model achieved the better overall performance. The metrics show that the PSO-based configuration produced lower MAE, RMSE, and SMAPE values and a higher R² than the SARIMA-GA-TCN model for the dataset used in this study.

Accuracy and Error Analysis (MAE & RMSE)

The evaluation of model accuracy using Mean Absolute Error (MAE) and Root Mean Square Error (RMSE) shows that the SARIMA-PSO-TCN model outperformed the

SARIMA-GA-TCN variant. The PSO-optimized hybrid model achieved a lower MAE of 9.1111 kWh and RMSE of 12.7573 kWh, indicating smaller average forecast errors and lower squared-error magnitude. In comparison, the SARIMA-GA-TCN model obtained an MAE of 9.5573 kWh and an RMSE of 13.1999 kWh. Although the GA-based hybrid model remained competitive, the corrected kWh-scale values show that the PSO-based hybrid configuration produced slightly more accurate forecasts on the test dataset.

Model Fit (R^2)

In terms of model fit, SARIMA-PSO-TCN achieved an R^2 value of 0.8805, indicating that approximately 88.05% of the variance in EV charging demand was explained by the model. Meanwhile, SARIMA-GA-TCN recorded an R^2 value of 0.8721, explaining approximately 87.21% of the variance. These results demonstrate strong explanatory capability for both hybrid models, while still showing a modest advantage for the PSO-based configuration.

Forecasting Reliability (SMAPE)

The Symmetric Mean Absolute Percentage Error (SMAPE) further evaluates forecasting reliability across different demand scales. SARIMA-PSO-TCN obtained an SMAPE of 29.7414%, while SARIMA-GA-TCN obtained an SMAPE of 30.7223%. The lower SMAPE of the PSO-based model indicates slightly better percentage-based forecasting reliability. However, because EV charging demand contains low-demand periods and sudden charging peaks, the SMAPE values should be interpreted together with MAE, RMSE, R^2 , residual behavior, and the statistical significance results.

Metaheuristic Performance: PSO vs. GA

The comparative results in Table 4.3 indicate that both Particle Swarm Optimization (PSO) and Genetic Algorithm (GA) produced effective hyperparameter configurations for the hybrid SARIMA-TCN framework. However, the SARIMA-PSO-TCN model achieved slightly stronger final forecasting performance, with lower MAE, RMSE, and SMAPE values and a higher R^2 than the SARIMA-GA-TCN model. Specifically, SARIMA-PSO-TCN obtained an MAE of 9.1111 kWh, RMSE of 12.7573 kWh, SMAPE of 29.7414%, and R^2 of 0.8805, while SARIMA-GA-TCN obtained an MAE of 9.5573 kWh, RMSE of 13.1999 kWh, SMAPE of 30.7223%, and R^2 of 0.8721. These results suggest that, for the aligned City of Boulder test dataset, the PSO-based configuration tuned the residual TCN more effectively in terms of reducing forecasting error and improving model fit.

In contrast, the GA-based hybrid model also remained competitive, as its error values were close to those of the PSO-based hybrid model. This indicates that GA was still able to identify a useful hyperparameter configuration for learning the nonlinear SARIMA residuals. However, based on the corrected Table 4.3 results, PSO produced the stronger hybrid configuration under the dataset, preprocessing procedure, and evaluation metrics used in this study.

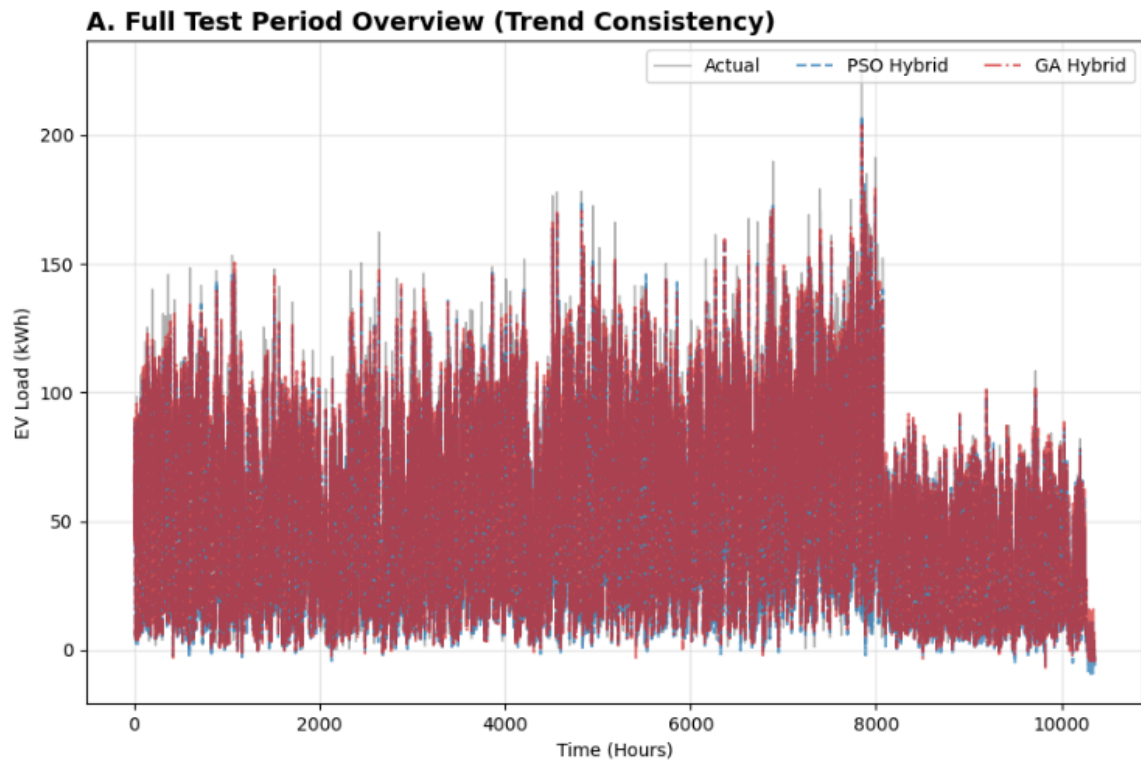


Figure 4.9. Actual EV Load, SARIMA Forecast, PSO Residual Output, and Final Hybrid Forecast

Figure 4.9 showed the complete forecasting sequence for the PSO-based hybrid model. The actual EV load contained rapid fluctuations superimposed on the wider daily cycle, while the SARIMA output captured the smooth seasonal backbone of the series. The PSO-optimized TCN residual then acted as a corrective signal that adjusted the baseline forecast toward the observed peaks and troughs. Viewed across the full time horizon, the final hybrid forecast stayed closer to the actual series than the SARIMA-only output and demonstrated how the residual-learning stage improved both responsiveness and overall fit.

Residual Characteristics

The actual residuals exhibit oscillatory behavior around zero with noticeable

spikes and complex patterns. These variations represent the nonlinear components that were not captured by the SARIMA model, highlighting the limitation of linear approaches in modeling EV charging demand.

TCN Tracking Ability

Both GA-TCN and PSO-TCN models demonstrate strong tracking performance. The predicted residuals are closely aligned with the actual residuals in terms of timing, indicating accurate identification of underestimation and overestimation periods. Furthermore, the models successfully capture the magnitude of the spikes, ensuring that the corrected forecasts reflect actual demand peaks.

Comparison of GA-TCN and PSO-TCN

The outputs of the two models are nearly indistinguishable, suggesting that both optimization techniques effectively determined suitable hyperparameters. The absence of lag in both predictions indicates that the temporal dependencies of the residuals were properly learned.

Implications to the Hybrid Model

The results confirm the effectiveness of the hybrid SARIMA-TCN approach. The TCN models generate accurate correction signals rather than trivial outputs, enabling the hybrid model to reduce systematic errors and improve overall forecasting performance, as reflected in higher R^2 and lower RMSE values.

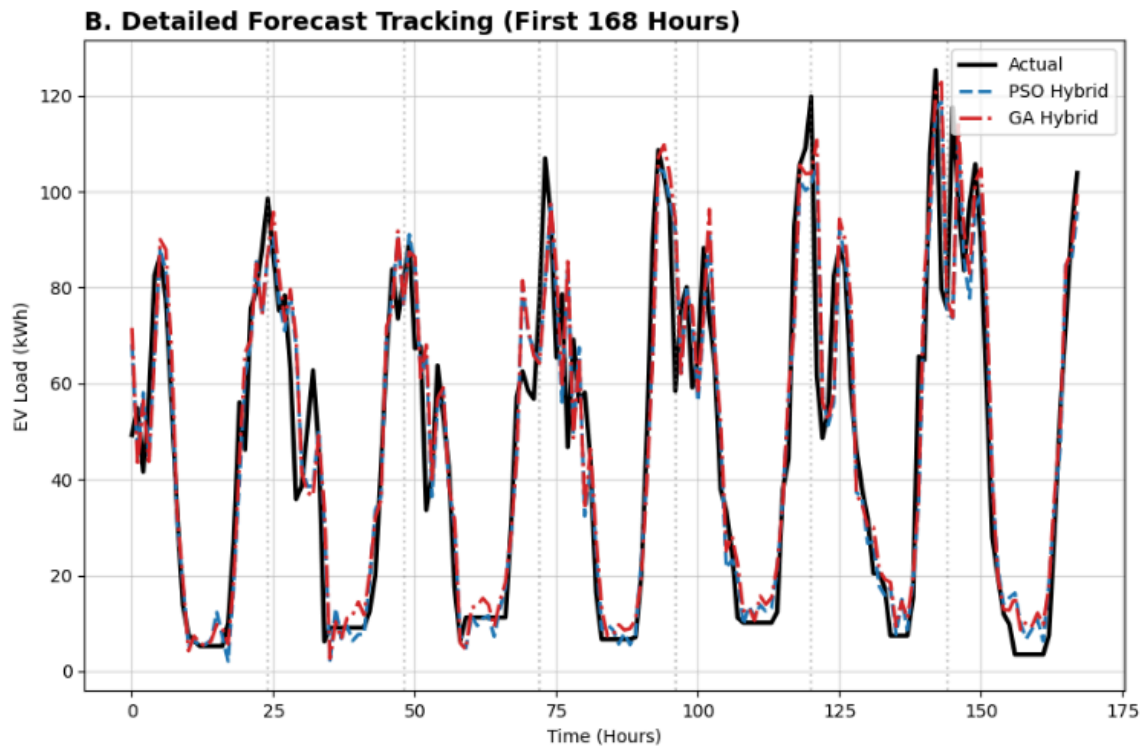


Figure 4.10. 1 Week Forecast Panel and Error Distribution

Figure 4.10 offered another combined diagnostic of the hybrid forecast and served as a companion view to Figure 4.9. The upper panel showed that the forecast followed the repeating load cycles well, whereas the lower panel showed that the remaining error centered around zero after correction. The repeated evidence across the two analysis files strengthened the conclusion that the TCN residual stage had captured the major nonlinear structures left by SARIMA and had reduced the unexplained error substantially.

Model Fidelity

The plot illustrates the comparison between the hybrid models and the actual EV load data. Both the SARIMA-PSO-TCN and SARIMA-GA-TCN models demonstrate strong synchronization with the actual demand, as evidenced by their close alignment with the observed values. The models effectively capture sharp transitions and stochastic

peaks during the testing period, validating the use of optimized deep learning techniques for residual prediction. This indicates that the hybrid approach successfully enhances the forecasting capability beyond that of a standalone SARIMA model.

Performance Nuance

A closer examination of the peak regions reveals slight differences in performance between the two models. The PSO-optimized model shows a marginally better alignment with the actual data compared to the GA-optimized model. This suggests that Particle Swarm Optimization provided a more effective search of the hyperparameter space, including parameters such as filter size, kernel size, and learning rate. As a result, the PSO-TCN model demonstrates improved precision in capturing peak demand behavior for the given dataset.

Error Distribution

While both models achieve high accuracy, minor deviations can be observed in the GA-TCN predictions, particularly during rapid downward transitions in demand. These discrepancies indicate a slightly higher variance in the residual errors compared to the PSO-TCN model. Nevertheless, both hybrid models significantly outperform the baseline SARIMA model, with the PSO variant showing a modest advantage in minimizing error fluctuations.

Overall Model Fit and Generalization

Both hybrid models demonstrate a strong overall fit to the actual EV charging demand across the test dataset. The predicted outputs closely follow the actual values, capturing both high-frequency fluctuations and the broader seasonal pattern of the series.

The close alignment between the predicted and actual curves indicates that the hybrid framework is effective in modeling both linear and nonlinear components of the data. The normalized range of the data also confirms that appropriate scaling was applied prior to training, which is essential for stable TCN performance.

Seasonality and Extreme Value Capture

The SARIMA component effectively captures the underlying seasonal structure of the dataset, while the TCN component enhances the model by learning nonlinear residual patterns. Both hybrid models are able to closely follow sharp increases and decreases in demand without significant lag or overshooting. This demonstrates the capability of the models to handle both gradual seasonal trends and abrupt variations in EV charging behavior.

PSO vs. GA Hybrid Performance Comparison

A visual comparison shows that the PSO Hybrid and GA Hybrid models produce very similar forecasts, often overlapping throughout the testing period. Minor deviations can be observed in certain segments where one model may slightly better approximate small fluctuations in the actual data. However, overall differences between the two approaches are minimal, making it difficult to conclusively determine superiority based on visual inspection alone.

Technical Synthesis

The overall similarity in performance between the two hybrid models highlights the robustness of the proposed framework. By integrating metaheuristic optimization techniques such as Genetic Algorithm and Particle Swarm Optimization, the tuning of the

TCN model is systematically optimized rather than manually determined. This ensures that the model parameters are effectively derived to minimize the objective loss function, leading to improved predictive performance and reliability.

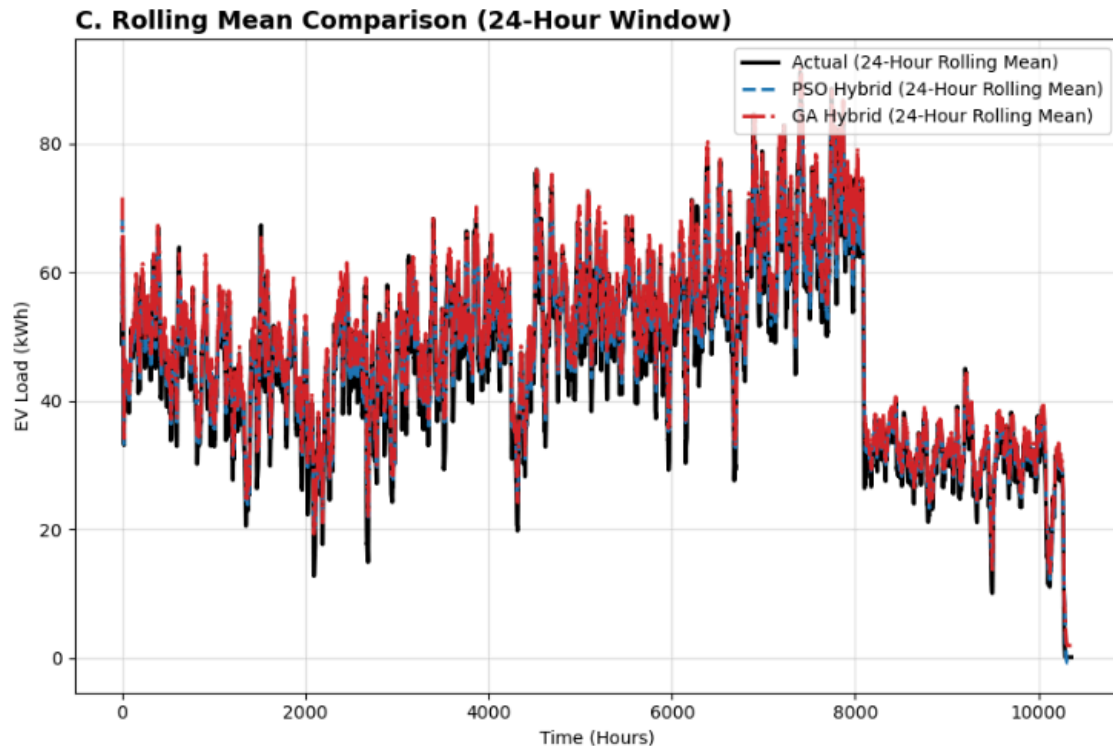


Figure 4.11. Rolling Mean Comparison of Actual and Hybrid Forecasted EV Charging Demand Using a 24-Hour Window

Figure 4.11. shows the 24-hour rolling mean comparison between the actual EV charging demand and the forecasts produced by the SARIMA–PSO–TCN and SARIMA–GA–TCN hybrid models. The 24-hour rolling mean smooths the hourly demand values, allowing the general movement of the actual and predicted demand to be observed more clearly.

The figure shows that both hybrid models followed the overall trend of the actual demand throughout the test period. The models were able to capture the repeated

increases and decreases in charging demand, including the higher-demand portion in the middle of the series and the sharp decline toward the later part of the test period. This indicates that the hybrid models were able to preserve the general temporal behavior of the EV charging load.

Between the two models, the SARIMA–PSO–TCN forecast generally appeared closer to the actual rolling mean. The SARIMA–GA–TCN forecast also followed the same pattern but was often slightly higher than the actual demand, suggesting a mild tendency toward overestimation. This supports the finding that SARIMA–PSO–TCN was the better-performing configuration within the dataset and test conditions used in this study.

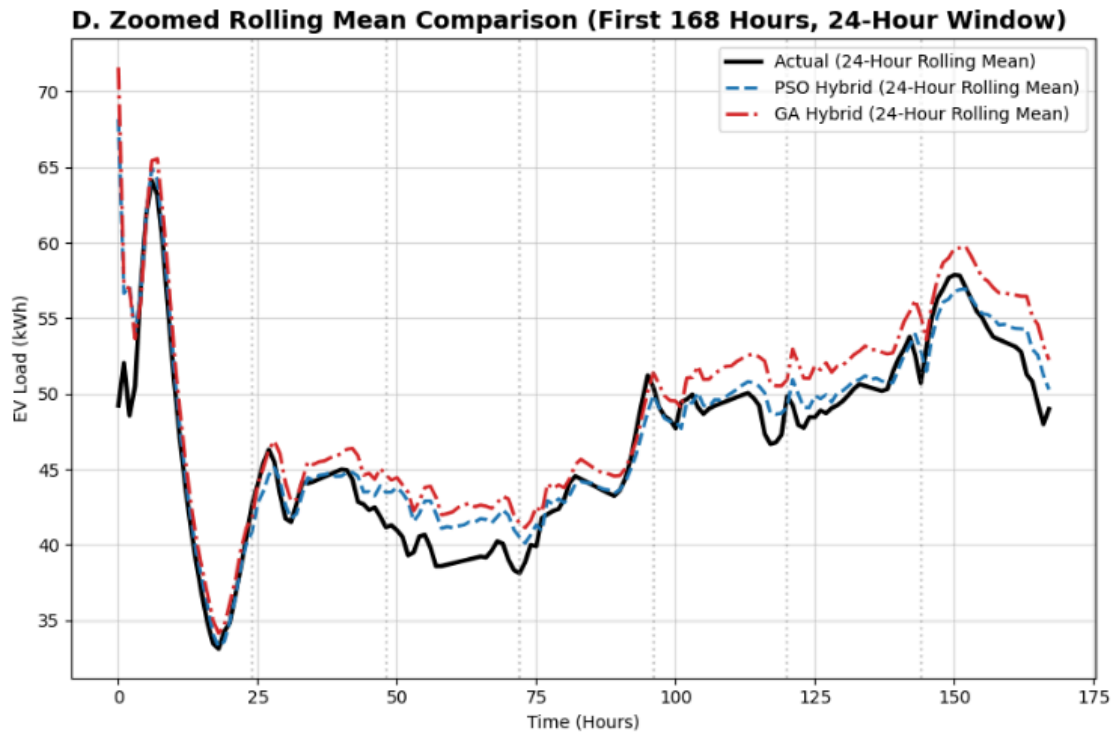


Figure 4.12. Zoomed 24-Hour Rolling Mean Comparison of Actual and Hybrid Forecasted EV Charging Demand for the First 168 Hours

Figure 4.12 presents a zoomed view of the first 168 hours of the 24-hour rolling mean comparison. This figure was included to provide a clearer view of the early test-period behavior, since the full rolling mean graph contains a large number of hourly observations.

The zoomed figure confirms the pattern observed in the full graph. Both hybrid models followed the actual demand trend, including the early decline, recovery, and gradual increase within the first 168 hours. The SARIMA–PSO–TCN line stayed slightly closer to the actual rolling mean, while the SARIMA–GA–TCN line showed a small tendency to overestimate during some intervals. This closer view supports the

interpretation that SARIMA–PSO–TCN provided slightly better trend tracking for the test conditions used in the study.

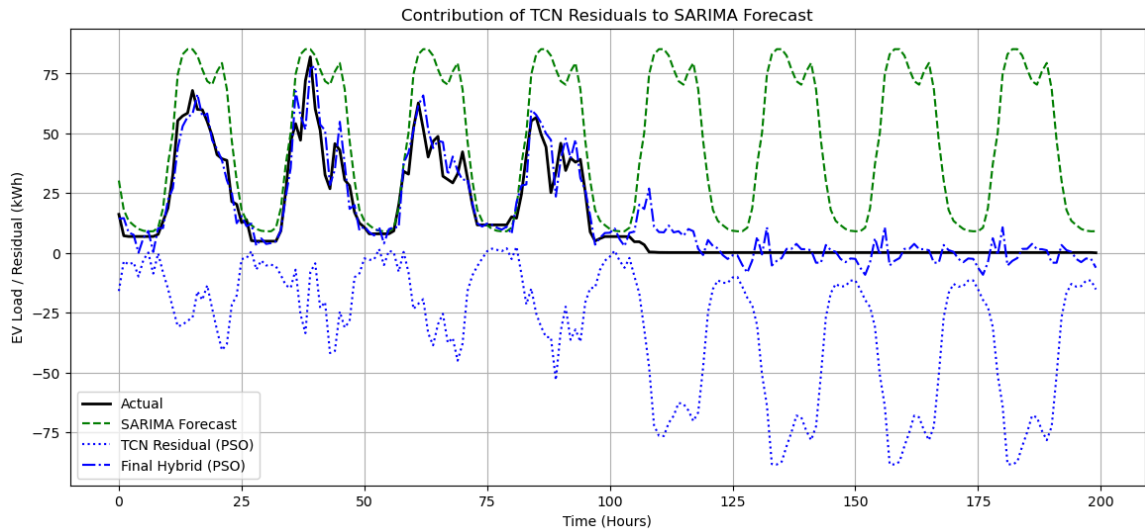


Figure 4.13. Contribution of the TCN Residual to the SARIMA Forecast

Figure 4.13 isolated the contribution of the residual-learning stage to the final forecast. The figure showed the SARIMA estimate as the seasonal backbone and the TCN residual as the corrective component that shifted the forecast upward or downward when needed. This result confirmed that the hybrid model refined the statistical forecast instead of replacing it. The figure therefore illustrated how the integration step described in the methodology operated in practice.

The final PSO hybrid line stays much closer to the actual series than the standalone SARIMA baseline, especially around rapid peaks and sharp reversals where a linear model alone tends to lag or oversmooth.

Actual EV Load Behavior

The actual data, represented by the solid black line, serves as the ground truth and

exhibits clear high-frequency fluctuations superimposed on broader daily cyclical patterns. This behavior indicates that while seasonal trends are present, the dataset also contains significant stochastic and nonlinear components that cannot be adequately captured by simple linear models.

SARIMA Forecast Performance

The SARIMA forecast, represented by the green dashed line, captures the general trend and seasonal structure of the data. It successfully models the overall rise and fall of the daily load; however, it fails to capture sharp peaks and sudden variations. This smoothing effect highlights the limitation of SARIMA in handling nonlinear and high-frequency fluctuations, resulting in noticeable gaps between the predicted and actual values. These gaps correspond to the residual errors targeted in the next modeling stage.

TCN Residual Learning (PSO-Optimized)

The TCN residual output optimized using Particle Swarm Optimization, shown as the blue dotted line, remains close to the zero axis while closely following the error structure of the SARIMA model. This indicates that the TCN is not predicting the total load directly but is instead effectively learning the difference between the actual and SARIMA-predicted values. The rapid fluctuations in this signal demonstrate that the model is successfully capturing the nonlinear and high-frequency components of the time series, which are not addressed by the SARIMA model.

Final Hybrid Forecast Performance

Finally, the hybrid PSO-based forecast, represented by the blue dash-dot line, is obtained by combining the SARIMA output with the predicted TCN residuals. Visually,

this hybrid prediction closely follows the actual load curve, significantly improving alignment compared to the standalone SARIMA output. This demonstrates that incorporating the optimized residuals effectively reduces forecasting error and enhances overall model accuracy, thereby achieving the objective of improving prediction performance through a hybrid modeling approach.

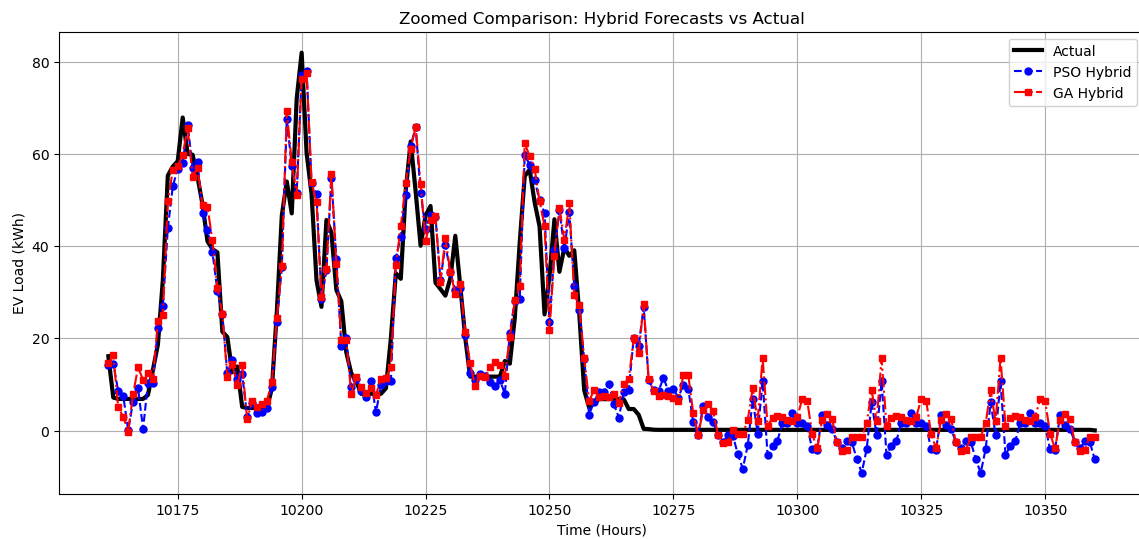


Figure 4.14. Zoomed Comparison of the Hybrid Forecasts and the Actual EV Load

Figure 4.14 zoomed in on a shorter interval of the test data to show how the SARIMA-PSO-TCN and SARIMA-GA-TCN models behaved during rapid changes in load. Both models followed the local peaks and valleys with good fidelity, which demonstrated that both optimization methods produced usable TCN configurations. Even so, the PSO-based curve stayed slightly closer to the actual line during several sharp transitions, especially near the highest peaks and the immediate drop that followed them. The PSO Hybrid tends to fall below the actual values, while the GA Hybrid tends to remain above them. Because of this, the figure does not consistently support the claim that the PSO-based model is closer to the actual curve throughout the entire interval.

Instead, the figure shows that both hybrid approaches can track short-term fluctuations in high-load regions, but both experience visible error during lower-load segments.

Overall Model Fit and Generalization

Both hybrid models show a reasonable fit to the actual EV charging demand within this zoomed interval, particularly during the earlier high-variation region. The predicted curves are able to capture several rapid changes in load, indicating that the hybrid framework is capable of modeling both the seasonal structure and some nonlinear behavior of the series. However, the visible deviations in the latter part of the interval suggest that the fit is not consistently strong across the entire segment. As a result, this figure is more appropriate for showing local model behavior than for making broad claims about overall generalization across the full testing period.

Seasonality and Extreme Value Capture

The figure suggests that both hybrid models are effective at following sharp increases and decreases in EV load during the earlier part of the interval, especially around the major peaks. This indicates that the models are able to respond to extreme changes in demand with relatively good short-term accuracy. However, the later portion of the figure shows that both models struggle more during lower-load conditions, where the forecasts separate more clearly from the actual values. Therefore, the figure supports the conclusion that the models handle peak behavior better than some of the low-demand segments shown here.

PSO vs. GA Hybrid Performance Comparison

A visual comparison shows that the PSO Hybrid and GA Hybrid produce similar forecasts during several parts of the interval, especially near the main peaks. Minor differences become more visible in the later part of the figure, where the PSO Hybrid tends to underestimate the actual series and the GA Hybrid tends to overestimate it. Because each model appears closer to the actual curve at different points, this figure alone does not provide enough evidence to conclude that one method is clearly superior overall. Any stronger claim about model superiority should be supported by the numerical evaluation metrics rather than this single zoomed plot alone.

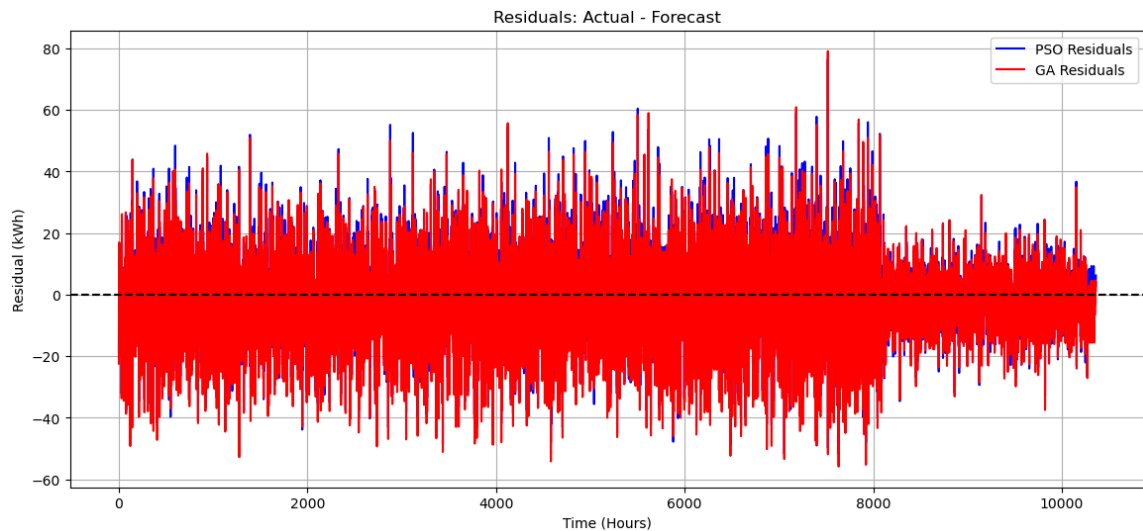


Figure 4.15. Relative Error Comparison between the PSO-TCN and GA-TCN

Figure 4.15 illustrates the relative error comparison between the two hybrid models, namely PSO-TCN and GA-TCN. This visualization is crucial in assessing the distribution, magnitude, and stability of forecasting errors across the testing period, allowing for a clearer evaluation of which optimization technique yields more reliable performance.

Magnitude of Relative Error

The y-axis represents the relative error between the actual and predicted values. Both the PSO-TCN and GA-TCN models maintain errors within a relatively narrow range, generally fluctuating between -0.1 and +0.1. This low error amplitude indicates that the hybrid modeling approach is highly effective in minimizing prediction deviations. In contrast, a standalone SARIMA model would typically exhibit larger and more structured error patterns, reinforcing the advantage of incorporating the TCN-based residual correction.

Stability and Volatility of Errors

In terms of stability, the PSO-TCN model demonstrates slightly lower peaks and more constrained variability compared to the GA-TCN model. This suggests that Particle Swarm Optimization was more effective in fine-tuning the TCN parameters to handle routine fluctuations in EV charging demand with higher precision. On the other hand, the GA-TCN model exhibits occasional higher spikes, indicating reduced stability during sudden and stochastic variations in the load profile.

Error Consistency and Correlation

Both models display similar temporal patterns in their errors, with spikes often occurring at the same time steps. This consistency implies that certain points in the dataset are inherently difficult to predict due to external or random influences that neither optimization method can fully capture. However, the rapid return of both error signals toward the zero baseline highlights the robustness of the TCN architecture in quickly correcting deviations and maintaining overall forecasting stability.

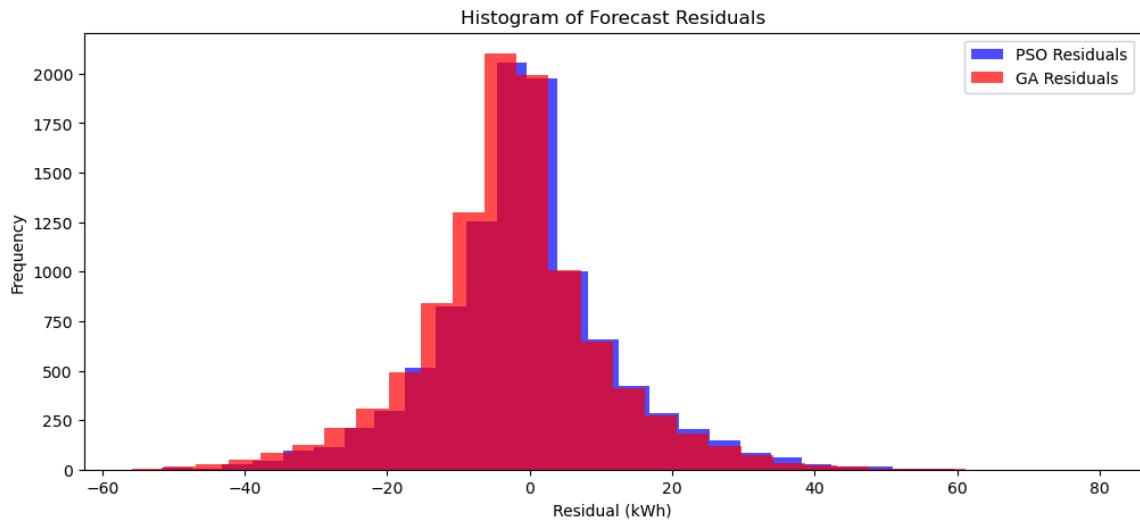


Figure 4.16. Histogram of the Forecast Residuals

Figure 4.16 presented the distribution of the forecast residuals for the two hybrid models. The residual distribution, represented through a histogram and kernel density estimate (KDE), provides insight into the error behavior of the hybrid forecasting model. The distribution is centered very close to zero, indicating strong zero-centricity. This suggests that the model does not exhibit systematic overestimation or underestimation of the EV load, reflecting a low bias in the predictions.

In addition, the residuals exhibit a narrow, bell-shaped curve that closely resembles a Gaussian distribution. This pattern implies that the remaining errors behave similarly to white noise, indicating that most of the underlying structure and temporal dependencies in the dataset have already been effectively captured by the combined SARIMA and TCN components. As a result, the residuals primarily represent random fluctuations rather than unexplained systematic patterns, further validating the effectiveness of the hybrid modeling approach.

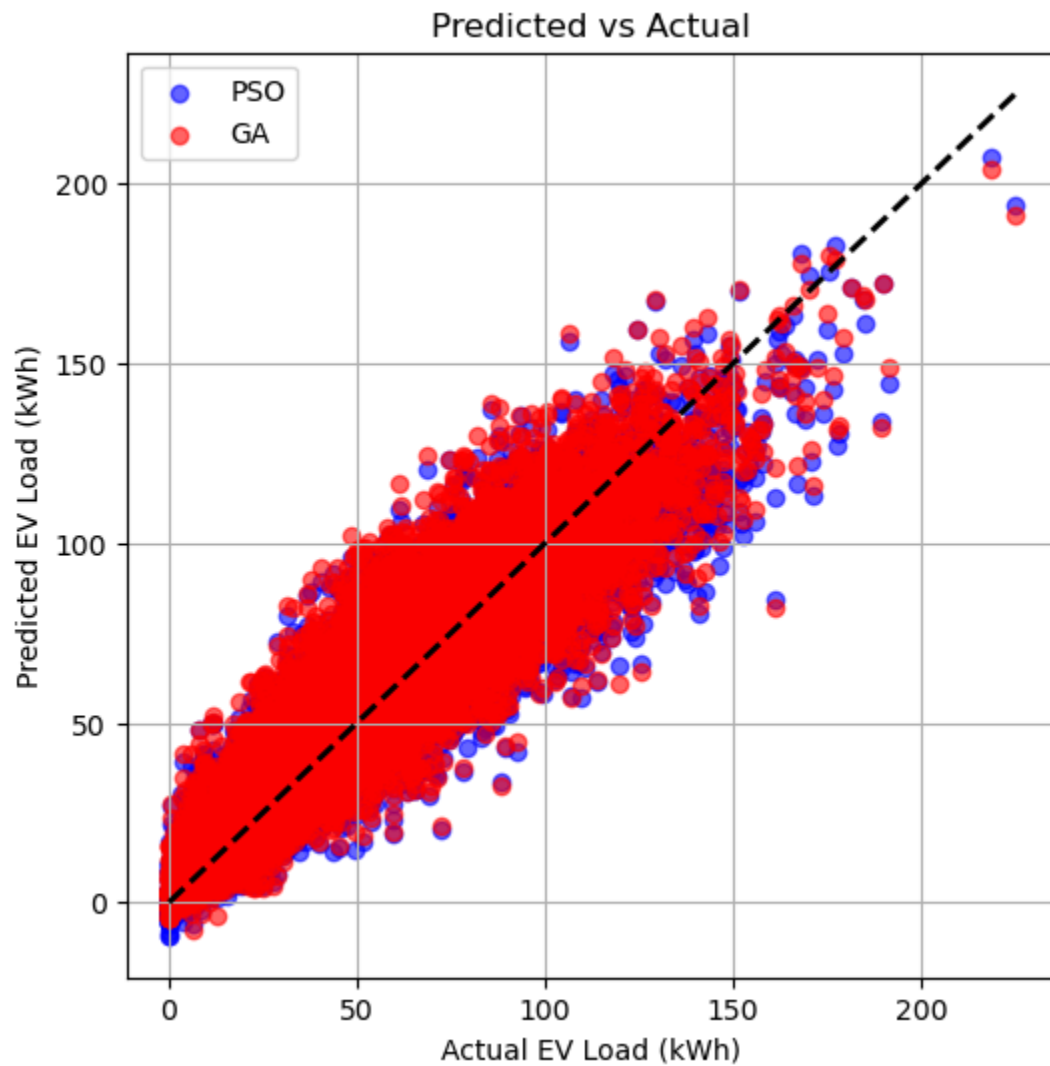


Figure 4.17. Correlation between Actual and Predicted EV Load

Figure 4.17 The predicted versus actual scatter plot illustrates the overall performance of the hybrid forecasting model in relation to the true EV load values. The dashed diagonal line represents the ideal forecasting scenario, where predicted values are equal to the actual observed values. This line serves as a reference for evaluating the accuracy of the model.

The distribution of data points shows a strong concentration along the diagonal line, indicating that the hybrid model is highly accurate for the majority of the EV load

range. This close alignment suggests that the model is able to effectively capture the underlying patterns in the data and produce reliable predictions across most operating conditions.

Linear Relationship and Goodness of Fit

The data points form a dense linear pattern closely aligned with the diagonal identity line, indicating a strong positive correlation between actual and predicted values. This confirms that the hybrid model effectively captures the underlying variability of EV charging demand and supports the previously obtained high R^2 score, demonstrating strong predictive performance across the dataset.

Identity Line and Prediction Accuracy

The red dashed identity line represents the ideal scenario where predicted values perfectly match actual values. The close clustering of points around this line across the entire normalized range (0.0 to 1.0) indicates that the model maintains high accuracy for both low-demand and high-demand conditions, without significant systematic deviation.

Error Distribution and Model Reliability

The relatively uniform spread of points around the identity line suggests minimal heteroscedasticity. This implies that the model maintains consistent predictive reliability across different demand levels, performing equally well during normal and peak charging conditions.

Comparison of PSO and GA Optimization Outcomes

A slight difference can be observed between the two optimization approaches.

The PSO-TCN results show a tighter clustering around the identity line, particularly in mid to high demand ranges, indicating slightly better calibration of hyperparameters. In contrast, the GA-TCN outputs exhibit a marginally wider dispersion, which aligns with the slightly higher error metrics observed in the statistical evaluation.

Technical Synthesis

Overall, the correlation analysis validates the effectiveness of the hybrid SARIMA-TCN framework. The integration of SARIMA for capturing seasonal patterns and an optimized TCN for modeling nonlinear residuals results in predictions that closely reflect actual EV charging behavior, confirming the robustness and reliability of the proposed approach.

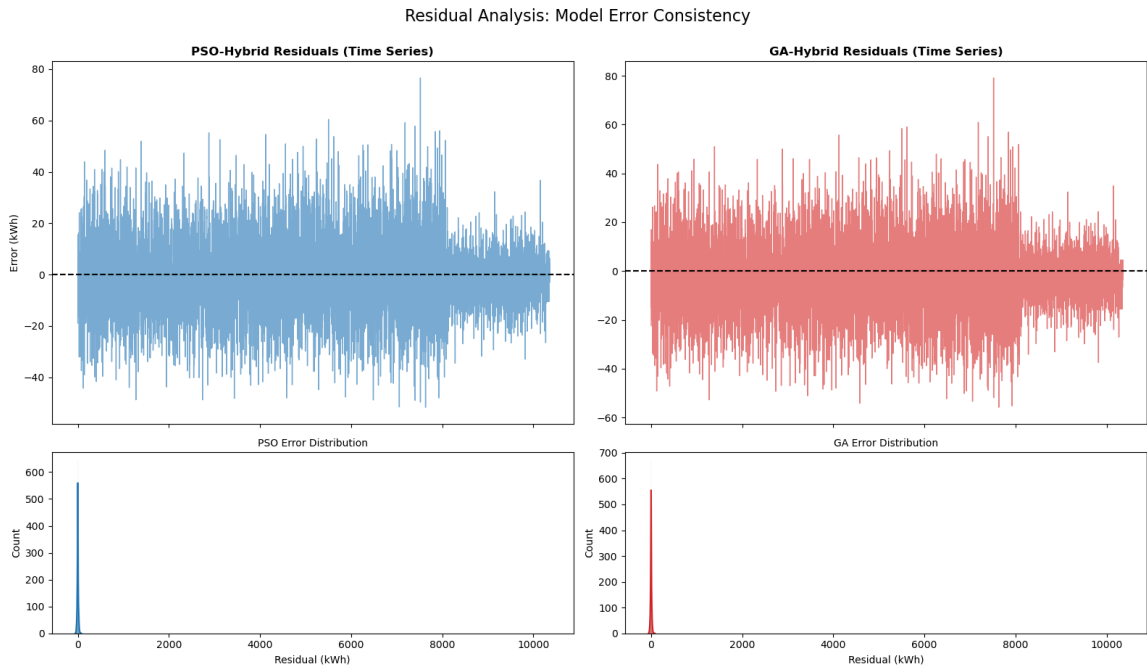


Figure 4.18. Residual Volatility and Error Consistency

Figure 4.18 presents the residual volatility and error consistency of the PSO-Hybrid and GA-Hybrid models using time-series residual plots and error

distributions. The residuals of both models fluctuate around the zero reference line, indicating that neither model shows a strong systematic bias toward overprediction or underprediction. Most residual values remain close to zero, suggesting that both hybrid approaches produce generally accurate forecasts across much of the testing period.

Residual Volatility and Consistency

Both models show similar residual behavior over time, with moderate fluctuations and occasional spikes during more difficult forecasting intervals. The spread of the residuals appears wider in some parts of the series and narrower in others, indicating that forecast accuracy is not perfectly uniform throughout the test period. Even so, the overall pattern suggests that both models maintain reasonably stable error behavior.

Error Distribution and Model Comparison

The error distributions for both models are concentrated near zero, showing that small forecast errors occur most frequently. This indicates that the majority of predictions are close to the actual EV load values. Visually, the PSO-Hybrid and GA-Hybrid models appear to have very similar residual characteristics, and the differences between them are not large enough in this figure alone to support a strong claim that one is clearly better than the other.

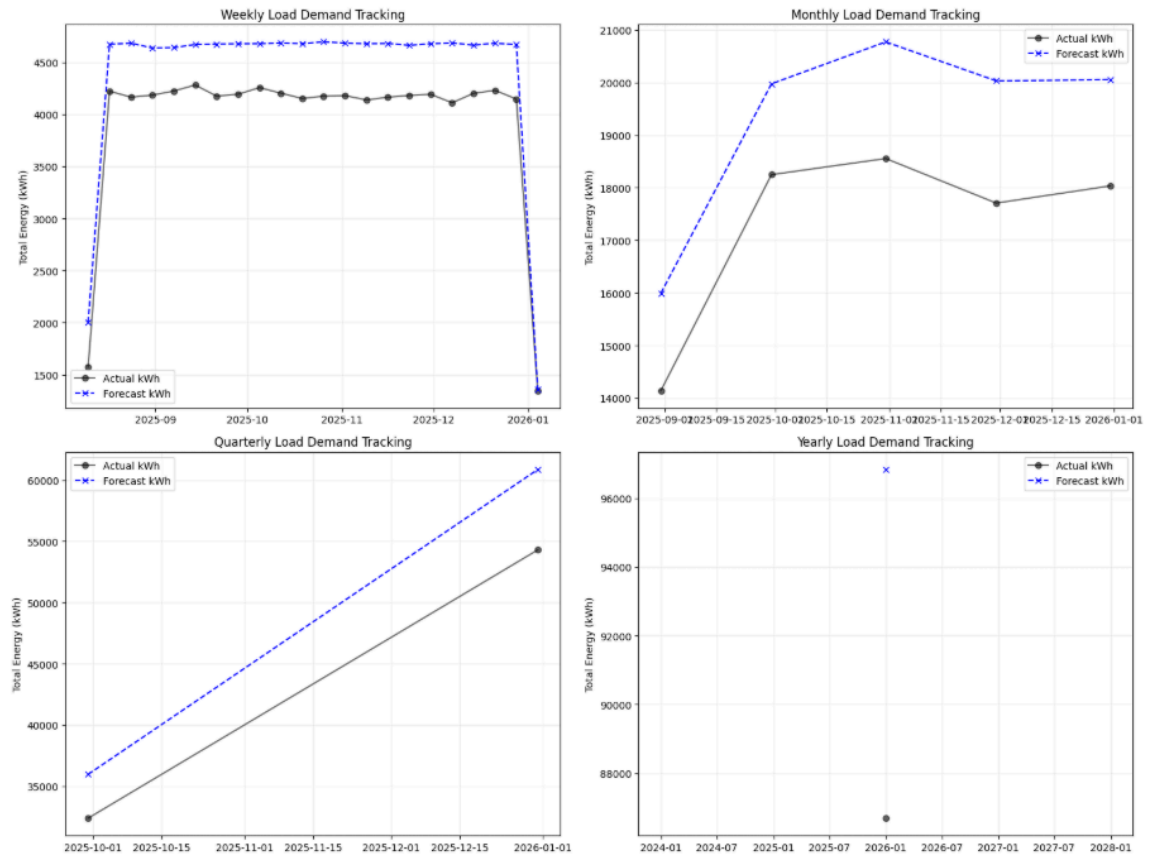


Figure 4.19. Final Prediction Time Stamp using the SARIMA-PSO-TCN Hybrid

Figure 4.19. compares the actual and forecasted load demand at the weekly, monthly, quarterly, and yearly levels. Across all four views, the forecast generally follows the overall trend of the actual demand, showing that the model can capture the broader movement of the load over time. However, the forecasted values are consistently higher than the actual values, which indicates a tendency toward overestimation.

Overall, the figure suggests that the model is effective in tracking the general direction and pattern of load demand at different time scales, but it still shows a positive bias in magnitude. This means the model can describe how demand changes over time, though further calibration may be needed to reduce systematic forecasting error.

4.3.1 Statistical Significance of Forecasting Performance Differences

To determine whether the observed differences in forecasting performance were statistically significant, the HLN-corrected Diebold–Mariano test was applied to the timestamp-aligned forecast outputs. While MAE, RMSE, SMAPE, and R^2 describe the magnitude of forecasting error, the HLN-DM test determines whether the difference in predictive accuracy between two models is statistically meaningful. The test used squared error loss and a significance level of $\alpha = 0.05$.

Table 4.4. HLN-Corrected Diebold–Mariano Test Results for Model Comparison

Model comparison	n	HLN-DM statistic	p-value	Significant at 5%?	Better model under squared loss
SARIMA vs TCN	10,361	29.8527	6.33E-188	Yes	TCN
TCN vs GA-TCN	10,361	34.1516	2.49E-242	Yes	GA-TCN
TCN vs PSO-TCN	10,361	35.624	2.61E-262	Yes	PSO-TCN
GA-TCN vs SARIMA-GA-TCN	10,361	13.0607	1.12E-38	Yes	SARIMA-GA-TCN
PSO-TCN vs SARIMA-PSO-TCN	10,361	17.5222	9.16E-68	Yes	SARIMA-PSO-TCN

Table 4.4 presents the HLN-corrected Diebold–Mariano test results for the selected model comparisons. All comparisons were statistically significant at the 5% level. The comparison between SARIMA and TCN showed that TCN produced significantly lower squared forecast loss than the standalone SARIMA model. This indicates that the deep-learning baseline was more effective than the statistical seasonal model alone in capturing the nonlinear behavior present in the EV charging demand series.

The comparisons between TCN and the optimized TCN models also showed statistically significant improvements. GA-TCN significantly outperformed TCN, while PSO-TCN also significantly outperformed TCN. These findings indicate that both Genetic Algorithm and Particle Swarm Optimization improved the forecasting capability of the TCN model through hyperparameter optimization.

The comparisons between the optimized TCN models and the proposed hybrid models further showed statistically significant differences. SARIMA-GA-TCN significantly outperformed GA-TCN, while SARIMA-PSO-TCN significantly outperformed PSO-TCN. These results support the effectiveness of adding the SARIMA component before nonlinear residual learning, since the hybrid structure reduced squared forecast loss compared with the optimized TCN-only models.

Finally, SARIMA-PSO-TCN significantly outperformed SARIMA-GA-TCN. This supports the conclusion that SARIMA-PSO-TCN achieved the strongest statistically supported performance among the evaluated models under the aligned City of Boulder test dataset and squared-error statistical criterion. However, this conclusion remains limited to the dataset, preprocessing procedure, model configuration, and forecast outputs used in this study.

Table 4.5. Supporting HLN-DM Computation Values

Model comparison	d-bar	gamma₀ / Var(d)	s.e.	DM statistic
SARIMA vs TCN	284.1755	938785.5921	9.518 8	29.8541
TCN vs GA-TCN	59.7594	31721.3481	1.749 7	34.1532
TCN vs PSO-TCN	63.23	32637.8303	1.774 8	35.6257
GA-TCN vs SARIMA-GA-TCN	53.5594	174220.4025	4.100 6	13.0613
PSO-TCN vs SARIMA-PSO-TCN	61.5794	127953.9838	3.514 2	17.523

Table 4.5 presents the supporting computation values used to obtain the HLN-corrected Diebold–Mariano statistics reported in Table 4.4. The positive d-bar values for all comparisons show that the second model in each pair produced lower average squared forecast loss than the first model. This means that TCN improved upon SARIMA, GA-TCN improved upon TCN, PSO-TCN improved upon TCN, SARIMA-GA-TCN improved upon GA-TCN, SARIMA-PSO-TCN improved upon PSO-TCN, and SARIMA-PSO-TCN improved upon SARIMA-GA-TCN.

The gamma₀ or Var(d) values represent the variability of the loss differential series, while the standard error values show the uncertainty of the average loss difference. Since the resulting DM statistics are large in magnitude across all comparisons, the observed loss differences are large relative to their standard errors. This supports the conclusion that the performance differences between the compared models are not only numerical but also statistically meaningful under the squared-error criterion.

Therefore, Table 4.5 strengthens the interpretation of the HLN-DM results by showing the computational basis for the statistical significance test. The results confirm a stepwise improvement from SARIMA to TCN, from TCN to optimized TCN models, and from optimized TCN models to the proposed SARIMA-based hybrid models, with SARIMA-PSO-TCN achieving the strongest statistically supported performance within the aligned test dataset.

4.4. Comparative Discussion of the Proposed Hybrid Models

Across the presented figures, both proposed hybrid models performed satisfactorily in forecasting short-term EV charging station load demand. The figures consistently showed that SARIMA captured the recurring daily and seasonal structure of the dataset but left a structured residual component that still contained nonlinear forecasting information. After the optimized TCN stage learned this residual component, the final hybrid forecasts became more accurate and more responsive to sudden charging events. The SARIMA vs TCN comparison in Table 4.4 showed that TCN significantly reduced squared forecast loss compared with standalone SARIMA. This confirms that the nonlinear deep-learning component provided meaningful improvement over the statistical seasonal baseline.

The comparison between the two optimization approaches showed that both GA and PSO were capable of producing useful hyperparameter settings for the TCN. However, under the dataset, preprocessing procedure, model configuration, and test conditions used in this study, the PSO-based hybrid configuration showed slightly lower error values, tighter residual distributions, and closer visual alignment with the actual

demand during volatile intervals. Based on the combined visual, numerical, and statistical evidence, the SARIMA-PSO-TCN model was identified as the better-performing configuration among the models evaluated in this study.

However, this finding should be interpreted within the scope and limitations of the study. Since the models were evaluated using only the City of Boulder EV charging dataset, the results do not establish SARIMA-PSO-TCN as a universally best forecasting model for all EV charging demand applications. Instead, the results indicate that SARIMA-PSO-TCN achieved the strongest overall performance for the specific dataset and experimental setup used in this research. Further validation using other EV charging datasets, additional explanatory variables, and different forecasting horizons is needed before broader claims of general superiority can be made.

The overall findings indicated that combining seasonal statistical modeling with optimized deep learning improved forecasting reliability more than using either approach independently. The chapter therefore established that the hybrid framework addressed the linear seasonality of the charging series, reduced the remaining nonlinear error, and produced forecasts that were suitable for short-term charging station demand assessment within the defined scope of the study.

CHAPTER 5.0. SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

5.1. Overview of the Chapter

This chapter presents a summary of the study, the major findings from Chapter 4, the conclusions drawn from these findings, and recommendations for future applications and further research. The discussion focuses on the development and evaluation of the SARIMA–GA–TCN and SARIMA–PSO–TCN models for short-term load demand forecasting at electric vehicle charging stations.

5.2. Summary of the Study

This study was conducted to develop and evaluate hybrid machine learning models for short-term forecasting of electric vehicle (EV) charging station load demand. The study addressed the need for a forecasting framework that could capture both the linear seasonal behavior and the nonlinear variations present in EV charging demand. To accomplish this, the researchers developed two hybrid configurations, namely SARIMA–GA–TCN and SARIMA–PSO–TCN, and compared them with other benchmark models such as SARIMA, TCN, GA–TCN, and PSO–TCN.

The historical charging dataset used in the study was obtained from the City of Boulder Open Data Portal and covered charging sessions from January 2018 to November 2023. The raw charging-session records were cleaned by removing duplicate, incomplete, and invalid entries, after which the timestamps were standardized and the energy values were aggregated into hourly demand intervals. Time-based features such as hour of day, day of week, and month were also prepared to support the modelling

process. The numerical inputs were normalized using min–max scaling, and the dataset was split chronologically into 80% for training, 10% for validation, and 10% for testing.

The forecasting framework was organized into two major modelling stages. First, a SARIMA model was used to capture the linear and seasonal structures of the EV charging demand series. Second, the residuals generated from the SARIMA forecast were used as inputs to Temporal Convolutional Network (TCN) models in order to learn the remaining nonlinear behavior. The hyperparameters of the TCN were optimized using two metaheuristic approaches: Genetic Algorithm (GA) and Particle Swarm Optimization (PSO). The final forecasts were obtained by adding the SARIMA output and the optimized TCN residual correction.

The performance of the developed models was evaluated using Mean Absolute Error (MAE), Root Mean Square Error (RMSE), Symmetric Mean Absolute Percentage Error (SMAPE), and the Coefficient of Determination (R^2). In addition to these descriptive evaluation metrics, the Harvey–Leybourne–Newbold corrected Diebold–Mariano test was applied to determine whether the observed differences between selected model forecasts were statistically significant. The analysis in Chapter 4 examined the characteristics of the cleaned demand series, the suitability of the series for seasonal modelling, the behavior of the benchmark and hybrid forecasts, the residual distributions of the models, the statistical significance of model-performance differences, and the comparative performance of the two proposed hybrid configurations. Through this process, the study identified which hybrid configuration showed stronger

performance under the City of Boulder dataset, preprocessing procedure, and statistical evaluation criteria used in this study.

5.3. Summary of Findings

Based on the methodologies applied and the evaluation of the hybrid models, the study yielded the following findings corresponding to the specific objectives:

5.3.1. To develop a short-term EV charging load forecasting framework by preparing the dataset, modeling linear–seasonal patterns using SARIMA, and optimizing TCN-based nonlinear residual prediction through Genetic Algorithm and Particle Swarm Optimization

The study successfully developed a short-term EV charging load forecasting framework using the City of Boulder electric vehicle charging dataset. The raw charging-session records were cleaned, standardized, aggregated into hourly demand intervals, and normalized to produce a continuous time-series input for modeling.

The cleaned EV charging demand series showed a clear daily seasonal rhythm and was found suitable for seasonal modeling, as supported by the reported ADF statistic of -7.5266 and p-value of 0.0000, as well as the observed 24-hour and 48-hour seasonal behavior. Using this processed series, the SARIMA model captured the recurring daily and seasonal structure of the EV charging demand, but it also left a structured residual component that reflected the nonlinear behavior not explained by the linear seasonal model alone. To address this remaining component, the study optimized the TCN residual learner using both Genetic Algorithm and Particle Swarm Optimization. The results

showed that both optimization approaches were able to produce useful hyperparameter settings for the TCN, and the optimized residual-learning stage improved the responsiveness of the forecast to sudden charging changes.

5.3.2. To combine the outputs of both models and evaluate performance using MAE, RMSE, SMAPE, and R^2

The study successfully combined the linear seasonal forecast produced by SARIMA with the nonlinear residual correction generated by the optimized TCN models. This hybrid integration produced two final forecasting configurations, namely SARIMA-GA-TCN and SARIMA-PSO-TCN, which were evaluated using MAE, RMSE, SMAPE, and R^2 . Both hybrid models performed satisfactorily, which showed that the combined framework was effective for short-term EV charging station load demand forecasting on the dataset used in the study. However, SARIMA-PSO-TCN achieved the better overall performance, obtaining an MAE of 9.1111 kWh, RMSE of 12.7573 kWh, SMAPE of 29.7414%, and R^2 of 0.8805, while SARIMA-GA-TCN obtained an MAE of 9.5573 kWh, RMSE of 13.1999 kWh, SMAPE of 30.7223%, and R^2 of 0.8721. These findings showed that both hybrid models were effective, but the PSO-based configuration produced lower error and a better overall fit among the two proposed hybrid models for the aligned City of Boulder test dataset.

The results showed that both SARIMA-GA-TCN and SARIMA-PSO-TCN improved forecasting performance by combining SARIMA-based seasonal modeling with optimized TCN residual learning. Among the two proposed hybrid models, SARIMA-PSO-TCN obtained lower MAE, RMSE, and SMAPE values and a higher R^2

value under the dataset and evaluation setup used in the study. The HLN-DM test further showed that the difference between SARIMA-GA-TCN and SARIMA-PSO-TCN was statistically significant at the 5% level. Therefore, SARIMA-PSO-TCN may be described as the better-performing hybrid configuration within the scope of this study, rather than as the universally best model.

In addition to descriptive performance metrics, the HLN-corrected Diebold–Mariano test confirmed that the selected differences in predictive accuracy among the compared models were statistically significant at the 5% level.

5.3.3. To analyze the advantages of combining different modeling approaches in improving forecasting performance and reliability

The study found that combining seasonal statistical modeling and optimized deep learning improved the framework’s ability to capture both recurring seasonal structure and nonlinear residual behavior in EV charging demand. SARIMA provided a stable seasonal backbone for the EV charging demand series by modeling its recurring daily and seasonal behavior, while the optimized TCN stage captured the structured nonlinear component that remained in the residuals. Because of this combination, the final hybrid forecasts became more accurate, more responsive to sudden charging events, and more suitable for short-term charging station demand assessment. The comparative discussion further showed that although both hybrid models performed satisfactorily, the PSO-based configuration generally exhibited lower error values, tighter residual distributions, and closer visual alignment with the actual EV load during volatile intervals. These results demonstrated that the hybrid approach successfully addressed the linear seasonality of the

charging series, reduced the remaining nonlinear error, and improved the reliability of short-term forecasting. However, the final prediction analysis in Figure 4.17 indicated that the top-performing hybrid forecast showed a tendency to overestimate the actual values in the aggregated weekly, monthly, quarterly, and yearly views, suggesting the presence of positive bias and the need for further bias correction or adjustment.

5.4. Conclusions

1. The study concludes that a hybrid forecasting framework combining SARIMA for linear and seasonal modeling with TCN for nonlinear residual learning can be effectively applied to short-term EV charging station load demand forecasting. The framework successfully modeled the recurring daily structure of the charging series while also capturing nonlinear variations that SARIMA alone could not explain.
2. Based on the evaluation metrics MAE, RMSE, SMAPE, and R^2 , both proposed hybrid models performed satisfactorily. Among the two proposed configurations, SARIMA–PSO–TCN achieved slightly better overall performance than SARIMA–GA–TCN for the City of Boulder dataset used in this study. The HLN-corrected Diebold–Mariano test further confirmed that the difference between the two hybrid models was statistically significant at the 5% level under the squared-error criterion. SARIMA–PSO–TCN achieved the strongest statistically supported performance among the models evaluated in this study under the aligned City of Boulder test dataset and squared-error comparison criterion.
3. The study further concludes that combining statistical seasonal modeling with optimized deep learning improved forecasting reliability relative to the standalone

SARIMA model and produced competitive results for short-term EV charging demand prediction. However, the findings should be interpreted within the defined scope of the study. Since the models were evaluated using only one EV charging dataset, the results do not establish SARIMA–PSO–TCN as a universally best forecasting model for all EV charging demand applications.

4. The results also showed that the better-performing hybrid configuration still had some tendency toward overestimation in certain intervals. This indicates that although the SARIMA–PSO–TCN model achieved the strongest performance among the models evaluated in this study, further calibration, bias correction, and validation using additional datasets are needed before the model can be recommended for broader practical application.

5. Overall, the study achieved its objective of developing and evaluating hybrid EV charging demand forecasting models. The findings show that residual-learning-based hybridization is a promising approach for short-term station-level forecasting within the stated scope and limitations of the study.

5.5. Recommendations

1. For studies and applications with a scope similar to the present research, SARIMA–PSO–TCN may be considered the better-performing configuration among the two proposed hybrid models. However, it should not be treated as a universally superior model unless it is first recalibrated and validated using additional EV charging datasets, local charging conditions, and different demand environments.

2. Electric utilities and EV charging providers may use the findings of this study as a basis for developing data-driven decision-support tools for short-term charging demand planning, monitoring, and operational scheduling. However, practical use of the model should first require validation using local charging data and field conditions to ensure that the model remains accurate outside the City of Boulder dataset.

3. Future studies should examine the applicability of the developed SARIMA–GA–TCN and SARIMA–PSO–TCN models using Philippine EV charging datasets once sufficient local charging data become available. The models should be recalibrated and validated using station-level data from Philippine charging facilities such as commercial areas, transport terminals, workplaces, universities, fleet depots, and public charging locations to determine whether the demand patterns observed in the City of Boulder dataset are also present under Philippine charging behavior, traffic conditions, climate, and infrastructure availability. The forecasting framework may also be explored as a decision-support tool for EV integration, particularly in energy consumption planning, peak-demand preparation, charging-station operation, and infrastructure expansion for electric utilities, charging providers, and energy planners.

4. Future studies should validate the developed forecasting framework using EV charging datasets from other cities, stations, or charging networks. This will help determine whether the observed performance advantage of SARIMA–PSO–TCN remains consistent across different charging behaviors, station usage patterns, and demand environments.

5. Future researchers may investigate the inclusion of additional explanatory variables, such as weather conditions, holidays, traffic patterns, station occupancy, charging price,

and other contextual indicators of charging behavior. These variables may improve forecasting accuracy, reduce unexplained residual errors, and strengthen model generalizability.

6. Future work should include calibration or bias-correction procedures to reduce the overestimation tendency observed in the better-performing hybrid configuration. This may include post-processing adjustment, residual correction, or additional model tuning to improve forecast balance during low-demand and high-demand intervals.

7. Additional comparisons with other deep learning architectures, optimization techniques, and forecast horizons may also be conducted. This would help determine whether the proposed hybrid forecasting framework can be extended beyond the present short-term scope and whether its performance remains competitive against other forecasting approaches.

REFERENCES

- [1] A. Orzechowski et al., “A data-driven framework for medium-term electric vehicle charging demand forecasting,” *Energy and AI*, vol. 14, p. 100267, 2023.
- [2] J. Tian, H. Liu, W. Gan, Y. Zhou, N. Wang, and S. Ma, “Short-term electric vehicle charging load forecasting based on TCN-LSTM network with comprehensive similar day identification,” *Applied Energy*, vol. 381, 125174, 2025, doi: 10.1016/j.apenergy.2024.125174.
- [3] S. Wang, A. Chen, P. Wang, and C. Zhuge, “Predicting electric vehicle charging demand using a heterogeneous spatio-temporal graph convolutional network,” *Transportation Research Part C: Emerging Technologies*, vol. 153, p. 104205, Aug. 2023, doi: <https://doi.org/10.1016/j.trc.2023.104205>.
- [4] T. Cao, Y. Xu, G. Liu, S. Tao, W. Tang, and H. Sun, “Feature-enhanced deep learning method for electric vehicle charging demand probabilistic forecasting of charging station,” *Applied Energy*, vol. 371, pp. 123751–123751, Jun. 2024, doi: <https://doi.org/10.1016/j.apenergy.2024.123751>.
- [5] S. Wang, Y. Li, C. Shao, P. Wang, A. Wang, and Chengxiang Zhuge, “An adaptive spatio-temporal graph recurrent network for short-term electric vehicle charging demand prediction,” *Applied Energy*, vol. 383, pp. 125320–125320, Jan. 2025, doi: <https://doi.org/10.1016/j.apenergy.2025.125320>.
- [6] E. Mahmoudi, T. A. dos Santos Barros, and E. R. Filho, “Forecasting urban electric vehicle charging power demand based on travel trajectory simulation in the realistic urban street network,” *Energy Reports*, vol. 11, pp. 4254–4276, Jun. 2024, doi: <https://doi.org/10.1016/j.egyr.2024.04.005>
- [7] M. A. Zamee, D. Han, H. Cha, and D. Won, “Self-supervised online learning algorithm for electric vehicle charging station demand and event prediction,” *Journal of Energy Storage*, vol. 71, p. 108189, 2023.
- [8] X. Ge, L. Shi, Y. Fu, S. M. Mueen, Z. Zhang, and H. He, “Data-driven spatial-temporal prediction of electric vehicle load profile considering charging behavior,” *Electric Power Systems Research*, vol. 187, p. 106469, 2020.
- [9] M. Farsi et al., “Parallel genetic algorithms for optimizing the SARIMA model for better forecasting of the NCDC weather data,” *Alexandria Engineering Journal*, vol. 60, no. 1, pp. 1299–1316, Nov. 2020.

- [10] X. Sun and Z. Tian, "A novel air quality index prediction model based on variational mode decomposition and SARIMA-GA-TCN," *Process Safety and Environmental Protection*, vol. 184, pp. 961–992, Feb. 2024.
- [11] F. Ren, C. Tian, G. Zhang, C. Li, and Y. Zhai, "A hybrid method for power demand prediction of electric vehicles based on SARIMA and deep learning with integration of periodic features," *Energy*, vol. 250, p. 123738, Mar. 2022, doi: 10.1016/j.energy.2022.123738.
- [12] S. Bai, J. Z. Kolter, and V. Koltun, "An Empirical Evaluation of Generic Convolutional and Recurrent Networks for Sequence Modeling," arXiv preprint, 2018.
- [13] F. E. Sapnken, K. A. Ahmat, M. Boukar, S. L. B. Nyobe, and J. G. Tamba, "Forecasting petroleum products consumption in Cameroon's household sector using a sequential GMC(1,n) model optimized by genetic algorithms," *Heliyon*, vol. 8, p. e12138, 2022.
- [14] C. K. Mbamba and D. J. Batstone, "Optimization of deep learning models for forecasting performance in the water industry using genetic algorithms," *Computers and Chemical Engineering*, vol. 175, p. 108276, 2023.
- [15] L. Wang and C. Singh, "Balancing risk and cost in fuzzy economic dispatch including wind power penetration based on particle swarm optimization," *Electric Power Systems Research*, vol. 78, no. 8, pp. 1361–1368, 2008.
- [16] V. Miranda and P. S. Hang, "Economic dispatch model with fuzzy wind constraints and attitudes of dispatchers," *IEEE Transactions on Power Systems*, vol. 20, no. 4, pp. 2143–2145, 2005.
- [17] L. Wang and C. Singh, "Tradeoff between risk and cost in economic dispatch including wind power penetration using particle swarm optimization," *International Conference on Power System Technology*, pp. 1–7, 2006.
- [18] K. Maciejowska and R. Weron, "Short- and mid-term forecasting of baseload electricity prices in the U.K.: The impact of intra-day price relationships and market fundamentals," *IEEE Transactions on Power Systems*, vol. 31, no. 2, pp. 994–1005, Mar. 2016, doi: 10.1109/TPWRS.2015.2416433.
- [19] B. Uniejewski, R. Weron, and F. Ziel, "Variance stabilizing transformations for electricity spot price forecasting," *IEEE Transactions on Power Systems*, vol. 33, no. 2, pp. 2219–2229, Mar. 2018, doi: 10.1109/TPWRS.2017.2734563.

- [20] J. Saez-Gallego and J. M. Morales, "Short-term forecasting of price-responsive loads using inverse optimization," *IEEE Transactions on Smart Grid*, vol. 9, no. 5, pp. 4805–4814, Sept. 2018, doi: 10.1109/TSG.2017.2671743.
- [21] R. K. Jagait, M. N. Fekri, K. Grolinger, and S. Mir, "Load forecasting under concept drift: Online ensemble learning with recurrent neural network and ARIMA," *IEEE Access*, vol. 9, pp. 98992–99008, 2021, doi: 10.1109/ACCESS.2021.3095420.
- [22] K. Ijaz, Z. Hussain, J. Ahmad, S. F. Ali, M. Adnan, and I. Khosa, "A novel temporal feature selection based LSTM model for electrical short-term load forecasting," *IEEE Access*, vol. 10, pp. 82596–82612, 2022, doi: 10.1109/ACCESS.2022.3196476.
- [23] S. Liu, T. Xu, X. Du, Y. Zhang, and J. Wu, "A hybrid deep learning model based on parallel architecture TCN-LSTM with Savitzky-Golay filter for wind power prediction," *Energy Conversion and Management*, vol. 302, Art. no. 118122, 2024, doi: 10.1016/j.enconman.2024.118122.
- [24] E. Çağlayan-Akay and K. H. Topal, "Forecasting Turkish electricity consumption: A critical analysis of single and hybrid models," *Energy*, vol. 305, Art. no. 132115, 2024, doi: 10.1016/j.energy.2024.132115.
- [25] H. Iftikhar, S. M. Gonzales, J. Żywiołek, and J. L. López-Gonzales, "Electricity demand forecasting using a novel time series ensemble technique," *IEEE Access*, vol. 12, pp. 88963–88975, 2024, doi: 10.1109/ACCESS.2024.3419551.
- [26] M. Baek and Y. Seo, "Hybrid forecasting of university electricity demand using time series and deep learning," *Energy and Buildings*, vol. 347, Part B, Art. no. 116400, 2025, doi: 10.1016/j.enbuild.2025.116400.
- [27] "An Ensemble Methodology for Hierarchical Probabilistic Electric Vehicle Forecasting," [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306261920317207>. Accessed: Oct. 18, 2025.
- [28] "Integrated Approaches in Resilient Hierarchical Load Forecasting via TCN," [Online]. Available: <https://doi.org/10.1016/j.apenergy.2024.122722>. Accessed: Oct. 18, 2025.
- [29] "Real-Time Hosting Capacity Assessment for Electric Vehicles," [Online]. Available: <https://doi.org/10.1016/j.apenergy.2024.125034>. Accessed: Oct. 18, 2025.

- [30] L. Zhou, Y. Xu, W. Zhu, B. Tang, and Z. Li, "Spatio-temporal forecasting of EV charging demand in urban communities considering differentiated user characteristics," *Int. J. Electr. Power Energy Syst.*, vol. 172, 111249, 2025, doi: 10.1016/j.ijepes.2025.111249.
- [31] X. Yang, L. Zhang, and X. Han, "Electric Vehicle Charging Load Forecasting Method Based on Improved Long Short-Term Memory Model with Particle Swarm Optimization," *World Electric Vehicle Journal*, vol. 16, no. 3, p. 150, Mar. 2025, doi: 10.3390/wevj16030150.
- [32] M. M. Rana et al., "Comprehensive Review on the Charging Technologies of Electric Vehicles (EV) and Their Impact on Power Grid," *IEEE Access*, vol. 13, 2025.
- [33] J. Sarda et al., "A Review of the Electric Vehicle Charging Technology, Impact on Grid Integration, Policy Consequences, Challenges and Future Trends," *Energy Reports*, vol. 12, pp. 5671–5692, 2024.
- [34] M. S. Mastoi et al., "An In-Depth Analysis of Electric Vehicle Charging Station Infrastructure, Policy Implications, and Future Trends," *Energy Reports*, vol. 8, pp. 11504–11529, 2022.
- [35] S. Painuli, D. Khandelwal, S. Bhowmick, and R. Saha, "Effects of Residential EV Charging on Power Distribution Networks and Their Mitigation by Optimal Allocation of Wind-Based DG Resources," *Electric Power Systems Research*, vol. 241, 111395, 2025.
- [36] J. Taylor, A. Maitra, M. Alexander, D. Brooks, and M. Duvall, "Evaluation of the Impact of Plug-in Electric Vehicle Loading on Distribution System Operations," in *Proc. IEEE Power & Energy Soc. Gen. Meeting*, 2009.
- [37] A. Khan et al., "Understanding the Machine Learning Workflow: A Comprehensive Overview of Model Development, Validation, and Deployment," *Applied Computing and Informatics*, vol. 21, no. 3, pp. 455–470, 2024.
- [38] J. Xu et al., "Machine Learning Pipeline Design for Energy Forecasting Applications," *Energy Reports*, vol. 13, pp. 612–625, 2025.
- [39] Y. Li and A. Jenna, "Impact of Electric Vehicle Charging Demand on Power Distribution Grid Congestion," *Proc. Natl. Acad. Sci. (PNAS)*, vol. 121, no. 18, e2317599121, 2024.

- [40] M. B. Arias, M. Kim, and S. Bae, "Prediction of electric vehicle charging-power demand in realistic urban traffic networks," *Applied Energy*, vol. 195, pp. 738–753, 2017.
- [41] A. Khan et al., "Understanding the Machine Learning Workflow: A Comprehensive Overview of Model Development, Validation, and Deployment," *Applied Computing and Informatics*, vol. 21, no. 3, pp. 455–470, 2024.
- [42] J. Xu et al., "Machine Learning Pipeline Design for Energy Forecasting Applications," *Energy Reports*, vol. 13, pp. 612–625, 2025.
- [43] Varunsly, "GitHub - varunsly/Time_Series_Forecasting_SARIMA: Using SARIMA model for time series forecasting," GitHub. https://github.com/varunsly/Time_Series_Forecasting_SARIMA/tree/master
- [44] A. van den Oord et al., "WaveNet: A Generative Model for Raw Audio," DeepMind Technologies, 2016.
- [45] Y. LeCun, Y. Bengio, and G. Hinton, "Deep learning," *Nature*, vol. 521, pp. 436–444, 2015.
- [46] M. Kreft, T. Brudermueller, E. Fleisch, and T. Staake, "Predictability of electric vehicle charging: Explaining extensive user behavior-specific heterogeneity," *Applied Energy*, vol. 370, pp. 123544–123544, Jun. 2024, doi: <https://doi.org/10.1016/j.apenergy.2024.123544>
- [47] R. Zhao et al., "Time series classification using deep convolutional neural networks," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, pp. 2664–2669, 2017.
- [48] K. Lea et al., "Temporal convolutional networks for action segmentation and detection," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, pp. 156–165, 2017.
- [49] "MFHChade/Medium-Term-Load-Forecasting-using-TCN-LSTM-ARIMA," [Online]. Available: <https://github.com/MFHChade/Medium-Term-Load-Forecasting-using-TCN-LSTM-ARIMA/tree/main>
- [50] Y. Zhao, Z. Wang, Z.-J. M. Shen, and F. Sun, "Data-driven framework for large-scale prediction of charging energy in electric vehicles," *Applied Energy*, vol. 282, p. 116175, Jan. 2021, doi: 10.1016/j.apenergy.2020.116175.
- [51] J. Wang et al., "Hybrid deep learning model for air quality prediction," *Environmental Modelling & Software*, vol. 168, 2023.

- [52] J. Lee et al., "Spatiotemporal analysis of EV charging demand," *Energy Reports*, vol. 8, pp. 550–563, 2022.
- [53] M. Yilmaz and P. T. Krein, "Review of the impact of EV charging on power distribution," *IEEE Trans. Power Electron.*, vol. 28, no. 12, pp. 5673–5689, 2013.
- [54] A. Rahman et al., "Feature encoding for cyclical temporal data in machine learning," *Appl. Intell.*, vol. 53, pp. 2225–2240, 2023.
- [55] P. Chen, A. Niu, D. Liu, W. Jiang, and B. Ma, "Time Series Forecasting of Temperatures using SARIMA: An Example from Nanjing," *IOP Conf. Ser. Mater. Sci. Eng.*, vol. 394, p. 052024, Aug. 2018, doi: 10.1088/1757-899X/394/5/052024.
- [56] A. H. Alridha, F. H. A. Alsharify, and Z. Al-Khafaji, "A review of Optimization Techniques: Applications and Comparative analysis," *Iraqi Journal for Computer Science and Mathematics*. <https://ijcsm.researchcommons.org/ijcsm/vol5/iss2/5/>
- [57] F. T. Lima and V. M. A. Souza, "A Large Comparison of Normalization Methods on Time Series," *Big Data Research*, vol. 34, p. 100407, 2023, doi: 10.1016/j.bdr.2023.100407.

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

APPENDIX 1: TIMETABLE OF RESEARCH

UPDATES as of: 04/23/2026

	Sem 1 Lecture Weeks				Exams		Sem 1 Lecture Weeks				Exams		SEM BREAK				Sem 2 Lecture Weeks				Exams		Sem 2 Lecture Weeks												
	August			September			October			November				December		January				February				March		April				May					
1					D	D	D	D	D																										
2								D	D																										
3								D	D																										
4									D	D																									
5										D	D																								
6											D	D																							
7												D	D																						
8													D	D																					
9														D	D																				
10															D	D																			
11																D	D	D	D	D	D	D	D												
12																	D	D	D	D	D	D	D												
13a																		D	D	D	D	D	D	D											
13b																		D	D	D	D	D	D	D											
13c																		D	D	D	D	D	D	D											
13d																		D	D	D	D	D	D	D											
14a																									D	D	D	D	D	D	D	D	D		
14b																									D	D	D	D	D	D	D	D	D		
14c																									D	D	D	D	D	D	D	D	D		
14d																									D	D	D	D	D	D	D	D	D		
15a																										D	D	D	D	D	D	D	D		
15b																										D	D	D	D	D	D	D	D		
15c																										D	D	D	D	D	D	D	D		
15d																										D	D	D	D	D	D	D	D		
16																											D	D	D	D	D	D	D	D	

Legends

D means the activity is 'DONE' **O** means this activity is 'ON GOING' **W** means this activity 'WILL BE FINISHED IN THE FUTURE'

LIST OF ACTIVITIES					
1	Topic Selection	11	Revision According to Panel and Adviser	15a	Data Gathering: Testing (Running the SARIMA and TCN Model)
2	Revision of Topic According to Adviser/Consultation	12	Revision: Formatting (Chapters 1, 2, and 3)	15b	Data Gathering: Testing (Running the GA-TCN and PSO-TCN Model)
3	Finalization of Topic with Adviser Approval	13a	Implementation (Modeling SARIMA and TCN Model)	15c	Data Gathering: Testing (Running the Hybrid SARIMA-GA-TCN Model)
4	Writing of Chapter 1 with Revisions (Adviser Recommendations)	13b	Implementation (Modeling GA-TCN and PSO-TCN Model)	15d	Data Gathering: Testing (Running the Hybrid SARIMA-PSO-TCN Model)
5	Writing of Chapter 2 with Revisions (Adviser Recommendations)	13c	Implementation (Modeling Hybrid SARIMA-GA-TCN Model)	16	Submission of Paper
6	Writing of Chapter 3 with Revisions (Adviser Recommendations)	13d	Implementation (Modeling Hybrid SARIMA-PSO-TCN Model)	17	
7	Proof Reading and Submission of Draft Paper	14a	Optimization and Polishing (Running the SARIMA and TCN Model)	18	
8	Preparation of Defence PowerPoint and Script	14b	Optimization and Polishing (Running the GA-TCN and PSO-TCN Model)	19	
9	Submission of Proposal Defense Forms and PPT	14c	Optimization and Polishing (Running the Hybrid SARIMA-GA-TCN Model)	20	
10	Proposal Defence Week	14d	Optimization and Polishing (Running the Hybrid SARIMA-PSO-TCN Model)	21	

APPENDIX 2: OTHER NEEDED DOCUMENTS

Table B.1. Budgetary Requirements

Budgetary Requirements							
Start Date	End Date	Detail	Amount	Payment			
				LMJ	IDM	JBM	CPP
March 17, 2026	April 1, 2026	Expert Assistance	₱ 28,000	₱ 7,000	₱ 7,000	₱ 7,000	₱ 7,000
Total Amount:			₱ 28,000	₱ 7,000	₱ 7,000	₱ 7,000	₱ 7,000

APPENDIX 3: GRAMMARLY REPORT

Table C.1 Grammarly Report

General metrics 13,099 characters 1,823 words 107 sentences 7 min 17 sec reading time 14 min 1 sec speaking time Score 91 This text scores better than 91% of all texts checked by Grammarly Writing Issues 36 Issues left 16 Critical 20 Advanced	General metrics 50,834 characters 7,271 words 476 sentences 29 min 5 sec reading time 55 min 55 sec speaking time Score 82 This text scores better than 82% of all texts checked by Grammarly Writing Issues 237 Issues left 95 Critical 142 Advanced
Grammarly Report for CHAPTER 1.0	Grammarly Report for CHAPTER 2.0
General metrics 48,014 characters 6,849 words 470 sentences 27 min 23 sec reading time 52 min 41 sec speaking time Score 80 This text scores better than 80% of all texts checked by Grammarly Writing Issues 239 Issues left 99 Critical 140 Advanced	General metrics 50,552 characters 7,287 words 431 sentences 29 min 8 sec reading time 56 min 3 sec speaking time Score 82 This text scores better than 82% of all texts checked by Grammarly Writing Issues 239 Issues left 90 Critical 149 Advanced
Grammarly Report for CHAPTER 3.0	Grammarly Report for CHAPTER 4.0
General metrics 14,064 characters 1,937 words 79 sentences 7 min 44 sec reading time 14 min 54 sec speaking time Score 84 This text scores better than 84% of all texts checked by Grammarly Writing Issues 56 Issues left 18 Critical 38 Advanced	
Grammarly Report for CHAPTER 5.0	

APPENDIX 4: CODING SNIPPETS FOR FORECASTING**Appendix 4.1. : SARIMA Model Fitting**

Python

#SARIMA Model

```

sarima = SARIMAX(train, order=(2,1,2), seasonal_order=(1,1,1,24))
sarima_fit = sarima.fit(method='powell', maxiter=200, disp=False,
                        low_memory=True, conserve_memory=True)

sarima_pred = sarima_fit.forecast(steps=len(test))
residuals = test.flatten() - sarima_pred.flatten()

```

Appendix 4.2. : TCN Model Architecture

Python

#TCN Model

```

def build_tcn(filters, kernel, lr):
    model = Sequential()
    model.add(Conv1D(filters=int(filters), kernel_size=int(kernel), activation='relu',
input_shape=(window,1)))
    model.add(Flatten())
    model.add(Dense(1))
    model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=lr), loss='mse')
    return model

def fitness_eval(filters, kernel, lr):
    model = build_tcn(filters, kernel, lr)
    model.fit(X_res, y_res, epochs=3, batch_size=32, verbose=0)
    pred = model.predict(X_res, verbose=0)
    return mean_squared_error(y_res, pred)

```

Appendix 4.3. : Genetic Algorithm Hyperparameter Optimization

Python

#GA Hyperparameter Optimization

```

creator.create("FitnessMin", base.Fitness, weights=(-1.0,))
creator.create("Individual", list, fitness=creator.FitnessMin)

toolbox = base.Toolbox()
toolbox.register("filters", random.randint, 16, 128)

```

```

toolbox.register("kernel", random.randint, 2, 5)
toolbox.register("lr", random.uniform, 0.0001, 0.01)
toolbox.register("individual", tools.initCycle, creator.Individual, (toolbox.filters,
toolbox.kernel, toolbox.lr), n=1)
toolbox.register("population", tools.initRepeat, list, toolbox.individual)

def eval_individual(ind):
    return (fitness_eval(ind[0], ind[1], ind[2]),)

toolbox.register("evaluate", eval_individual)
toolbox.register("mate", tools.cxBlend, alpha=0.5)
toolbox.register("mutate", tools.mutGaussian, mu=0, sigma=1, indpb=0.2)
toolbox.register("select", tools.selTournament, tournsize=3)

pop = toolbox.population(n=5)
algorithms.eaSimple(pop, toolbox, cxpb=0.5, mutpb=0.2, ngen=5, verbose=False)

best_ind = tools.selBest(pop, k=1)[0]
print("GA Best Hyperparameters:", best_ind)

ga_model = build_tcn(best_ind[0], best_ind[1], best_ind[2])
ga_model.fit(X_res, y_res, epochs=10, verbose=0)

```

Appendix 4.4. : Particle Swarm Optimization Hyperparameter Optimization

```

Python
#PSO Hyperparameter Optimization
bounds = ([16,2,0.0001],[128,5,0.01])
optimizer = ps.single.GlobalBestPSO(n_particles=5, dimensions=3,
options={'c1':0.5, 'c2':0.3, 'w':0.9}, bounds=bounds)

best_cost_pso, best_pos_pso = optimizer.optimize(
    lambda x: np.array([fitness_eval(p[0], p[1], p[2]) for p in x]), iters=5)

print("PSO Best Hyperparameters:", best_pos_pso)
pso_model = build_tcn(best_pos_pso[0], best_pos_pso[1], best_pos_pso[2])
pso_model.fit(X_res, y_res, epochs=10, verbose=0)

```

Appendix 4.5. : Hybrid Forecast Combination

```

Python
#final forecasts
res_pso = pso_model.predict(X_res).flatten()
res_ga = ga_model.predict(X_res).flatten()

final_pso = sarima_pred[window:].flatten() + res_pso
final_ga = sarima_pred[window:].flatten() + res_ga
actual = test.flatten()[window:]

```

Appendix 4.6. : Evaluation Metric Computation

```

Python
#eval metrics
def smape(y_true, y_pred):
    return 100/len(y_true) *
    np.sum(2*np.abs(y_pred-y_true)/(np.abs(y_true)+np.abs(y_pred)))

def evaluate(name, y_true, y_pred):
    mae = mean_absolute_error(y_true, y_pred)
    rmse = np.sqrt(mean_squared_error(y_true, y_pred))
    smape_val = smape(y_true, y_pred)
    r2 = r2_score(y_true, y_pred)
    print(f"\n{name} RESULTS")
    print(f"MAE: {mae:.4f}")
    print(f"RMSE: {rmse:.4f}")
    print(f"SMAPE: {smape_val:.2f}%")
    print(f"R2: {r2:.4f}")
    return mae, rmse, smape_val, r2

metrics_pso = evaluate("SARIMA-PSO-TCN", actual, final_pso)
metrics_ga = evaluate("SARIMA-GA-TCN", actual, final_ga)

```

APPENDIX 5: HLN-DM STATISTICAL ANALYSIS WORKBOOK**Appendix 5.1. : SARIMA vs TCN**

timestamp	actual_kwh	SARIMA_forecast	TCN_forecast
2022-09-29 12:00:00	49.210976	84.335415	57.462955
2022-09-29 13:00:00	54.853132	81.603691	38.429146
2022-09-29 14:00:00	41.590216	75.755336	43.810440
2022-09-29 15:00:00	56.343823	70.900492	36.388969
2022-09-29 16:00:00	82.536137	69.482506	48.658745
2022-09-29 17:00:00	86.629955	75.092499	63.174938
2022-09-29 18:00:00	77.653041	78.624969	58.218319
2022-09-29 19:00:00	56.368035	67.537914	49.447365
2022-09-29 20:00:00	33.044405	45.871432	33.514145
2022-09-29 21:00:00	13.790426	29.288844	24.016407
2022-09-29 22:00:00	7.594433	17.110927	12.125226
2022-09-29 23:00:00	6.255403	12.538576	6.968786
2022-09-30 00:00:00	5.231439	10.748649	5.584329
2022-09-30 01:00:00	5.231439	9.132325	3.152203
2022-09-30 02:00:00	5.231439	8.360530	3.238757
2022-09-30 03:00:00	5.231439	8.119365	3.800333
2022-09-30 04:00:00	5.231439	8.145340	5.688307
2022-09-30 05:00:00	10.049917	9.583994	6.946698
2022-09-30 06:00:00	26.996217	17.614061	17.299286
2022-09-30 07:00:00	56.034472	36.199990	36.299797
2022-09-30 08:00:00	46.164888	49.451377	58.685875

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss_1	squared_loss_2	d_t = L1 - L2
2022-09-29 12:00:00	-35.124439	-8.251980	1233.726232	68.095171	1165.631060
2022-09-29 13:00:00	-26.750560	16.423986	715.592434	269.747320	445.845114
2022-09-29 14:00:00	-34.165120	-2.220224	1167.255416	4.929396	1162.326020
2022-09-29 15:00:00	-14.556669	19.954853	211.896618	398.196164	-186.299546
2022-09-29 16:00:00	13.053630	33.877392	170.397263	1147.677681	-977.280418
2022-09-29 17:00:00	11.537455	23.455016	133.112874	550.137795	-417.024921
2022-09-29 18:00:00	-0.971927	19.434723	0.944642	377.708440	-376.763798
2022-09-29 19:00:00	-11.169879	6.920670	124.766186	47.895676	76.870510
2022-09-29 20:00:00	-12.827027	-0.469739	164.532609	0.220655	164.311954
2022-09-29 21:00:00	-15.498418	-10.22598	240.200955	104.570690	135.630265
2022-09-29 22:00:00	-9.516494	-4.530793	90.563655	20.528087	70.035567
2022-09-29 23:00:00	-6.283173	-0.713383	39.478263	0.508915	38.969348
2022-09-30 00:00:00	-5.517211	-0.352890	30.439613	0.124531	30.315081
2022-09-30 01:00:00	-3.900886	2.079236	15.216910	4.323223	10.893687
2022-09-30 02:00:00	-3.129091	1.992682	9.791211	3.970781	5.820430
2022-09-30 03:00:00	-2.887926	1.431106	8.340119	2.048064	6.292056
2022-09-30 04:00:00	-2.913902	-0.456868	8.490822	0.208728	8.282094
2022-09-30 05:00:00	0.465923	3.103219	0.217084	9.629967	-9.412883
2022-09-30 06:00:00	9.382156	9.696931	88.024845	94.030476	-6.005631
2022-09-30 07:00:00	19.834482	19.734675	393.406686	389.457387	3.949298
2022-09-30 08:00:00	-3.286490	-12.52099	10.801014	156.775119	-145.974105

Appendix 5.2. : TCN vs GA-TCN

timestamp	actual_kwh	TCN_forecast	GA-TCN_forecast
2022-09-29 12:00:00	49.210976	57.462955	61.990646
2022-09-29 13:00:00	54.853132	38.429146	41.777206
2022-09-29 14:00:00	41.590216	43.810440	52.054855
2022-09-29 15:00:00	56.343823	36.388969	41.194576
2022-09-29 16:00:00	82.536137	48.658745	58.122673
2022-09-29 17:00:00	86.629955	63.174938	71.557510
2022-09-29 18:00:00	77.653041	58.218319	64.811470
2022-09-29 19:00:00	56.368035	49.447365	53.509117
2022-09-29 20:00:00	33.044405	33.514145	36.727882
2022-09-29 21:00:00	13.790426	24.016407	22.406570
2022-09-29 22:00:00	7.594433	12.125226	10.312359
2022-09-29 23:00:00	6.255403	6.968786	5.607942
2022-09-30 00:00:00	5.231439	5.584329	4.654098
2022-09-30 01:00:00	5.231439	3.152203	5.265955
2022-09-30 02:00:00	5.231439	3.238757	3.760767
2022-09-30 03:00:00	5.231439	3.800333	3.798259
2022-09-30 04:00:00	5.231439	5.688307	5.748342
2022-09-30 05:00:00	10.049917	6.946698	7.563222
2022-09-30 06:00:00	26.996217	17.299286	16.865055
2022-09-30 07:00:00	56.034472	36.299797	34.501526
2022-09-30 08:00:00	46.164888	58.685875	62.178688

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss _1	squared_loss _2	d_t = L1 - L2
2022-09-29 12:00:00	-8.251980	-12.779671	68.095171	163.319984	-95.224813
2022-09-29 13:00:00	16.423986	13.075926	269.747320	170.979828	98.767492
2022-09-29 14:00:00	-2.220224	-10.464640	4.929396	109.508682	-104.579286
2022-09-29 15:00:00	19.954853	15.149246	398.196164	229.499664	168.696501
2022-09-29 16:00:00	33.877392	24.413464	1147.677681	596.017208	551.660473
2022-09-29 17:00:00	23.455016	15.072444	550.137795	227.178575	322.959220
2022-09-29 18:00:00	19.434723	12.841571	377.708440	164.905957	212.802483
2022-09-29 19:00:00	6.920670	2.858918	47.895676	8.173411	39.722264
2022-09-29 20:00:00	-0.469739	-3.683477	0.220655	13.568002	-13.347347
2022-09-29 21:00:00	-10.22598 1	-8.616145	104.570690	74.237947	30.332743
2022-09-29 22:00:00	-4.530793	-2.717926	20.528087	7.387122	13.140965
2022-09-29 23:00:00	-0.713383	0.647461	0.508915	0.419206	0.089709
2022-09-30 00:00:00	-0.352890	0.577341	0.124531	0.333323	-0.208792
2022-09-30 01:00:00	2.079236	-0.034517	4.323223	0.001191	4.322032
2022-09-30 02:00:00	1.992682	1.470672	3.970781	2.162877	1.807904
2022-09-30 03:00:00	1.431106	1.433180	2.048064	2.054004	-0.005941
2022-09-30 04:00:00	-0.456868	-0.516903	0.208728	0.267188	-0.058460
2022-09-30 05:00:00	3.103219	2.486695	9.629967	6.183650	3.446317
2022-09-30 06:00:00	9.696931	10.131162	94.030476	102.640445	-8.609969
2022-09-30 07:00:00	19.734675	21.532946	389.457387	463.667760	-74.210373
2022-09-30 08:00:00	-12.52098 7	-16.013800	156.775119	256.441798	-99.666679

Appendix 5.3. : TCN vs PSO-TCN

timestamp	actual_kwh	TCN_forecast	PSO-TCN_forecast
2022-09-29 12:00:00	49.210976	57.462955	67.310188
2022-09-29 13:00:00	54.853132	38.429146	40.049889
2022-09-29 14:00:00	41.590216	43.810440	49.172150
2022-09-29 15:00:00	56.343823	36.388969	41.609974
2022-09-29 16:00:00	82.536137	48.658745	55.910892
2022-09-29 17:00:00	86.629955	63.174938	68.730766
2022-09-29 18:00:00	77.653041	58.218319	66.287292
2022-09-29 19:00:00	56.368035	49.447365	55.454117
2022-09-29 20:00:00	33.044405	33.514145	34.814606
2022-09-29 21:00:00	13.790426	24.016407	22.017439
2022-09-29 22:00:00	7.594433	12.125226	9.653413
2022-09-29 23:00:00	6.255403	6.968786	6.662847
2022-09-30 00:00:00	5.231439	5.584329	5.943464
2022-09-30 01:00:00	5.231439	3.152203	5.336954
2022-09-30 02:00:00	5.231439	3.238757	6.052881
2022-09-30 03:00:00	5.231439	3.800333	5.304204
2022-09-30 04:00:00	5.231439	5.688307	6.041340
2022-09-30 05:00:00	10.049917	6.946698	6.334083
2022-09-30 06:00:00	26.996217	17.299286	15.376759
2022-09-30 07:00:00	56.034472	36.299797	36.577663
2022-09-30 08:00:00	46.164888	58.685875	69.763725

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss_1	squared_loss_2	d_t = L1 - L2
2022-09-29 12:00:00	-8.251980	-18.099213	68.095171	327.581499	-259.486328
2022-09-29 13:00:00	16.423986	14.803243	269.747320	219.136013	50.611307
2022-09-29 14:00:00	-2.220224	-7.581934	4.929396	57.485722	-52.556326
2022-09-29 15:00:00	19.954853	14.733849	398.196164	217.086297	181.109868
2022-09-29 16:00:00	33.877392	26.625244	1147.677681	708.903629	438.774052
2022-09-29 17:00:00	23.455016	17.899188	550.137795	320.380943	229.756853
2022-09-29 18:00:00	19.434723	11.365749	377.708440	129.180250	248.528190
2022-09-29 19:00:00	6.920670	0.913918	47.895676	0.835246	47.060429
2022-09-29 20:00:00	-0.469739	-1.770200	0.220655	3.133609	-2.912954
2022-09-29 21:00:00	-10.225981	-8.227013	104.570690	67.683743	36.886947
2022-09-29 22:00:00	-4.530793	-2.058980	20.528087	4.239399	16.288688
2022-09-29 23:00:00	-0.713383	-0.407444	0.508915	0.166010	0.342905
2022-09-30 00:00:00	-0.352890	-0.712025	0.124531	0.506980	-0.382448
2022-09-30 01:00:00	2.079236	-0.105515	4.323223	0.011133	4.312090
2022-09-30 02:00:00	1.992682	-0.821442	3.970781	0.674768	3.296013
2022-09-30 03:00:00	1.431106	-0.072765	2.048064	0.005295	2.042769
2022-09-30 04:00:00	-0.456868	-0.809901	0.208728	0.655940	-0.447211
2022-09-30 05:00:00	3.103219	3.715834	9.629967	13.807426	-4.177458
2022-09-30 06:00:00	9.696931	11.619459	94.030476	135.011818	-40.981342
2022-09-30 07:00:00	19.734675	19.456808	389.457387	378.567392	10.889995
2022-09-30 08:00:00	-12.520987	-23.598837	156.775119	556.905130	-400.130011

Appendix 5.4. : GA-TCN vs SARIMA-GA-TCN

timestamp	actual_kwh	GA-TCN_forecast	SARIMA-GA-TCN_forecast
2022-09-29 12:00:00	49.210976	61.990646	71.598244
2022-09-29 13:00:00	54.853132	41.777206	42.772682
2022-09-29 14:00:00	41.590216	52.054855	56.466876
2022-09-29 15:00:00	56.343823	41.194576	43.718159
2022-09-29 16:00:00	82.536137	58.122673	65.524938
2022-09-29 17:00:00	86.629955	71.557510	89.996462
2022-09-29 18:00:00	77.653041	64.811470	87.891228
2022-09-29 19:00:00	56.368035	53.509117	66.399765
2022-09-29 20:00:00	33.044405	36.727882	32.970371
2022-09-29 21:00:00	13.790426	22.406570	19.162957
2022-09-29 22:00:00	7.594433	10.312359	4.233107
2022-09-29 23:00:00	6.255403	5.607942	7.300462
2022-09-30 00:00:00	5.231439	4.654098	5.883617
2022-09-30 01:00:00	5.231439	5.265955	5.169201
2022-09-30 02:00:00	5.231439	3.760767	6.298534
2022-09-30 03:00:00	5.231439	3.798259	9.766784
2022-09-30 04:00:00	5.231439	5.748342	8.906267
2022-09-30 05:00:00	10.049917	7.563222	5.655901
2022-09-30 06:00:00	26.996217	16.865055	18.945598
2022-09-30 07:00:00	56.034472	34.501526	43.798153
2022-09-30 08:00:00	46.164888	62.178688	66.388700

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss_1	squared_loss_2	d_t = L1 - L2
2022-09-29 12:00:00	-12.779671	-22.387268	163.319984	501.189777	-337.869792
2022-09-29 13:00:00	13.075926	12.080450	170.979828	145.937267	25.042562
2022-09-29 14:00:00	-10.464640	-14.876660	109.508682	221.315013	-111.806331
2022-09-29 15:00:00	15.149246	12.625664	229.499664	159.407381	70.092283
2022-09-29 16:00:00	24.413464	17.011199	596.017208	289.380894	306.636314
2022-09-29 17:00:00	15.072444	-3.366507	227.178575	11.333372	215.845204
2022-09-29 18:00:00	12.841571	-10.238187	164.905957	104.820468	60.085489
2022-09-29 19:00:00	2.858918	-10.031730	8.173411	100.635605	-92.462194
2022-09-29 20:00:00	-3.683477	0.074035	13.568002	0.005481	13.562521
2022-09-29 21:00:00	-8.616145	-5.372531	74.237947	28.864091	45.373856
2022-09-29 22:00:00	-2.717926	3.361326	7.387122	11.298512	-3.911390
2022-09-29 23:00:00	0.647461	-1.045059	0.419206	1.092149	-0.672943
2022-09-30 00:00:00	0.577341	-0.652178	0.333323	0.425336	-0.092013
2022-09-30 01:00:00	-0.034517	0.062237	0.001191	0.003874	-0.002682
2022-09-30 02:00:00	1.470672	-1.067095	2.162877	1.138692	1.024185
2022-09-30 03:00:00	1.433180	-4.535345	2.054004	20.569359	-18.515354
2022-09-30 04:00:00	-0.516903	-3.674828	0.267188	13.504363	-13.237175
2022-09-30 05:00:00	2.486695	4.394016	6.183650	19.307378	-13.123728
2022-09-30 06:00:00	10.131162	8.050619	102.640445	64.812474	37.827972
2022-09-30 07:00:00	21.532946	12.236319	463.667760	149.727506	313.940254
2022-09-30 08:00:00	-16.013800	-20.223812	256.441798	409.002561	-152.560763

Appendix 5.5. : PSO-TCN vs SARIMA-PSO-TCN

timestamp	actual_kwh	PSO-TCN_forecast	SARIMA-PSO-TCN_forecast
2022-09-29 12:00:00	49.210976	67.310188	68.181435
2022-09-29 13:00:00	54.853132	40.049889	45.088944
2022-09-29 14:00:00	41.590216	49.172150	58.067783
2022-09-29 15:00:00	56.343823	41.609974	44.907420
2022-09-29 16:00:00	82.536137	55.910892	66.968718
2022-09-29 17:00:00	86.629955	68.730766	88.216929
2022-09-29 18:00:00	77.653041	66.287292	83.148439
2022-09-29 19:00:00	56.368035	55.454117	58.811981
2022-09-29 20:00:00	33.044405	34.814606	31.235633
2022-09-29 21:00:00	13.790426	22.017439	18.279379
2022-09-29 22:00:00	7.594433	9.653413	5.389494
2022-09-29 23:00:00	6.255403	6.662847	6.369204
2022-09-30 00:00:00	5.231439	5.943464	5.245664
2022-09-30 01:00:00	5.231439	5.336954	5.588007
2022-09-30 02:00:00	5.231439	6.052881	6.494133
2022-09-30 03:00:00	5.231439	5.304204	12.250702
2022-09-30 04:00:00	5.231439	6.041340	7.171559
2022-09-30 05:00:00	10.049917	6.334083	2.087757
2022-09-30 06:00:00	26.996217	15.376759	18.387943
2022-09-30 07:00:00	56.034472	36.577663	39.848972
2022-09-30 08:00:00	46.164888	69.763725	61.376557

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss_1	squared_loss_2	d_t = L1 - L2
2022-09-29 12:00:00	-18.099213	-18.970460	327.581499	359.878343	-32.296843
2022-09-29 13:00:00	14.803243	9.764188	219.136013	95.339362	123.796651
2022-09-29 14:00:00	-7.581934	-16.477567	57.485722	271.510211	-214.024489
2022-09-29 15:00:00	14.733849	11.436403	217.086297	130.791314	86.294983
2022-09-29 16:00:00	26.625244	15.567419	708.903629	242.344520	466.559109
2022-09-29 17:00:00	17.899188	-1.586974	320.380943	2.518488	317.862455
2022-09-29 18:00:00	11.365749	-5.495398	129.180250	30.199399	98.980851
2022-09-29 19:00:00	0.913918	-2.443946	0.835246	5.972871	-5.137624
2022-09-29 20:00:00	-1.770200	1.808773	3.133609	3.271660	-0.138051
2022-09-29 21:00:00	-8.227013	-4.488953	67.683743	20.150697	47.533046
2022-09-29 22:00:00	-2.058980	2.204939	4.239399	4.861755	-0.622356
2022-09-29 23:00:00	-0.407444	-0.113801	0.166010	0.012951	0.153060
2022-09-30 00:00:00	-0.712025	-0.014225	0.506980	0.000202	0.506777
2022-09-30 01:00:00	-0.105515	-0.356568	0.011133	0.127141	-0.116007
2022-09-30 02:00:00	-0.821442	-1.262694	0.674768	1.594396	-0.919629
2022-09-30 03:00:00	-0.072765	-7.019263	0.005295	49.270057	-49.264763
2022-09-30 04:00:00	-0.809901	-1.940121	0.655940	3.764068	-3.108128
2022-09-30 05:00:00	3.715834	7.962160	13.807426	63.395994	-49.588568
2022-09-30 06:00:00	11.619459	8.608274	135.011818	74.102381	60.909437
2022-09-30 07:00:00	19.456808	16.185500	378.567392	261.970397	116.596996
2022-09-30 08:00:00	-23.598837	-15.211669	556.905130	231.394869	325.510261

Appendix 5.6. : SARIMA-GA-TCN vs SARIMA-PSO-TCN

timestamp	actual_kwh	SARIMA-GA-TCN_forec ast	SARIMA-PSO-TCN_forec ast
2022-09-29 12:00:00	49.210976	71.598244	68.181435
2022-09-29 13:00:00	54.853132	42.772682	45.088944
2022-09-29 14:00:00	41.590216	56.466876	58.067783
2022-09-29 15:00:00	56.343823	43.718159	44.907420
2022-09-29 16:00:00	82.536137	65.524938	66.968718
2022-09-29 17:00:00	86.629955	89.996462	88.216929
2022-09-29 18:00:00	77.653041	87.891228	83.148439
2022-09-29 19:00:00	56.368035	66.399765	58.811981
2022-09-29 20:00:00	33.044405	32.970371	31.235633
2022-09-29 21:00:00	13.790426	19.162957	18.279379
2022-09-29 22:00:00	7.594433	4.233107	5.389494
2022-09-29 23:00:00	6.255403	7.300462	6.369204
2022-09-30 00:00:00	5.231439	5.883617	5.245664
2022-09-30 01:00:00	5.231439	5.169201	5.588007
2022-09-30 02:00:00	5.231439	6.298534	6.494133
2022-09-30 03:00:00	5.231439	9.766784	12.250702
2022-09-30 04:00:00	5.231439	8.906267	7.171559
2022-09-30 05:00:00	10.049917	5.655901	2.087757
2022-09-30 06:00:00	26.996217	18.945598	18.387943
2022-09-30 07:00:00	56.034472	43.798153	39.848972
2022-09-30 08:00:00	46.164888	66.388700	61.376557

UNIVERSITY OF SANTO TOMAS ELECTRICAL ENGINEERING DEPARTMENT

timestamp	error_1	error_2	squared_loss_1	squared_loss_2	d_t = L1 - L2
2022-09-29 12:00:00	-22.387268	-18.970460	501.189777	359.878343	141.311434
2022-09-29 13:00:00	12.080450	9.764188	145.937267	95.339362	50.597904
2022-09-29 14:00:00	-14.876660	-16.477567	221.315013	271.510211	-50.195198
2022-09-29 15:00:00	12.625664	11.436403	159.407381	130.791314	28.616067
2022-09-29 16:00:00	17.011199	15.567419	289.380894	242.344520	47.036374
2022-09-29 17:00:00	-3.366507	-1.586974	11.333372	2.518488	8.814884
2022-09-29 18:00:00	-10.238187	-5.495398	104.820468	30.199399	74.621069
2022-09-29 19:00:00	-10.031730	-2.443946	100.635605	5.972871	94.662734
2022-09-29 20:00:00	0.074035	1.808773	0.005481	3.271660	-3.266178
2022-09-29 21:00:00	-5.372531	-4.488953	28.864091	20.150697	8.713394
2022-09-29 22:00:00	3.361326	2.204939	11.298512	4.861755	6.436757
2022-09-29 23:00:00	-1.045059	-0.113801	1.092149	0.012951	1.079198
2022-09-30 00:00:00	-0.652178	-0.014225	0.425336	0.000202	0.425134
2022-09-30 01:00:00	0.062237	-0.356568	0.003874	0.127141	-0.123267
2022-09-30 02:00:00	-1.067095	-1.262694	1.138692	1.594396	-0.455705
2022-09-30 03:00:00	-4.535345	-7.019263	20.569359	49.270057	-28.700699
2022-09-30 04:00:00	-3.674828	-1.940121	13.504363	3.764068	9.740295
2022-09-30 05:00:00	4.394016	7.962160	19.307378	63.395994	-44.088616
2022-09-30 06:00:00	8.050619	8.608274	64.812474	74.102381	-9.289907
2022-09-30 07:00:00	12.236319	16.185500	149.727506	261.970397	-112.242891
2022-09-30 08:00:00	-20.223812	-15.211669	409.002561	231.394869	177.607692

CURRICULUM VITAE

PERSONAL INFORMATION

Last Name: Javier
First Name: Leister James
Middle Name: Mondido
Student Number: 2022167692
E-mail Address: leisterjames.javier.eng@ust.edu.ph
Contact Number: 09469725137
Current Address: G/F, One Torre de Santo Tomas,
860 M.F. Jhocson, corner España Blvd,
Sampaloc, Manila, 1015 Metro Manila
Permanent Address: Purok 2, Brgy. Buboy
Nagcarlan, Laguna
Date of Birth: January 5, 2004



EDUCATIONAL BACKGROUND

Tertiary:	University of Santo Tomas (UST-Manila) Bachelor of Science in Electrical Engineering (BSEE) 2022-present
Secondary:	Laguna College San Pablo City, Laguna 2020 -2022 Saint Mary's Academy of Nagcarlan Nagcarlan, Laguna 2016 -2020
Primary:	Nagcarlan Montessori Center Nagcarlan, Laguna 2010-2016

CURRICULUM VITAE

PERSONAL INFORMATION

Last Name: Marantan

First Name: Ivan Nickol

Middle Name: De vera

Student Number: 2022178132

E-mail Address: ivannickol.marantan.eng@ust.edu.ph

Contact Number: 09338594703

Current Address: 1228 Alfredo, Sampaloc, Metro Manila

Permanent Address: Tabacuhan, Santa Rita, Olongapo

Date of Birth: September 26, 2004



EDUCATIONAL BACKGROUND

Tertiary: University of Santo Tomas (UST-Manila)
Bachelor of Science in Electrical Engineering (BSEE)
2022-present

Secondary: Regional Science High School III
East Kalayaan, Subic Bay, Freeport Zone, Olongapo
2016-2022.

Primary: Olongapo City Christian School
20th St, Olongapo, Zambales
2010-2016

CURRICULUM VITAE

PERSONAL INFORMATION

Last Name: Matias
First Name: Justine
Middle Name: Belmonte
Student Number: 2021-150269
E-mail Address: justine.matias.eng @ust.edu.ph
Contact Number: 09260189602
Current Address: Studio 42, San Anton St.,
Barangay 397, Sampaloc,
Metro Manila
Permanent Address: Bagumbong, Caloocan City
Date of Birth: November 25, 2002



EDUCATIONAL BACKGROUND

Tertiary:	University of Santo Tomas (UST-Manila) Bachelor of Science in Electrical Engineering (BSEE) 2021 - present
Secondary:	Pamantasan ng Lungsod ng Valenzuela Maysan Rd, Valenzuela City 2019 - 2021 Bignay National High School Bignay, Valenzuela City 2015 - 2019
Primary:	Roberta De Jesus Elementary School Bignay, Valenzuela City 2009 - 2015

CURRICULUM VITAE

PERSONAL INFORMATION

Last Name: Pineda

First Name: Carlos Louis

Middle Name: Pasion

Student Number: 2022-177618

E-mail Address: carloslouis.pineda.eng@ust.edu.ph

Contact Number: 09925378651

Current Address: University Tower 4, P. Noval, 1425,
Barangay 397, Sampaloc,
Metro Manila

Permanent Address: San Antonio, Bacolor, Pampanga

Date of Birth: June 16, 2004



EDUCATIONAL BACKGROUND

Tertiary: University of Santo Tomas (UST-Manila)
Bachelor of Science in Electrical Engineering (BSEE)
2022-present

Secondary: Pampanga High School
City of San Fernando, Pampanga
2020-2022

Don Honorio Ventura State University
Cabambangan, Bacolor, Pampanga
2016-2020

Primary: San Antonio Elementary School (P)
San Antonio, Bacolor, Pampanga
2010-2016